

Tesina per il corso di Advanced Cybersecurity for IT

A Zero Trust Architecture for HR and Enterprise Operations

Enforcing least-privilege principles across a multi-tier organizational ecosystem.

Redatto da:

Antonio Di Placido

Pietro Salvatore

Daniel Diocis

Nicola Sebastianelli

Daniele Giuliani

Sotto la supervisione del **Prof. Luca Spalazzi**



Università Politecnica delle Marche

Dipartimento di Ingegneria dell'Informazione

Corso di Laurea Magistrale in Ingegneria Informatica e dell'Automazione

Tesina di:

Advanced Cybersecurity for IT

Esame per il corso da 9 CFU erogato da Luca Spalazzi durante l'anno accademico 2025/2026

Redazione del documento a cura di:

- **Antonio Di Placido** (Matr. 1126477) – s1126477@studenti.univpm.it
- **Pietro Salvatore** (Matr. 1125572) – s1125572@studenti.univpm.it
- **Daniel Diocis** (Matr. 1125569) – s1125569@studenti.univpm.it
- **Nicola Sebastianelli** (Matr. 1122744) – s1122744@studenti.univpm.it
- **Daniele Giuliani** (Matr. 1121883) – s1121883@studenti.univpm.it

Indice

I Introduzione al progetto 6
1.1 Il contesto attuale . . . 6
1.2 Perché la scelta Zero Trust? . . . 6
1.3 Descrizione del progetto . . . 7
1.3.1 Tipologie di utenti e modello d'accesso . . . 8
1.3.2 Risorse protette . . . 8
1.3.3 Dinamiche di interazione e Continuous Verification . . . 9

II Architettura generale del sistema 10
2.1 Visione d'insieme . . . 10
2.2 Suddivisione logica dell'architettura . . . 10
2.3 Componenti principali . . . 11
2.4 Ruolo di Envoy come punto di enforcement . . . 11
2.5 Ruolo di OPA come motore decisionale . . . 12
2.6 Gestione dell'identità utente e del dispositivo . . . 12
2.7 Flusso generale di una richiesta di accesso . . . 13
2.8 Architettura di rete containerizzata . . . 13
2.9 Separazione tra autenticazione, autorizzazione e monitoraggio . . . 14
2.10 Osservabilità e valutazione del rischio . . . 14
2.11 Applicazione dei principi Zero Trust . . . 14
2.12 Considerazioni sull'architettura . . . 14

III Strumenti e tecnologie utilizzate 15
3.1 Introduzione . . . 15
3.2 Docker e Docker Compose . . . 16
3.3 Node.js, TypeScript ed Express . . . 17
3.4 React, Vite e TypeScript . . . 18
3.5 MongoDB . . . 19
3.6 JSON Web Token . . . 20
3.7 Envoy Proxy . . . 21
3.8 mTLS e certificati X.509 . . . 21
3.9 CA Service . . . 22
3.10 OPA . . . 22
3.11 Rego . . . 22
3.12 Splunk . . . 23
3.13 Splunk AI Toolkit e modello Random Forest . . . 24
3.14 Snort . . . 24
3.15 nftables . . . 24
3.16 Kali Linux . . . 25
3.17 Postman . . . 25
3.18 TPM e Secure Enclave . . . 26
3.19 Variabili d'ambiente e gestione delle configurazioni . . . 26
3.20 Sintesi degli strumenti utilizzati . . . 26
3.21 Considerazioni finali . . . 27

IV Implementazione applicativa e flussi principali 28
4.1 Introduzione . . . 28
4.2 Struttura generale del backend . . . 28
4.3 Rotte principali del backend . . . 29
4.4 Autenticazione dell'utente . . . 30
4.5 Middleware di verifica del JWT . . . 30
4.6 Frontend e interazione con l'utente . . . 31
4.7 Risorse protette del caso studio . . . 31
4.8 Catena di protezione delle risorse . . . 32
4.9 Envoy come punto di ingresso sicuro . . . 32
4.10 Comunicazione sicura tramite mTLS . . . 32
4.11 Processo di device enrollment . . . 33
4.12 Stati del dispositivo . . . 34
4.13 Approvazione del dispositivo e rilascio del certificato . . . 34
4.14 CA Service . . . 35

4.15 Identità hardware-bound tramite TPM e Secure Enclave . . . 35
4.16 Revoca e sospensione dei dispositivi . . . 38
4.17 Sincronizzazione dei dispositivi fidati . . . 39
4.18 Persistenza dei dati con MongoDB . . . 39
4.19 Seeder e dati dimostrativi . . . 39
4.20 Endpoint di valutazione applicativa dell'accesso . . . 40
4.21 Logging applicativo . . . 41
4.22 Flusso complessivo implementato . . . 41
4.23 Sintesi dei flussi del capitolo . . . 42
4.24 Considerazioni finali . . . 42

V Policy, autorizzazione e valutazione del rischio 43
5.1 Introduzione . . . 43
5.2 OPA come Policy Decision Point . . . 43
5.3 Policy Enforcement Point e Policy Decision Point . . . 44
5.4 Struttura delle policy Rego . . . 44
5.5 Decisione finale di accesso . . . 45
5.6 Controlli sull'utente . . . 45
5.7 Controlli sul dispositivo . . . 46
5.8 Controlli sul contesto di rete . . . 46
5.9 Motivi di negazione . . . 46
5.10 Esempio di risposta decisionale . . . 47
5.11 Valutazione del rischio . . . 47
5.12 Risk score e soglie di accesso . . . 48
5.13 Endpoint di valutazione del rischio . . . 48
5.14 Integrazione tra OPA e servizio di rischio . . . 49
5.15 Fallback prudenziale . . . 49
5.16 Ruolo di Splunk nella valutazione del rischio . . . 50
5.17 Splunk AI Toolkit e Random Forest . . . 50
5.18 Audit log e osservabilità . . . 50
5.19 Logica BARAC . . . 51
5.20 Webhook Splunk e blocco adattivo . . . 51
5.21 Blocklist degli utenti . . . 52
5.22 Aggiornamento dinamico delle decisioni . . . 52
5.23 Confronto tra RBAC, ABAC e approccio adottato . . . 53
5.24 Esempi di scenari decisionali . . . 53
5.24.1 Accesso consentito . . . 53
5.24.2 Accesso negato per ruolo non autorizzato . . . 53
5.24.3 Accesso negato per dispositivo non fidato . . . 53
5.24.4 Accesso negato per rischio elevato . . . 54
5.24.5 Accesso negato per blocklist . . . 54
5.25 Sintesi dei controlli decisionali . . . 54
5.26 Considerazioni finali . . . 54

VI Integrazione SIEM e Machine Learning 55
6.1 Introduzione . . . 55
6.2 Architettura e Workflow Analitico di Splunk . . . 55
6.3 Il Modello di Machine Learning: Random Forest . . . 56
6.4 Il Calcolo Combinato del Rischio . . . 57
6.5 Validazione e Test dell'Architettura . . . 58
6.5.1 Fase 1: Addestramento del Modello Predittivo . . . 58
6.5.2 Fase 2: Intervento Dinamico in Tempo Reale . . . 58

VII WSL2 e Configurazione nftables 60
7.1 Introduzione . . . 60
7.2 Configurazione del sistema host (WSL2) . . . 60
7.2.1 Abilitazione IP forwarding . . . 60
7.3 Installazione e configurazione di nftables . . . 61
7.3.1 Installazione . . . 61
7.3.2 Avvio e verifica . . . 61
7.4 Struttura delle regole firewall . . . 61
7.4.1 Policy di base . . . 61

7.4.2 Gestione forwarding Docker . . . 61
7.5 Test di sicurezza con Kali Linux . . . 61
7.5.1 Test di scansione . . . 62
7.5.2 Osservazione del forwarding . . . 62
7.6 Problemi riscontrati . . . 62
7.7 Conclusioni . . . 62

VIII Test 63
8.1 Introduzione . . . 63
8.2 Test tramite Postman della logica di accesso . . . 63
8.2.1 Test di autenticazione . . . 63
8.2.2 Test di verifica del JWT . . . 64
8.2.3 Test di accesso a una risorsa protetta senza certificato client . . . 65
8.2.4 Test della logica di autorizzazione basata su attributi . . . 65
8.2.5 Test di validazione dell'input . . . 66
8.2.6 Sintesi dei risultati . . . 67
8.3 Test enrollment di un dispositivo . . . 68
8.3.1 Secure Enclave . . . 68
8.3.2 TPM . . . 71
8.3.3 Software Fallback . . . 73
8.4 Test di rilevamento degli attacchi di rete (IDS e mTLS) . . . 74
8.4.1 Test di Port Scanning e rilevamento tramite IDS . . . 74
8.4.2 Test di attacco Brute Force su endpoint mTLS . . . 74
8.5 Test e configurazione di Splunk . . . 75
8.5.1 Fase 1: Generazione dei dati e Addestramento del Modello . . . 75
8.5.2 Fase 2: Blocco Dinamico Predittivo in Tempo Reale . . . 76

IX Conclusioni 78
9.1 Sintesi del progetto realizzato . . . 78
9.2 Livelli reali di privacy e sicurezza raggiungibili . . . 79
9.3 Limiti del prototipo . . . 79
9.4 Tecnologie emergenti e sviluppi futuri . . . 80
9.5 Considerazioni finali . . . 80



Introduzione al progetto

1.1 Il contesto attuale

Negli ultimi anni, le organizzazioni — siano esse pubbliche amministrazioni o imprese private — hanno visto dissolversi il tradizionale perimetro che separava nettamente l'interno dall'esterno. La progressiva adozione di servizi cloud, la diffusione del lavoro da remoto, l'impiego di dispositivi personali (*Bring Your Own Device*) e l'interconnessione con fornitori e terze parti hanno dato vita a un ecosistema digitale privo di confini rigidi, in cui dati, applicazioni e utenti risiedono e si muovono ben oltre i locali fisici dell'organizzazione.

In questo scenario, il modello di sicurezza storicamente dominante — fondato sull'idea di un perimetro fortificato, in cui tutto ciò che si trova all'interno della rete aziendale è implicitamente considerato affidabile e tutto ciò che è esterno potenzialmente ostile — non risulta più adeguato. Tale assunzione, infatti, trascura la possibilità che una minaccia possa già trovarsi *all'interno* del perimetro: un attaccante che abbia ottenuto un primo accesso, un dipendente infedele o, più semplicemente, un account legittimo le cui credenziali siano state compromesse.

Una volta superata la barriera perimetrale, l'attaccante può muoversi in un ambiente che tende a considerarlo fidato, sfruttando credenziali valide, privilegi eccessivi o configurazioni poco restrittive per raggiungere risorse critiche. Il problema non riguarda quindi soltanto la protezione dall'esterno, ma anche la capacità di controllare in modo continuo ciò che avviene all'interno dell'infrastruttura.

È evidente, dunque, che in un contesto di questo tipo la protezione di dati sensibili — per loro natura soggetti a requisiti stringenti di riservatezza, integrità e conformità normativa — si configura come una sfida più complessa rispetto al passato. Garantire che ad ogni richiesta di accesso corrisponda una verifica esplicita e contestuale dell'identità, del dispositivo e dei privilegi, anziché affidarsi a una fiducia concessa *a priori* sulla base della posizione di rete, diventa pertanto un requisito fondamentale. Proprio da questa esigenza nasce il presente progetto.

1.2 Perché la scelta Zero Trust?

L'architettura **Zero Trust** nasce per rispondere alle sfide poste dalla dissoluzione del perimetro tradizionale. Il suo principio cardine è tanto semplice nella formulazione quanto radicale nelle conseguenze: ***Never Trust, Always Verify***. In una rete Zero Trust viene meno l'idea di una fiducia implicita: la posizione logica o fisica dell'utente — il fatto di trovarsi "dentro" la rete aziendale o di essere connesso a un segmento considerato sicuro — non conferisce di per sé alcun diritto d'accesso.

Ogni richiesta deve invece essere **autenticata**, **autorizzata** e valutata sulla base di parametri **dinamici**, rivalutati di volta in volta in funzione del contesto. Tra questi rientrano, ad esempio, l'identità dell'utente, il ruolo, il dispositivo utilizzato, lo stato del dispositivo, la risorsa richiesta, l'azione da eseguire, la zona di rete e il livello di rischio associato alla richiesta.

Il cambio di prospettiva è sostanziale. Si abbandona un modello di **sicurezza statica**, fondato sulla difesa di un confine e sulla distinzione netta tra interno affidabile ed esterno ostile, per adottare una **sicurezza centrata sull'identità e sul contesto**, in cui ogni interazione è trattata come potenzialmente non fidata finché non ne viene dimostrata la legittimità.

Applicare questo paradigma a un sistema di gestione di risorse umane e dati enterprise rappresenta una scelta particolarmente significativa. In questo ambito, infatti, convergono dati personali, informazioni aziendali sensibili e processi critici per l'organizzazione. L'obiettivo non è soltanto impedire accessi esterni non autorizzati, ma anche limitare il rischio derivante da credenziali compromesse, dispositivi non affidabili, accessi da zone di rete non consentite o azioni non coerenti con il ruolo dell'utente.

I pilastri strategici del progetto

L'adozione del modello Zero Trust per il presente lavoro si fonda su tre esigenze principali, che costituiscono i pilastri strategici del progetto.

- **Micro-segmentazione.** L'infrastruttura viene organizzata in componenti separati e logicamente distinti, in modo da ridurre la possibilità che la compromissione di una singola componente si propaghi all'intero sistema. Nel progetto questa idea si riflette nella separazione tra i diversi servizi containerizzati, tra le API applicative, il proxy Envoy, il motore di policy OPA, il database, il servizio di certificazione e gli strumenti di monitoraggio. In questo modo, l'accesso a una risorsa non dipende dalla semplice presenza nella rete, ma dal superamento di controlli specifici.
- **Principio del minimo privilegio (PoLP).** A ciascun attore del sistema vengono concessi esclusivamente i privilegi necessari allo svolgimento delle proprie funzioni. Il controllo non si basa soltanto sul ruolo dell'utente, ma anche sull'azione richiesta e sulla risorsa a cui si vuole accedere. Questo consente di ridurre la superficie di attacco e di contenere l'impatto potenziale di un account compromesso o di un uso improprio dei permessi.
- **Verifica continua e adattiva.** L'accesso al sistema non viene considerato come un evento unico e definitivo. Ogni richiesta viene valutata nel proprio contesto, tenendo conto dell'identità dell'utente, dello stato del dispositivo, della rete di provenienza, della risorsa richiesta e di eventuali segnali di rischio. La fiducia diventa quindi una condizione dinamica, che deve essere riconfermata nel tempo e che può essere revocata quando il contesto non soddisfa più i requisiti previsti.

Considerati congiuntamente, questi tre pilastri si rafforzano a vicenda: la micro-segmentazione limita la propagazione di un'eventuale compromissione, il minimo privilegio riduce l'esposizione delle risorse e la verifica continua consente di adattare la decisione di accesso al contesto della singola richiesta. È dalla loro combinazione che deriva la robustezza complessiva dell'architettura proposta.

1.3 Descrizione del progetto

Per tradurre i principi teorici finora esposti in una soluzione applicativa concreta, il presente lavoro analizza il caso studio di **ZeroTrustHR**, un'organizzazione fittizia che rappresenta una realtà aziendale moderna, caratterizzata da ruoli differenziati, risorse con diversi livelli di sensibilità e necessità di controllare in modo puntuale l'accesso ai dati.

ZeroTrustHR gestisce alcune risorse rappresentative di un contesto aziendale e HR. Nel prototipo sono considerate, in particolare, risorse pubbliche o a bassa sensibilità, come `public_products`, risorse interne aziendali, come `company_policies`, e risorse sensibili, come `employee_records`. Questa scelta permette di verificare il comportamento delle policy di accesso in scenari differenti, senza introdurre un numero eccessivo di risorse non necessarie alla dimostrazione del modello.

La scelta di focalizzarsi su un'organizzazione HR non è casuale. Il settore della gestione del personale rappresenta un contesto in cui il conflitto tra sicurezza e usabilità è particolarmente evidente. Da un lato, figure come amministratori e responsabili HR devono poter accedere a informazioni sensibili per svolgere le proprie attività; dall'altro, tali accessi devono essere limitati, tracciati e subordinati a controlli rigorosi.

Un amministratore di sistema, pur disponendo di un livello di privilegio elevato, non dovrebbe poter eseguire qualunque operazione da una rete non autorizzata o da un dispositivo non registrato. Analogamente, un utente esterno o un dipendente standard non dovrebbe poter accedere a dati riservati, anche nel caso in cui riesca ad autenticarsi con credenziali valide. Il progetto nasce quindi con l'obiettivo di dimostrare come un'architettura Zero Trust possa ridurre questi rischi attraverso una valutazione contestuale e multi-attributo di ogni richiesta.

Dal punto di vista implementativo, ZeroTrustHR è stato realizzato come prototipo containerizzato. L'architettura comprende un frontend sviluppato in React, un backend Node.js con TypeScript, un database MongoDB, un proxy Envoy con supporto a mTLS, il motore di policy OPA, un servizio di certificazione per la gestione dei certificati client e componenti di monitoraggio e analisi come Splunk e Snort. Il sistema integra inoltre meccanismi di identità del dispositivo basati su certificati client e, dove disponibile, su tecnologie hardware-bound come TPM e Secure Enclave.

1.3.1 Tipologie di utenti e modello d'accesso

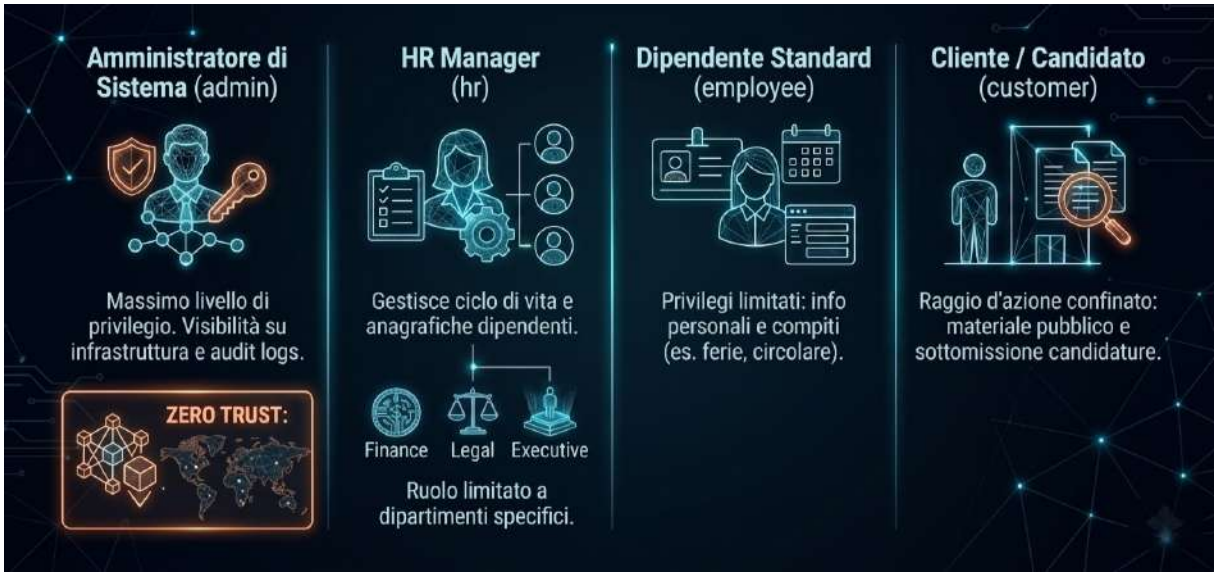


Figura 1: Schema del modello ibrido d'accesso (RBAC + ABAC)

L'organizzazione adotta un modello ibrido che combina il controllo degli accessi basato sui ruoli (**RBAC**) con quello basato sugli attributi (**ABAC**). Il ruolo dell'utente rappresenta un primo elemento di valutazione, ma non è sufficiente da solo a determinare l'esito della richiesta. La decisione finale dipende anche da attributi contestuali, come dispositivo, stato del dispositivo, risorsa, azione, rete e rischio.

Nel modello sono previste diverse figure logiche:

- **Amministratore di sistema (admin):** rappresenta il profilo con il livello di privilegio più elevato all'interno del sistema. Può accedere a funzionalità amministrative e a risorse sensibili, ma rimane comunque soggetto a restrizioni basate sul contesto, come la zona di rete, il dispositivo utilizzato e l'azione richiesta.
- **HR Manager (hr):** rappresenta l'utente responsabile della gestione di informazioni relative al personale. Può accedere a risorse HR, ma soltanto nei limiti previsti dalle policy e in presenza di condizioni coerenti con il livello di sensibilità della risorsa.
- **Dipendente standard (employee):** dispone di privilegi limitati, orientati alla consultazione delle proprie informazioni e all'interazione con funzionalità operative di base. Non può accedere a risorse riservate o a dati relativi ad altri utenti se non espressamente autorizzato.
- **Cliente o candidato (customer):** rappresenta un utente esterno, il cui raggio d'azione è limitato a risorse pubbliche o a funzionalità non sensibili, come la consultazione di informazioni generali o la sottomissione di candidature.

Questa suddivisione consente di applicare il principio del minimo privilegio, evitando che il possesso di un ruolo generico comporti automaticamente accesso indiscriminato alle risorse. Ogni richiesta viene infatti valutata rispetto alla specifica combinazione di ruolo, risorsa e azione.

1.3.2 Risorse protette

Nel progetto le risorse non vengono trattate tutte allo stesso modo, ma sono distinte in base al loro livello di sensibilità e al tipo di utenti che possono accedervi. Questa scelta consente di applicare policy differenti a seconda della risorsa richiesta, dell'azione da eseguire e del contesto della richiesta.

Le principali risorse considerate nel prototipo sono:

- `public_products`**:** rappresenta una risorsa pubblica o a bassa sensibilità, accessibile anche a utenti esterni o con privilegi limitati. È pensata per modellare contenuti che non richiedono un livello elevato di protezione, come informazioni generali o materiale consultabile senza accesso a dati riservati.

- `company_policies`: rappresenta una risorsa aziendale interna. Contiene informazioni utili all'organizzazione, come regolamenti, procedure o documentazione interna. Pur non avendo la stessa criticità dei dati personali dei dipendenti, richiede comunque controlli di accesso, poiché non è necessariamente destinata a qualunque utente esterno.

- `employee_records`: rappresenta una risorsa sensibile, relativa ai dati dei dipendenti. Per questo motivo è soggetta a policy più restrittive. L'accesso è consentito solo in presenza di condizioni coerenti con la policy, come ruolo autorizzato, dispositivo trusted, stato del dispositivo attivo, azione consentita e zona di rete ammessa.

Questa suddivisione permette di applicare concretamente il principio del minimo privilegio. Un utente può essere autorizzato ad accedere a una risorsa pubblica o interna, ma non necessariamente a dati più sensibili come quelli relativi ai dipendenti. Allo stesso modo, anche un utente con ruolo elevato, come l'amministratore, non ottiene automaticamente qualunque permesso: l'azione richiesta e il contesto della richiesta devono comunque rispettare le condizioni previste dalla policy.

Nel prototipo, la verifica multifattore non è implementata come flusso completo di autenticazione MFA. Essa viene rappresentata tramite l'attributo contestuale `mfa_verified`, utilizzato dalle policy e dai test per simulare la presenza o l'assenza di una verifica multifattore. In questo modo è possibile valutare come il sistema reagisca a richieste in cui tale requisito logico risulti soddisfatto o meno, senza introdurre un vero provider MFA.

La distinzione tra le risorse consente quindi di mostrare un comportamento coerente con il paradigma Zero Trust: l'accesso non viene deciso in modo generico, ma sulla base della specifica combinazione tra utente, dispositivo, risorsa, azione e contesto.

1.3.3 Dinamiche di interazione e Continuous Verification

Il valore aggiunto dell'approccio adottato risiede nella natura dinamica e persistente delle autorizzazioni. A differenza dei modelli tradizionali, in cui la fiducia viene spesso concessa al momento dell'autenticazione e mantenuta per l'intera sessione, nel modello Zero Trust ogni singola richiesta di accesso viene sottoposta a una valutazione autonoma.

È questo il principio della **Continuous Verification**: la postura di sicurezza non è un dato acquisito, ma una condizione che deve essere continuamente riconfermata. Il login permette di identificare l'utente, ma non garantisce automaticamente l'accesso a tutte le risorse. Ogni richiesta viene valutata considerando il contesto specifico in cui avviene.

Tale paradigma si fonda su un'architettura multi-livello, nella quale la decisione di consentire o negare un accesso non dipende mai da un unico fattore, ma dalla convergenza di più dimensioni di controllo valutate simultaneamente. Da un lato si collocano le verifiche relative all'identità e al dispositivo, che accertano la legittimità dell'utente, lo stato del dispositivo e la sua associazione con l'identità dichiarata. Dall'altro lato intervengono le regole di autorizzazione, che stabiliscono se quello specifico ruolo può compiere una determinata azione sulla risorsa richiesta.

A questi elementi si aggiunge una valutazione dinamica del rischio, integrata nel progetto attraverso componenti di monitoraggio e analisi. Splunk, Snort e il backend contribuiscono alla raccolta e all'elaborazione di eventi utili per stimare il livello di rischio associato a utenti, richieste o comportamenti anomali. Nel prototipo, tale componente ha lo scopo di mostrare come le decisioni di accesso possano essere arricchite da segnali provenienti dall'ambiente operativo, pur rimanendo all'interno di un contesto sperimentale e controllato.

La forza del modello deriva dalla logica congiunta con cui questi fattori vengono combinati. Il venir meno anche di un solo requisito rilevante può determinare il diniego dell'accesso. Una credenziale valida, da sola, non è sufficiente; allo stesso modo, un dispositivo trusted non garantisce l'accesso se la zona di rete non è consentita o se l'azione richiesta non è prevista dalla policy.

Infine, ogni decisione di autorizzazione o diniego viene tracciata, conservando le informazioni principali che hanno determinato l'esito della richiesta. Questa registrazione costituisce un supporto per l'analisi degli eventi di sicurezza, per l'osservabilità del sistema e per un eventuale affinamento successivo delle policy.



Architettura generale del sistema

2.1 Visione d'insieme

L'architettura realizzata per il progetto **ZeroTrustHR** è stata progettata come un sistema distribuito e containerizzato, nel quale ogni richiesta di accesso viene valutata dinamicamente prima di raggiungere le risorse applicative. L'obiettivo non è semplicemente autenticare un utente al momento del login, ma verificare continuamente che l'identità, il dispositivo, il contesto di rete e il livello di rischio siano coerenti con la risorsa richiesta.

Il sistema implementa quindi una logica coerente con il paradigma *Zero Trust*, secondo cui nessun soggetto viene considerato automaticamente affidabile solo perché interno alla rete aziendale o perché già autenticato. Ogni accesso deve essere valutato singolarmente, tenendo conto di più fattori:

- identità dell'utente;
- ruolo e dipartimento di appartenenza;
- risorsa richiesta;
- azione da eseguire sulla risorsa;
- stato e affidabilità del dispositivo;
- presenza di autenticazione forte;
- posizione o zona di rete;
- punteggio di rischio associato alla richiesta.

Dal punto di vista architetturale, il sistema è composto da più servizi indipendenti, eseguiti tramite **Docker Compose** all'interno di una rete dedicata. Questa scelta permette di separare logicamente i componenti, simulando una micro-architettura aziendale nella quale frontend, backend, database, motore di policy, proxy, SIEM e servizi di sicurezza cooperano per applicare le decisioni di accesso.

2.2 Suddivisione logica dell'architettura

L'architettura può essere suddivisa in quattro aree principali:

- **Presentation Layer**, rappresentato dal frontend React/Vite, attraverso cui l'utente interagisce con il sistema;
- **Application Layer**, costituito dal backend Node.js/TypeScript, che espone le API applicative, gestisce autenticazione, dispositivi, richieste di accesso, risorse protette e log;
- **Security Control Plane**, formato da Envoy, OPA, CA Service e dai meccanismi di valutazione del rischio;

- **Data, Observability e Risk Layer**, composto da MongoDB, Splunk, Snort e nftables, utilizzati rispettivamente per la persistenza dei dati, la raccolta e l'analisi degli eventi, il rilevamento di traffico sospetto, il controllo del traffico e il supporto alla valutazione dinamica del rischio.

Questa suddivisione riflette una separazione tra il piano applicativo e il piano di sicurezza. Le API del backend non vengono considerate automaticamente affidabili: prima di consentire l'accesso a una risorsa protetta, la richiesta attraversa componenti dedicati alla verifica dell'identità, del dispositivo e delle policy.

2.3 Componenti principali

I principali componenti del sistema sono riportati nella Tabella 1.

Componente	Ruolo nell'architettura
Frontend React/Vite	Interfaccia utente del sistema. Permette di effettuare login, interagire con le funzionalità applicative, accedere alle risorse protette e supportare il flusso di device enrollment, inclusa la generazione di identità software e il download dei materiali necessari al dispositivo.
Backend Node.js/TypeScript	Espone le API REST, gestisce autenticazione, utenti, dispositivi, risorse, richieste di accesso, logging e integrazione con OPA, Splunk e CA Service.
MongoDB	Database utilizzato per memorizzare utenti, dispositivi, certificati, policy, richieste di accesso e dati applicativi del caso studio HR.
Envoy Proxy	Componente posto davanti alle risorse protette. Agisce come *Policy Enforcement Point* e intercetta le richieste, applicando mTLS e delegando la decisione autorizzativa a OPA tramite `ext_authz`.
OPA	Motore di policy utilizzato come *Policy Decision Point*. Valuta le regole scritte in Rego e restituisce una decisione di tipo *allow* o *deny*.
CA Service	Servizio interno incaricato di firmare le richieste di certificato dei dispositivi, senza esporre direttamente la chiave privata della CA al backend.
Splunk	Sistema SIEM utilizzato per raccogliere e analizzare log, eventi di sicurezza e decisioni di accesso. Nella versione definitiva del progetto contribuisce anche alla valutazione dinamica del rischio, integrando saved searches, lookup, webhook e un modello di Machine Learning per stimare la probabilità di comportamento anomalo.
Snort	IDS impiegato per osservare il traffico che attraversa Envoy e generare alert in formato JSON relativi a possibili comportamenti sospetti. Gli eventi prodotti possono contribuire alla componente di rischio di rete.
nftables	Componente firewall a livello L3/L4 utilizzato per il controllo e il logging del traffico di rete.
Kali Linux	Container utilizzato come ambiente di test offensivo e simulazione di traffico ostile, collocato su una rete separata rispetto ai servizi principali.

Tabella 1: Componenti principali dell'architettura ZeroTrustHR.

2.4 Ruolo di Envoy come punto di enforcement

Uno degli elementi centrali dell'architettura è **Envoy Proxy**. Nel progetto, Envoy svolge il ruolo di *Policy Enforcement Point* (PEP), cioè il punto in cui la decisione di accesso viene effettivamente applicata.

Le richieste dirette alle risorse protette non vengono considerate valide solo perché provenienti dal frontend o da un utente autenticato. Prima di essere inoltrate al backend, esse passano attraverso Envoy, che:

- riceve la richiesta in ingresso;
- verifica la presenza e la validità del certificato client tramite mTLS;
- aggiunge informazioni contestuali alla richiesta, come IP del client e dettagli del certificato;
- invia la richiesta a OPA tramite il filtro `ext_authz`;
- consente o blocca il traffico in base alla decisione ricevuta.

In questo modo, il backend non è l'unico responsabile della sicurezza. L'accesso viene controllato già a livello di proxy, prima che la richiesta raggiunga la logica applicativa. Questo rispecchia un principio importante delle architetture Zero Trust: spostare il controllo il più vicino possibile al punto di accesso alla risorsa.

2.5 Ruolo di OPA come motore decisionale

OPA, acronimo di *Open Policy Agent*, rappresenta il componente incaricato di prendere le decisioni autorizzative. Nel progetto, OPA riceve da Envoy una richiesta strutturata contenente gli attributi dell'accesso e valuta le policy scritte in linguaggio **Rego**.

Le regole sono state organizzate in più file, separando logicamente i controlli relativi a:

- utente e ruolo;
- dispositivo;
- rischio;
- risorsa richiesta;
- funzioni di supporto per estrarre header, token e attributi.

La decisione finale viene prodotta solo se tutte le condizioni previste risultano soddisfatte. In caso contrario, OPA restituisce una decisione negativa insieme ai motivi del rifiuto, ad esempio ruolo non autorizzato, dispositivo non fidato, MFA assente, rischio troppo elevato o accesso fuori dall'orario consentito.

Questa separazione tra codice applicativo e policy rappresenta un vantaggio importante: le regole di sicurezza non sono disperse all'interno del backend, ma sono definite in modo dichiarativo e centralizzato. Ciò rende il sistema più controllabile, più estendibile e più vicino a un modello reale di governance della sicurezza.

2.6 Gestione dell'identità utente e del dispositivo

Nel progetto l'identità non è limitata al solo utente. Un accesso viene considerato valido solo se è possibile stabilire anche l'affidabilità del dispositivo da cui la richiesta proviene.

Il sistema utilizza quindi due livelli principali di identità:

- **identità utente**, gestita tramite login e token JWT;
- **identità dispositivo**, gestita tramite certificati, mTLS e informazioni contestuali come sistema operativo, stato del dispositivo, indirizzo IP e fingerprint JA3.

Questa distinzione è fondamentale in una logica Zero Trust. Un utente con credenziali corrette potrebbe comunque tentare l'accesso da un dispositivo non registrato, compromesso o non conforme. Per questo motivo, il progetto prevede un processo di *device enrollment*, attraverso il quale un dispositivo può essere registrato e associato a un certificato client.

Il certificato consente al sistema di verificare che la richiesta provenga da un dispositivo riconosciuto. Per rafforzare ulteriormente questo legame, il progetto include agenti locali per macOS e Windows: il primo integra meccanismi basati sulla **Secure Enclave**, mentre il secondo utilizza il **TPM**. In questo modo, l'identità del dispositivo non dipende soltanto da un certificato software, ma viene collegata più strettamente all'hardware fisico, riducendo il rischio che una chiave privata venga esportata o riutilizzata su un altro dispositivo.

2.7 Flusso generale di una richiesta di accesso

Il funzionamento dell'architettura può essere riassunto attraverso il seguente flusso:

1. l'utente effettua il login tramite il frontend;
2. il backend verifica le credenziali e rilascia un token JWT;
3. l'utente richiede l'accesso a una risorsa protetta;
4. la richiesta attraversa Envoy, che verifica il certificato client tramite mTLS;
5. Envoy invia a OPA gli attributi della richiesta;
6. OPA valuta le policy Rego sulla base di ruolo, azione, risorsa, dispositivo e rischio;
7. se la decisione è positiva, Envoy inoltra la richiesta al backend;
8. il backend restituisce la risorsa richiesta;
9. l'evento viene registrato e inviato a Splunk per finalità di audit e monitoraggio.

Il flusso può essere rappresentato in forma sintetica come segue:

Utente / Frontend → Envoy PEP → OPA PDP → Backend → MongoDB
Log e decisioni → Splunk / SIEM

Questa sequenza evidenzia che il backend non viene esposto come unico punto di controllo. La decisione di accesso è il risultato della collaborazione tra più componenti, ciascuno con una responsabilità specifica.

2.8 Architettura di rete containerizzata

I principali servizi del progetto sono eseguiti tramite Docker Compose e organizzati all'interno di reti dedicate. La rete principale è denominata `zerotrust_net` e ospita i componenti fondamentali dell'architettura, come backend, frontend, MongoDB, OPA, Envoy, Splunk e CA Service.

Nel file di orchestrazione sono definiti diversi servizi, tra cui:

- `mongodb`, per la persistenza dei dati;
- `backend`, per le API applicative;
- `frontend`, per l'interfaccia utente;
- `opa`, per la valutazione delle policy;
- `envoy`, per l'enforcement delle richieste;
- `splunk`, per la raccolta e l'analisi degli eventi;
- `ca`, per la gestione della firma dei certificati;
- `snort`, per il rilevamento di traffico sospetto;
- `nftables`, per il controllo e il logging del traffico a livello di rete;
- `kali`, per la simulazione di traffico esterno o potenzialmente ostile.

La containerizzazione consente di simulare un ambiente aziendale composto da più servizi indipendenti, mantenendo al tempo stesso una configurazione riproducibile. La rete `zerotrust_net` permette ai servizi principali di comunicare tra loro in modo controllato, mentre la rete `wan_net` viene utilizzata per rappresentare un ambiente esterno, ad esempio tramite il container Kali Linux.

Un aspetto rilevante della configurazione riguarda Snort, che viene eseguito utilizzando il `network_mode` del servizio Envoy. In questo modo, Snort può osservare il traffico che attraversa il proxy, cioè uno dei punti più critici dell'architettura. Il servizio nftables, invece, viene eseguito con privilegi di rete specifici e con `network_mode: host`, così da poter simulare controlli e logging a livello L3/L4.

La presenza di *health check*, volumi persistenti e logging centralizzato verso Splunk migliora l'affidabilità del prototipo durante le fasi di test. In particolare, i dati di MongoDB e Splunk vengono mantenuti tramite volumi dedicati, mentre i log dei servizi vengono inviati a Splunk tramite il driver di logging Docker o tramite meccanismi specifici dei singoli componenti.

2.9 Separazione tra autenticazione, autorizzazione e monitoraggio

Un aspetto rilevante del progetto è la separazione tra tre concetti spesso confusi tra loro:

- **Autenticazione**: verifica chi è l'utente, ad esempio tramite credenziali e JWT;
- **Autorizzazione**: decide se l'utente, in quel contesto, può accedere a una specifica risorsa;
- **Monitoraggio**: registra gli eventi e analizza il comportamento nel tempo.

Nel sistema ZeroTrustHR, il login non implica automaticamente l'autorizzazione. Un utente può essere autenticato correttamente ma ricevere comunque un rifiuto se il suo ruolo non è compatibile con la risorsa, se il dispositivo non è fidato, se il livello di rischio è troppo alto o se la richiesta viene effettuata in condizioni non conformi.

Questa distinzione rappresenta uno dei punti cardine del progetto: la sicurezza non dipende da un unico controllo iniziale, ma da una valutazione continua e contestuale.

2.10 Osservabilità e valutazione del rischio

Il progetto integra una componente di osservabilità tramite **Splunk**. I log generati dai servizi, dalle decisioni di accesso, da Envoy, da OPA, da nftables e dagli alert di Snort vengono raccolti per permettere attività di audit, monitoraggio e analisi del comportamento.

Nella versione definitiva del progetto, Splunk non svolge soltanto una funzione passiva di raccolta log, ma contribuisce anche alla valutazione dinamica del rischio. In particolare, il sistema utilizza eventi applicativi e di rete per stimare la probabilità che una richiesta o un utente presentino un comportamento anomalo.

La logica di rischio considera due componenti principali:

- una componente applicativa, derivata dagli eventi di accesso e dalle decisioni prodotte dal sistema;
- una componente di rete, alimentata anche dagli alert generati da Snort.

Il rischio complessivo viene poi messo in relazione con l'impatto della risorsa richiesta. Risorse più sensibili, come dati HR o finanziari, hanno un impatto maggiore rispetto a risorse pubbliche o a bassa criticità. In questo modo, la decisione di accesso non dipende soltanto dal comportamento dell'utente, ma anche dal valore della risorsa e dal tipo di azione richiesta.

Splunk supporta inoltre una logica adattiva di tipo BARAC, basata sul confronto tra rischio e beneficio. Attraverso saved searches, lookup e webhook, il sistema può individuare utenti con comportamento sospetto e aggiornare dinamicamente le informazioni utilizzate dal motore di policy.

Questa impostazione consente di avvicinare il prototipo a un modello di *Continuous Verification*, nel quale il comportamento passato e gli eventi osservati contribuiscono alla valutazione delle richieste future.

2.11 Applicazione dei principi Zero Trust

L'architettura realizzata applica diversi principi fondamentali del modello Zero Trust.

In primo luogo, viene applicato il principio del **Least Privilege**: ogni utente può accedere solo alle risorse coerenti con il proprio ruolo, il proprio dipartimento e le condizioni definite dalle policy.

In secondo luogo, il sistema applica una logica di **verifica continua**: l'autenticazione iniziale non è sufficiente, poiché ogni richiesta viene valutata nuovamente in base al contesto.

Un ulteriore principio applicato è quello della **riduzione della fiducia implicita**. La rete interna non viene considerata automaticamente sicura, e anche dispositivi e utenti interni devono dimostrare di soddisfare le condizioni richieste.

Infine, l'architettura introduce elementi di **osservabilità e risposta adattiva**, poiché gli eventi vengono registrati e analizzati per supportare decisioni più dinamiche e consapevoli.

2.12 Considerazioni sull'architettura

L'architettura proposta non ha l'obiettivo di rappresentare un sistema enterprise completo e pronto per la produzione, ma costituisce un prototipo realistico per dimostrare l'applicazione concreta dei principi Zero Trust in un contesto HR.

La scelta di utilizzare componenti separati permette di evidenziare in modo chiaro le responsabilità di ciascun modulo. Envoy applica le decisioni, OPA le calcola, il backend gestisce la logica applicativa, MongoDB conserva i dati, Splunk raccoglie e analizza gli eventi contribuendo anche al calcolo dinamico del rischio, mentre il CA Service supporta la gestione dell'identità dei dispositivi.

Nel complesso, il progetto mostra come un'architettura Zero Trust non sia basata su un singolo strumento, ma sulla cooperazione di più livelli di sicurezza: identità, dispositivo, rete, policy, rischio e monitoraggio.



Strumenti e tecnologie utilizzate

3.1 Introduzione

La realizzazione del progetto **ZeroTrustHR** ha richiesto l'integrazione di diversi strumenti, ciascuno con un ruolo specifico all'interno dell'architettura. La scelta delle tecnologie non è stata casuale, ma è stata guidata dall'obiettivo di simulare un ambiente aziendale moderno, distribuito e coerente con i principi della *Zero Trust Architecture*.

Il sistema non si basa su un singolo meccanismo di protezione, ma su una combinazione di componenti applicativi, infrastrutturali e di sicurezza. In particolare, sono stati utilizzati strumenti per:

- sviluppare il frontend e il backend dell'applicazione;
- gestire la persistenza dei dati;
- orchestrare i servizi tramite container;
- applicare controlli di accesso centralizzati;
- gestire policy dichiarative;
- proteggere le comunicazioni tramite certificati e mTLS;
- raccogliere, indicizzare e analizzare eventi di sicurezza;
- monitorare il traffico di rete e rilevare comportamenti sospetti;
- simulare traffico esterno o ostile durante i test;
- supportare la valutazione dinamica del rischio.

Nei paragrafi successivi vengono descritti gli strumenti principali utilizzati nel progetto, evidenziando non solo la loro funzione generale, ma soprattutto il modo in cui sono stati integrati all'interno del sistema ZeroTrustHR.

3.2 Docker e Docker Compose

Nel contesto della moderna ingegneria del software e della sicurezza informatica, la containerizzazione rappresenta un paradigma fondamentale per garantire l'isolamento, la portabilità e la riproducibilità delle applicazioni.



Figura 2: Logo ufficiale di Docker

Docker è una piattaforma open-source che consente di pacchettizzare un'applicazione e tutte le sue dipendenze (librerie, file di configurazione, runtime) all'interno di un'unità leggera e isolata chiamata *container*. A differenza delle macchine virtuali tradizionali, che richiedono un intero sistema operativo guest, i container condividono il kernel del sistema operativo ospite (*host*). L'isolamento è garantito a livello logico grazie a funzionalità native del kernel Linux, come i *namespaces* (che isolano la vista del sistema operativo per ogni container) e i *control groups* o *cgroups* (che limitano l'allocazione delle risorse hardware). Dal punto di vista della cybersecurity, questo approccio riduce drasticamente la superficie di attacco complessiva, poiché la compromissione di un singolo servizio non si estende immediatamente all'intero sistema ospite.



Figura 3: Logo ufficiale di Docker Compose

Quando l'architettura di un sistema è composta da molteplici componenti indipendenti (microservizi), la gestione dei singoli container può diventare complessa. Per ovviare a questo problema si utilizza **Docker Compose**, uno strumento di orchestrazione semplificata che permette di definire e coordinare applicazioni multi-container. Tramite un unico file di configurazione in formato YAML, Docker Compose consente di descrivere l'intera infrastruttura (servizi, reti, volumi e variabili d'ambiente), abilitando il deployment dell'intero ecosistema con un singolo comando.

Docker è stato utilizzato per containerizzare i principali componenti del progetto, mentre **Docker Compose** ha permesso di orchestrare l'intera infrastruttura attraverso un unico file di configurazione.

Nel progetto, Docker Compose definisce i diversi servizi necessari al funzionamento del sistema, tra cui:

- `mongodb`, per la persistenza dei dati;

- `backend`, per le API applicative;
- `frontend`, per l'interfaccia utente;
- `opa`, per la valutazione delle policy;
- `envoy`, per l'enforcement delle richieste;
- `splunk`, per la raccolta, l'indicizzazione e l'analisi degli eventi;
- `ca`, per la firma dei certificati;
- `snort`, per il rilevamento di traffico sospetto;
- `nftables`, per il controllo e il logging del traffico a livello di rete;
- `kali`, per la simulazione di traffico esterno o ostile durante i test.

L'uso di Docker ha permesso di isolare i servizi e di rendere l'ambiente più facilmente riproducibile. Ogni componente viene eseguito in un container separato, con proprie dipendenze e configurazioni. Questo approccio risulta particolarmente adatto a un progetto Zero Trust, poiché consente di rappresentare in modo realistico un'infrastruttura composta da servizi indipendenti.

Un altro aspetto importante è la presenza di reti Docker dedicate. La rete principale, denominata `zerotrust_net`, consente ai servizi applicativi e di sicurezza di comunicare tra loro in modo controllato. Accanto a essa è prevista anche una rete separata, `wan_net`, utilizzata per rappresentare un ambiente esterno rispetto ai servizi principali, ad esempio tramite il container Kali Linux. Sono inoltre presenti volumi persistenti per conservare dati MongoDB e dati Splunk anche dopo il riavvio dei container.

3.3 Node.js, TypeScript ed Express



(a) Node.js (b) TypeScript (c) Express

Figura 4: Stack tecnologico utilizzato per lo sviluppo del backend.

Il backend del progetto è stato sviluppato utilizzando **Node.js** con **TypeScript**. Node.js consente di realizzare applicazioni server-side basate su JavaScript, mentre TypeScript aggiunge un sistema di tipizzazione statica che migliora la leggibilità, la manutenibilità e la robustezza del codice.

Nel progetto ZeroTrustHR, il backend espone API REST per diverse funzionalità:

- autenticazione degli utenti;
- gestione delle richieste di accesso;
- gestione dei dispositivi registrati;
- enrollment dei dispositivi;
- accesso alle risorse protette;
- calcolo e aggiornamento del rischio;
- ricezione di webhook da Splunk;
- invio di log e audit event.

Il framework **Express** è stato utilizzato per definire le rotte HTTP e organizzare la logica applicativa. La struttura del backend è suddivisa in rotte, controller, middleware, servizi, modelli e tipi. Questa separazione rende il codice più ordinato e permette di distinguere chiaramente le responsabilità dei diversi moduli.

Dal punto di vista della sicurezza, il backend non si limita a gestire le normali funzionalità applicative, ma collabora con altri componenti dell'architettura. In particolare, interagisce con MongoDB per recuperare dati e configurazioni, con Splunk per l'invio dei log, con OPA per la sincronizzazione di alcune informazioni relative ai dispositivi e con il CA Service per il processo di firma dei certificati.

3.4 React, Vite e TypeScript



(a) React (b) Vite (c) Typescript

Figura 5: Stack tecnologico utilizzato per lo sviluppo del frontend.

Il frontend è stato realizzato utilizzando **React**, **Vite** e **TypeScript**. React è una libreria JavaScript per la costruzione di interfacce utente basate su componenti, mentre Vite è uno strumento moderno per lo sviluppo frontend che offre tempi di avvio e compilazione molto rapidi.

Nel progetto, il frontend rappresenta il punto di interazione tra l'utente e il sistema. Attraverso l'interfaccia è possibile eseguire operazioni come:

- login dell'utente;
- visualizzazione delle funzionalità disponibili;
- richiesta di accesso a risorse protette;
- interazione con il backend;
- avvio e supporto del processo di device enrollment;
- generazione di un'identità software di fallback;
- download dei materiali necessari alla configurazione del dispositivo.

Nel frontend sono presenti componenti e moduli dedicati al flusso di enrollment, come la pagina `DeviceEnrollment` e le funzioni di supporto per la comunicazione con le API di enrollment. Questo permette di integrare nell'interfaccia utente una parte del processo di registrazione del dispositivo, senza demandare l'intera procedura a richieste manuali.

Il frontend non contiene la logica decisionale di sicurezza. Questa scelta è importante: in un'architettura Zero Trust, il client non deve essere considerato affidabile. L'interfaccia può raccogliere informazioni e inviare richieste, ma la decisione finale deve essere presa lato server o da componenti dedicati, come OPA ed Envoy.

Vite è stato inoltre configurato per dialogare con il backend e con il percorso dedicato alle risorse protette. In questo modo, il frontend può interagire con i servizi applicativi mantenendo separati i flussi ordinari da quelli che richiedono enforcement Zero Trust.

3.5 MongoDB



Figura 6: Logo ufficiale di mongoDB

MongoDB è il database utilizzato per la persistenza dei dati del progetto. Si tratta di un database NoSQL orientato ai documenti, nel quale le informazioni vengono memorizzate in formato simile a JSON.

Nel contesto di ZeroTrustHR, MongoDB viene utilizzato per conservare diverse categorie di dati:

- utenti e relative informazioni di autenticazione;
- dispositivi registrati;
- policy e configurazioni applicative;
- dati relativi alle risorse HR;
- informazioni utili alla valutazione delle richieste;
- dati di supporto per i flussi di accesso.

La scelta di MongoDB risulta adatta a un progetto di questo tipo perché permette di gestire strutture dati flessibili. In un sistema Zero Trust, infatti, gli attributi da valutare possono essere numerosi e variare nel tempo: ruolo, dipartimento, stato del dispositivo, livello di rischio, risorsa richiesta e altre informazioni contestuali.

All'interno dell'ambiente Docker, MongoDB viene eseguito come servizio separato e protetto tramite credenziali definite nelle variabili d'ambiente. Il database è inoltre associato a un volume persistente, in modo da non perdere i dati al riavvio del container.

3.6 JSON Web Token



Figura 7: Logo ufficiale di JSON Web Token

Per la gestione dell'autenticazione utente è stato utilizzato il meccanismo dei **JSON Web Token** (JWT). Un JWT è un token firmato che permette di rappresentare in modo compatto alcune informazioni relative all'identità dell'utente autenticato.

Nel progetto, dopo il login, il backend verifica le credenziali dell'utente e rilascia un token. Tale token viene poi utilizzato nelle richieste successive per dimostrare che l'utente ha effettuato correttamente l'autenticazione.

È importante sottolineare che, nel progetto ZeroTrustHR, il JWT non è sufficiente da solo per ottenere l'accesso alle risorse protette. Il token permette di identificare l'utente, ma l'autorizzazione viene valutata separatamente sulla base di ulteriori fattori, tra cui:

- ruolo dell'utente;
- dipartimento;
- azione richiesta;
- risorsa richiesta;
- presenza di MFA;
- stato del dispositivo;
- livello di rischio.

Questa distinzione è coerente con il modello Zero Trust: autenticare un utente non significa concedergli automaticamente qualsiasi accesso.

3.7 Envoy Proxy

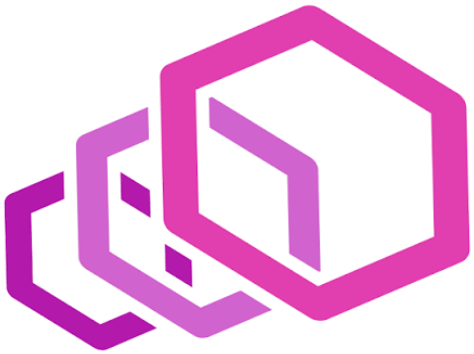


Figura 8: Logo ufficiale di Envoy Proxy

Envoy Proxy rappresenta uno dei componenti centrali dell'architettura. Nel progetto viene utilizzato come punto di ingresso per le risorse protette e svolge il ruolo di *Policy Enforcement Point*.

Il compito di Envoy è intercettare le richieste prima che raggiungano il backend e verificare se esse possono essere inoltrate. In particolare, Envoy:

- riceve le richieste dirette alle risorse protette;
- gestisce il listener HTTPS con mTLS;
- richiede il certificato client;
- estrae informazioni dal certificato e dalla richiesta;
- invia la richiesta a OPA tramite il filtro `ext_authz`;
- applica la decisione restituita dal motore di policy;
- produce log strutturati degli accessi.

Nel file di configurazione di Envoy sono presenti due listener principali. Il primo è dedicato alle richieste protette e richiede il certificato client. Il secondo è dedicato al processo di enrollment, consentendo l'accesso solo ad alcune rotte specifiche, come quelle per la generazione della challenge e per la registrazione del dispositivo.

Envoy è particolarmente importante perché impedisce che il backend venga raggiunto direttamente senza passare dai controlli di sicurezza. In questo modo, la decisione autorizzativa non è lasciata esclusivamente alla logica applicativa, ma viene applicata a monte, nel punto di ingresso dell'architettura.

3.8 mTLS e certificati X.509

Per rafforzare l'identità del dispositivo e proteggere le comunicazioni, il progetto utilizza il meccanismo di **mutual TLS** (mTLS). A differenza del normale TLS, nel quale solo il server presenta un certificato al client, con mTLS anche il client deve presentare un certificato valido.

Nel progetto, mTLS viene utilizzato in particolare nel flusso di accesso alle risorse protette. Envoy richiede un certificato client e lo valida rispetto alla CA configurata. Solo se il certificato risulta valido la richiesta può procedere verso la fase di autorizzazione tramite OPA.

Questo meccanismo introduce un controllo ulteriore rispetto alla sola autenticazione utente. Un attaccante che conoscesse le credenziali di un utente non potrebbe comunque accedere facilmente alle risorse protette senza possedere anche un certificato valido associato a un dispositivo registrato.

I certificati X.509 vengono quindi utilizzati come elemento tecnico per rappresentare l'identità del dispositivo. Essi permettono di collegare il dispositivo al processo di accesso e di rendere più forte il concetto di *trusted device*.

3.9 CA Service

Il progetto include un servizio interno denominato **CA Service**, incaricato di firmare le richieste di certificato dei dispositivi. Questo servizio permette di gestire il rilascio dei certificati senza esporre direttamente la chiave privata della Certification Authority al backend.

Nel processo di enrollment, il dispositivo genera o invia una richiesta di firma del certificato. Il backend si occupa della logica applicativa del processo, ma la firma vera e propria viene delegata al CA Service. Questa separazione è importante perché riduce l'esposizione di un elemento critico come la chiave privata della CA.

Il CA Service comunica con il backend tramite una rete interna Docker e utilizza un token di servizio per limitare l'accesso alle sue funzionalità. In questo modo, il servizio di firma rimane isolato e accessibile solo dai componenti autorizzati.

3.10 OPA

Open Policy Agent (OPA) è stato utilizzato come motore decisionale per le policy di accesso. All'interno dell'architettura, OPA svolge il ruolo di *Policy Decision Point*, cioè il componente che valuta le richieste e produce una decisione di autorizzazione.

OPA riceve da Envoy una richiesta strutturata contenente diversi attributi, come ruolo dell'utente, risorsa richiesta, azione, informazioni sul dispositivo, livello di rischio e altri dati contestuali. Sulla base di queste informazioni, OPA restituisce una decisione che indica se la richiesta deve essere consentita o negata.

Nel progetto, OPA viene eseguito come container separato e carica:

- file di policy;
- file di dati;
- configurazioni;
- input di test.

L'aspetto più importante è che OPA permette di separare la logica di autorizzazione dal codice applicativo. Le regole di accesso non sono codificate direttamente nel backend, ma sono definite in modo dichiarativo nei file di policy. Questo rende più semplice modificare, verificare e mantenere le regole di sicurezza.

3.11 Rego

Le policy valutate da OPA sono scritte in **Rego**, un linguaggio dichiarativo progettato per esprimere regole di autorizzazione.

Nel progetto, le policy Rego sono organizzate in più file, ciascuno dedicato a una parte specifica della valutazione:

- `access.rego`, per la decisione finale di accesso;
- `user.rego`, per i controlli relativi a ruolo, azione, dipartimento e MFA;
- `device.rego`, per i controlli relativi al dispositivo;
- `risk.rego`, per la valutazione del rischio;
- `helpers.rego`, per funzioni di supporto e lettura degli attributi.

La policy principale consente l'accesso solo se tutte le condizioni Zero Trust sono soddisfatte. Questo significa che il sistema non valuta un solo attributo, ma combina più fattori:

- ruolo autorizzato;
- MFA valida;
- azione consentita;
- dipartimento coerente;
- dispositivo fidato;
- dispositivo attivo;

- fingerprint JA3 non bloccato;
- IP appartenente a una zona consentita;
- sistema operativo supportato;
- rischio accettabile;
- accesso in orario consentito;
- utente non bloccato dal motore di rischio.

In caso di rifiuto, le policy restituiscono anche una lista di motivazioni, come ad esempio `role_not_allowed`, `untrusted_device`, `risk_score_too_high` oppure `outside_working_hours`. Questa scelta rende il sistema più trasparente e facilita le attività di debug, test e audit.

3.12 Splunk



Figura 9: Logo ufficiale di Splunk

Splunk è stato utilizzato come sistema di raccolta, indicizzazione e analisi degli eventi. Nel progetto svolge il ruolo di componente SIEM, ma contribuisce anche alla valutazione dinamica del rischio e alla logica adattiva di controllo degli accessi.

I servizi Docker sono configurati per inviare log a Splunk tramite HTTP Event Collector e tramite il driver di logging Docker. Vengono raccolti eventi provenienti da più componenti, tra cui backend, Envoy, OPA, MongoDB, nftables e Snort. Questa scelta permette di ottenere una visione centralizzata degli eventi applicativi, delle decisioni di accesso e degli alert di rete.

Nel progetto, Splunk è utilizzato per:

- raccogliere audit log;
- analizzare decisioni di accesso consentite e negate;
- osservare il comportamento degli utenti nel tempo;
- integrare eventi applicativi ed eventi di rete;
- supportare il calcolo dinamico del rischio;
- eseguire saved searches dedicate alla logica BARAC;
- attivare webhook verso il backend in caso di comportamento sospetto.

Un aspetto rilevante è la presenza di una app Splunk dedicata al progetto, contenente configurazioni, indici, saved searches e lookup. In particolare, i file `asset_weights.csv` e `resource_impact.csv` permettono di associare pesi, sensibilità e impatto alle risorse e alle azioni richieste.

Splunk viene inoltre utilizzato per supportare una logica di rischio più evoluta. Il rischio complessivo può essere interpretato come combinazione tra probabilità di comportamento anomalo e impatto della risorsa richiesta. In questo modo, una richiesta verso una risorsa sensibile produce un livello di rischio maggiore rispetto a una richiesta verso una risorsa pubblica, anche a parità di comportamento osservato.

La presenza di Splunk rende il sistema più vicino a uno scenario aziendale reale. In un'architettura Zero Trust, infatti, non è sufficiente bloccare o consentire richieste: è necessario osservare continuamente il comportamento di utenti e dispositivi, individuare anomalie e reagire in modo dinamico.

3.13 Splunk AI Toolkit e modello Random Forest

Per la componente di analisi del rischio è stato utilizzato **Splunk AI Toolkit**, che consente di applicare tecniche di Machine Learning direttamente sui dati raccolti in Splunk.

Nel progetto, il modello utilizzato è una **Random Forest**, addestrata per riconoscere comportamenti potenzialmente ostili sulla base di alcune feature derivate dagli eventi di accesso. Tra queste feature rientrano, ad esempio:

- numero totale di richieste negate;
- numero di deny concentrati nel tempo;
- numero di deny su risorse privilegiate;
- numero di deny legati all'autenticazione;
- numero di indirizzi IP distinti;
- rapporto tra richieste negate e richieste totali.

Il modello restituisce una probabilità di comportamento anomalo, che può essere utilizzata come componente applicativa del rischio. Tale valore viene poi combinato con altri segnali, come gli alert di rete prodotti da Snort e l'impatto della risorsa richiesta.

L'utilizzo di un modello di Machine Learning non sostituisce le policy Zero Trust, ma le arricchisce. Le policy continuano a stabilire le condizioni minime di accesso, mentre il modello contribuisce a individuare pattern sospetti che potrebbero non emergere da regole statiche.

3.14 Snort

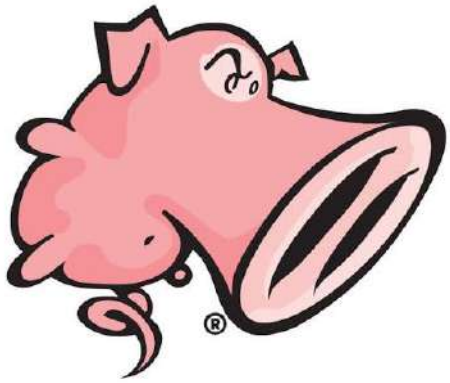


Figura 10: Logo ufficiale di Snort

Snort è stato inserito nel progetto come sistema IDS, cioè come strumento per il rilevamento di traffico sospetto. Il suo compito è osservare il traffico di rete e generare alert nel caso in cui vengano rilevati comportamenti potenzialmente anomali.

Nel file Docker Compose, Snort viene eseguito utilizzando il network mode del servizio Envoy. Questa configurazione gli permette di osservare il traffico che attraversa il proxy, cioè uno dei punti più importanti dell'architettura.

Gli alert prodotti da Snort possono essere salvati in formato JSON e raccolti successivamente da Splunk. In questo modo, gli eventi di rete possono contribuire all'osservabilità complessiva del sistema e, in prospettiva, alla valutazione dinamica del rischio.

All'interno del progetto, Snort rappresenta quindi un ulteriore livello di controllo: mentre OPA valuta le policy di accesso ed Envoy applica le decisioni, Snort osserva il traffico che attraversa il proxy e produce alert relativi a possibili anomalie di rete. Tali alert possono essere integrati in Splunk e contribuire alla componente di rischio di rete, indicata come `P_net` nella logica di calcolo del rischio.

3.15 nftables

nftables è stato utilizzato come componente firewall a livello L3/L4. Esso consente di definire regole per il controllo del traffico di rete e può essere usato per registrare o bloccare determinate comunicazioni.

Nel progetto, nftables è configurato come servizio separato e utilizza privilegi di rete specifici, come `NET_ADMIN` e `NET_RAW`. Inoltre, viene eseguito con `network_mode: host`, così da poter simulare controlli e logging a livello L3/L4 in modo più vicino a un ambiente reale. I log prodotti dal servizio possono essere inviati a Splunk, contribuendo alla raccolta centralizzata degli eventi.

Il ruolo di nftables è complementare rispetto a quello di Envoy e OPA. Envoy e OPA lavorano principalmente a livello applicativo e autorizzativo, mentre nftables opera a un livello più basso dello stack di rete. Questa distinzione permette di introdurre un controllo multilivello, coerente con l'idea di difesa in profondità.

3.16 Kali Linux

Nel progetto è presente anche un container basato su **Kali Linux**, utilizzato come ambiente di test offensivo. Il suo ruolo non è quello di fornire una funzionalità applicativa, ma di simulare traffico esterno o potenzialmente ostile verso l'architettura.

Kali viene collocato su una rete separata rispetto ai servizi principali, in modo da rappresentare un'origine esterna. Questo consente di testare il comportamento dei componenti di sicurezza, come Envoy, Snort, nftables e Splunk, in presenza di richieste anomale o non autorizzate.

La presenza di Kali permette quindi di verificare il sistema non solo dal punto di vista funzionale, ma anche dal punto di vista della resilienza rispetto a scenari di attacco simulati.

3.17 Postman



Figura 11: Logo ufficiale di Postman

Postman è stato utilizzato per testare le API e verificare il comportamento dei diversi flussi applicativi. Nel progetto è presente una collection dedicata, utile per inviare richieste al backend e simulare scenari di accesso.

Attraverso Postman è possibile testare, ad esempio:

- login degli utenti;
- accesso a risorse protette;
- richieste consentite;
- richieste negate;
- gestione dei dispositivi;
- enrollment;
- endpoint di rischio;
- webhook e rotte di servizio.

L'utilizzo di Postman è stato utile soprattutto durante la fase di sviluppo, perché ha permesso di verificare rapidamente il comportamento delle API senza dipendere necessariamente dal frontend.

3.18 TPM e Secure Enclave

Nel progetto sono presenti componenti funzionanti dedicati alla gestione dell'identità hardware-bound del dispositivo. In particolare, l'agente macOS integra la **Secure Enclave**, mentre l'agente Windows utilizza il **TPM**. Queste tecnologie permettono di proteggere chiavi crittografiche all'interno di componenti hardware dedicati, riducendo il rischio che una chiave privata possa essere esportata o copiata.

Nel contesto ZeroTrustHR, TPM e Secure Enclave sono collegati al tema dell'identità del dispositivo. Questo rafforza il processo di device enrollment, facendo in modo che il certificato client non sia semplicemente un file copiabile, ma sia collegato in modo più forte al dispositivo fisico.

Il progetto prevede anche una modalità software di fallback, utile per ambienti nei quali non sia disponibile una protezione hardware. Tale modalità è meno forte rispetto all'utilizzo di TPM o Secure Enclave, ma consente comunque di completare il flusso di enrollment e di testare il sistema.

TPM e Secure Enclave vengono quindi introdotti come tecnologie utili alla protezione dell'identità hardware-bound. La descrizione più dettagliata del relativo flusso implementativo viene trattata nel capitolo successivo, dedicato all'implementazione del sistema.

3.19 Variabili d'ambiente e gestione delle configurazioni

Un ulteriore aspetto tecnico rilevante riguarda l'uso delle variabili d'ambiente. Nel progetto, informazioni sensibili e parametri di configurazione vengono gestiti tramite file `.env` e variabili lette dai container.

Tra i valori configurati tramite variabili d'ambiente rientrano:

- credenziali di MongoDB;
- secret JWT;
- token Splunk HEC;
- secret per i webhook Splunk;
- token del CA Service;
- URL dei servizi interni;
- credenziali e parametri di accesso a Splunk.

Questa scelta evita di inserire direttamente nel codice sorgente valori sensibili o parametri specifici dell'ambiente di esecuzione. Si tratta di una pratica importante dal punto di vista della sicurezza e della manutenibilità, poiché permette di separare il codice applicativo dalla configurazione.

3.20 Sintesi degli strumenti utilizzati

La Tabella 2 riassume i principali strumenti utilizzati e il loro ruolo nel progetto.

Strumento	Utilizzo nel progetto
Docker / Docker Compose	Containerizzazione e orchestrazione dei servizi del sistema.
Node.js / TypeScript	Sviluppo del backend applicativo.
Express	Definizione delle API REST e delle rotte backend.
React / Vite / TypeScript	Sviluppo del frontend e dell'interfaccia utente.
MongoDB	Persistenza di utenti, dispositivi, policy e dati applicativi.
JWT	Autenticazione degli utenti dopo il login.
Envoy	Proxy e Policy Enforcement Point per le risorse protette.
mTLS / X.509	Autenticazione del dispositivo tramite certificati client.
CA Service	Firma delle richieste di certificato durante l'enrollment dei dispositivi.
OPA	Motore decisionale per la valutazione delle policy.
Rego	Linguaggio dichiarativo per la scrittura delle policy di autorizzazione.
Splunk	Raccolta, indicizzazione e analisi degli eventi di sicurezza, supporto al calcolo del rischio e attivazione di webhook adattivi.
Splunk AI Toolkit / Random Forest	Analisi del comportamento e stima della probabilità di comportamento anomalo.
Snort	IDS per il rilevamento di traffico sospetto e generazione di alert utilizzabili nella componente di rischio di rete.
nftables	Firewall e logging del traffico a livello L3/L4, integrato con Splunk per la raccolta degli eventi.
Kali Linux	Simulazione di traffico esterno o ostile durante i test.
Postman	Test manuale delle API e dei flussi applicativi.
TPM / Secure Enclave	Rafforzamento dell'identità dispositivo tramite protezione hardware delle chiavi.

Tabella 2: Sintesi degli strumenti e delle tecnologie utilizzate nel progetto ZeroTrustHR.

3.21 Considerazioni finali

L'insieme degli strumenti utilizzati mostra che il progetto ZeroTrustHR non si limita alla realizzazione di una semplice applicazione web, ma integra diversi livelli di sicurezza. Il frontend e il backend costituiscono la parte applicativa, MongoDB gestisce la persistenza, Docker Compose consente l'orchestrazione dei servizi, Envoy e OPA implementano rispettivamente enforcement e decisione delle policy, mentre Splunk, Snort, nftables e Kali Linux contribuiscono all'osservabilità, al controllo del traffico, alla simulazione di scenari ostili e alla valutazione dinamica del rischio.

La scelta di separare questi componenti permette di rappresentare in modo più realistico un'architettura Zero Trust. Ogni strumento ha una responsabilità specifica e contribuisce a ridurre la fiducia implicita nel sistema. L'accesso alle risorse non dipende da un unico controllo, ma dalla collaborazione tra identità utente, identità dispositivo, policy, rischio, rete e monitoraggio continuo.



Implementazione applicativa e flussi principali

4.1 Introduzione

Dopo aver descritto l'architettura generale del sistema e gli strumenti utilizzati, questo capitolo analizza l'implementazione applicativa e infrastrutturale del progetto **ZeroTrustHR**. L'obiettivo è mostrare come i principali componenti siano stati realizzati e collegati tra loro per costruire un prototipo funzionante di architettura Zero Trust.

In questa parte non vengono ancora approfondite nel dettaglio le policy di autorizzazione, la logica Rego, la valutazione del rischio e il comportamento adattivo del sistema, che saranno trattati nel capitolo successivo. Il focus di questo capitolo è invece rivolto alla struttura concreta dell'applicazione, ai flussi di autenticazione, alla gestione delle risorse protette, al processo di enrollment dei dispositivi e all'utilizzo dei certificati client.

L'implementazione può essere suddivisa nei seguenti blocchi principali:

- backend applicativo;
- frontend e flusso di enrollment lato client;
- autenticazione utente tramite JWT;
- gestione delle risorse protette;
- utilizzo di Envoy come punto di ingresso sicuro;
- gestione della comunicazione mTLS;
- processo di device enrollment;
- agenti locali per macOS e Windows;
- firma dei certificati tramite CA Service;
- gestione dei dispositivi fidati;
- persistenza dei dati tramite MongoDB.

Questi elementi costituiscono la base tecnica su cui vengono poi applicate le policy Zero Trust descritte nel capitolo successivo.

4.2 Struttura generale del backend

Il backend rappresenta il nucleo applicativo del progetto. Esso è stato sviluppato utilizzando **Node.js**, **TypeScript** ed **Express**. La scelta di TypeScript permette di ottenere un codice più leggibile e più controllabile, grazie alla presenza di tipi espliciti per utenti, dispositivi, richieste di accesso e configurazioni.

La struttura del backend è organizzata in modo modulare, separando le responsabilità principali in cartelle differenti. In particolare, sono presenti moduli dedicati a:

- definizione delle rotte HTTP;
- gestione della logica applicativa;
- middleware di controllo;
- connessione e interazione con MongoDB;
- modelli dati;
- servizi di supporto;
- gestione delle variabili d'ambiente.

Questa organizzazione consente di mantenere separati i diversi aspetti del sistema. Le rotte si occupano di esporre gli endpoint, i servizi contengono la logica applicativa, i middleware applicano controlli intermedi e i modelli permettono di strutturare i dati salvati nel database.

Le principali aree funzionali del backend riguardano:

- autenticazione degli utenti;
- gestione delle sessioni tramite JWT;
- gestione dei dispositivi;
- enrollment dei dispositivi;
- approvazione, sospensione e revoca dei device;
- accesso alle risorse protette;
- interazione con il servizio CA;
- persistenza delle informazioni in MongoDB.

Il backend non è quindi soltanto un normale server applicativo, ma rappresenta anche un punto di coordinamento tra identità utente, identità dispositivo, certificati, database e componenti di sicurezza.

4.3 Rotte principali del backend

Nel progetto sono presenti diverse rotte API, ognuna dedicata a una specifica area del sistema. Le principali possono essere riassunte nel modo seguente:

Rotta	**Funzione principale**
`/api/auth`	Gestisce login, autenticazione e recupero delle informazioni dell'utente corrente.
`/api/devices`	Gestisce challenge di enrollment, registrazione del dispositivo, approvazione, rifiuto, revoca e sincronizzazione dei dispositivi fidati verso il motore di policy.
`/api/protected`	Espone le risorse protette del caso studio HR.
`/api/access`	Permette di simulare e valutare richieste di accesso a livello applicativo.
`/api/risk`	Espone funzionalità legate alla valutazione del rischio, approfondite nel capitolo successivo.
`/api/splunk-webhook`	Riceve notifiche o eventi provenienti da Splunk, trattati nel capitolo dedicato a policy e rischio.
`/health`	Fornisce un endpoint di controllo dello stato del servizio backend.

Tabella 3: Principali rotte API del backend.

In questo capitolo l'attenzione è posta soprattutto sulle rotte di autenticazione, sulle rotte dei dispositivi e sulle risorse protette. Le rotte relative al rischio e al comportamento adattivo verranno invece riprese nel Capitolo 6, poiché sono strettamente collegate alle policy e alla valutazione dinamica degli accessi.

4.4 Autenticazione dell'utente

Il primo flusso applicativo del sistema è l'autenticazione dell'utente. L'utente inserisce le proprie credenziali tramite frontend oppure tramite una richiesta API diretta. Il backend riceve username e password, verifica la presenza dei campi obbligatori e controlla la corrispondenza con i dati salvati nel database.

La rotta principale per il login è:

```
POST /api/auth/login
```

Se le credenziali sono corrette, il backend genera un **JSON Web Token**. Il token contiene alcune informazioni essenziali sull'utente, come identificativo, username, ruolo e dipartimento. Queste informazioni saranno poi utilizzate dai componenti successivi del sistema per ricostruire il contesto della richiesta.

Un esempio semplificato di risposta al login è il seguente:

Esempio di risposta al login

```
{
  message: Login completato,
  token: ...,
  user: {
    id: ...,
    username: admin,
    role: admin,
    department: it,
    name: ...,
    email: ...,
    status: active,
    mfa_enabled: true
  }
}
```

Il token viene successivamente inviato dal client nelle richieste protette tramite l'header HTTP:

```
Authorization:  Bearer <token>
```

È importante sottolineare che il JWT non rappresenta da solo una decisione di accesso. Esso dimostra che l'utente si è autenticato, ma non implica automaticamente che l'utente possa accedere a qualsiasi risorsa. In un'architettura Zero Trust, l'autenticazione è solo il primo livello di verifica.

4.5 Middleware di verifica del JWT

Per proteggere le rotte che richiedono autenticazione, il backend utilizza un middleware dedicato alla verifica del JWT. Il middleware intercetta la richiesta, legge l'header `Authorization`, estrae il token e ne verifica la validità.

Se il token è valido, le informazioni dell'utente vengono rese disponibili alla richiesta successiva. In questo modo, controller e servizi possono utilizzare attributi come:

- identificativo utente;
- username;
- ruolo;
- dipartimento;
- stato dell'utente;
- eventuale valore dell'attributo relativo alla MFA.

Se invece il token è assente, scaduto, malformato o firmato con un secret non valido, la richiesta viene bloccata. Questo meccanismo evita che endpoint riservati vengano raggiunti da utenti non autenticati.

La presenza del middleware consente inoltre di centralizzare la logica di autenticazione, evitando duplicazione di codice all'interno delle singole rotte.

4.6 Frontend e interazione con l'utente

Il frontend è stato realizzato con **React**, **Vite** e **TypeScript**. Esso rappresenta il livello di presentazione del sistema e consente all'utente di interagire con le funzionalità principali dell'applicazione.

Attraverso l'interfaccia utente è possibile:

- effettuare il login;
- conservare temporaneamente il token di autenticazione;
- inviare richieste al backend;
- accedere alle risorse disponibili;
- visualizzare gli esiti delle operazioni;
- avviare e completare parte del processo di device enrollment;
- generare un'identità software di fallback;
- scaricare i materiali necessari alla configurazione del dispositivo.

Nel frontend sono presenti componenti dedicati al flusso di enrollment del dispositivo. In particolare, la pagina `DeviceEnrollment` permette di guidare l'utente nella registrazione del device, mentre moduli di supporto gestiscono la comunicazione con le API di enrollment, la generazione dell'identità software e il download dei file necessari.

Questa scelta permette di rendere il processo di registrazione del dispositivo più vicino a un flusso applicativo reale. L'enrollment non viene quindi gestito soltanto tramite richieste manuali o strumenti di test, ma viene integrato nell'interfaccia utente.

Il frontend, tuttavia, non contiene la logica di autorizzazione principale. Questa scelta è fondamentale in un progetto Zero Trust: il client non deve essere considerato un componente affidabile. Anche se l'interfaccia utente mostra o nasconde determinate funzionalità, la decisione effettiva deve sempre essere verificata lato server e tramite i componenti di sicurezza.

Per questo motivo, ogni richiesta significativa viene comunque sottoposta ai controlli del backend, di Envoy e di OPA. Il frontend ha quindi un ruolo di interfaccia e supporto operativo, non di autorità decisionale.

4.7 Risorse protette del caso studio

Il caso studio ZeroTrustHR prevede alcune risorse protette, pensate per simulare un contesto aziendale di tipo HR. Le principali risorse utilizzate nel progetto sono:

- `employee_records`;
- `company_policies`;
- `public_products`.

La risorsa `employee_records` rappresenta dati sensibili relativi ai dipendenti. Si tratta quindi di una risorsa ad alta criticità, che deve essere accessibile solo a utenti autorizzati e in condizioni sicure.

La risorsa `company_policies` rappresenta documenti o regole aziendali. Ha un livello di sensibilità inferiore rispetto ai record dei dipendenti, ma richiede comunque controlli di accesso coerenti con il ruolo dell'utente.

La risorsa `public_products` rappresenta invece una risorsa a bassa sensibilità, utilizzata per mostrare come il sistema possa gestire anche dati meno critici con policy più permissive.

Le rotte principali associate a queste risorse sono:

- `GET /api/protected/employee-records`;
- `GET /api/protected/company-policies`;
- `GET /api/protected/public-products`.

Prima di restituire i dati, il backend verifica che la richiesta sia coerente con la risorsa effettivamente invocata. In particolare, viene controllato l'header `x-resource-name`, che deve corrispondere alla risorsa richiesta.

Ad esempio, una richiesta alla rotta:

```
GET /api/protected/employee-records
```

deve dichiarare:

```
x-resource-name:  employee_records
```

Questo controllo evita incoerenze tra la route applicativa e la risorsa dichiarata nella richiesta.

4.8 Catena di protezione delle risorse

L'accesso alle risorse protette non avviene tramite una semplice chiamata diretta al backend. La richiesta attraversa una catena di controlli pensata per verificare più aspetti dell'identità e del contesto.

La sequenza generale può essere rappresentata così:

Client → Envoy → controllo mTLS → autorizzazione OPA → backend → risorsa

A livello backend, vengono inoltre applicati middleware interni, tra cui:

- verifica della connessione mTLS;
- verifica del dispositivo fidato;
- verifica del JWT;
- controllo di coerenza della risorsa richiesta.

Nel progetto, il primo controllo forte avviene a livello di Envoy, che richiede il certificato client e consulta OPA prima di inoltrare la richiesta. I controlli lato backend rappresentano quindi un ulteriore livello di verifica e coerenza applicativa, utile a evitare che una richiesta raggiunta internamente venga trattata come automaticamente affidabile.

Questo modello realizza una forma di difesa multilivello. Anche se un controllo dovesse essere aggirato o configurato in modo non corretto, altri componenti continuerebbero a verificare la richiesta.

4.9 Envoy come punto di ingresso sicuro

Nel progetto, **Envoy** viene utilizzato come proxy posto davanti alle risorse protette. Esso riceve le richieste in ingresso e decide se inoltrarle al backend oppure bloccarle.

La configurazione prevede due percorsi principali:

- un listener dedicato alle richieste protette, con controllo mTLS;
- un listener dedicato al processo di enrollment del dispositivo.

Il primo listener è utilizzato per accedere alle risorse applicative protette. In questo caso il client deve presentare un certificato valido. Se il certificato non viene presentato o non è firmato dalla CA attesa, la comunicazione non viene considerata valida.

La configurazione mTLS richiede esplicitamente il certificato client, rendendo il possesso di un certificato valido una condizione preliminare per l'accesso alle risorse protette.

Il secondo listener è pensato per consentire le prime fasi dell'enrollment, quando il dispositivo non possiede ancora un certificato client approvato. Questa distinzione è necessaria perché il sistema deve permettere la registrazione iniziale dei dispositivi, ma senza esporre liberamente le risorse protette.

Envoy ha quindi un ruolo fondamentale: separa il traffico ordinario dal traffico di enrollment e impedisce l'accesso diretto non controllato alle API sensibili.

4.10 Comunicazione sicura tramite mTLS

Uno degli aspetti più importanti dell'implementazione è l'uso di **mutual TLS**. Nel normale TLS, il client verifica l'identità del server tramite il certificato server. Nel caso del mutual TLS, invece, anche il client deve presentare un proprio certificato.

Nel progetto, il certificato client viene utilizzato per rappresentare l'identità del dispositivo. Ciò consente di introdurre un livello di sicurezza aggiuntivo rispetto al solo login utente.

Il flusso generale è il seguente:

1. il client apre una connessione verso Envoy;
2. Envoy presenta il certificato server;
3. il client presenta il proprio certificato client;
4. Envoy verifica che il certificato client sia firmato dalla CA configurata;
5. se la verifica ha esito positivo, la richiesta può passare alla fase successiva;
6. in caso contrario, la richiesta viene rifiutata.

L'uso di mTLS consente di legare l'accesso non solo all'utente, ma anche al dispositivo. Un attaccante in possesso delle sole credenziali utente non sarebbe quindi sufficiente: dovrebbe possedere anche un certificato client valido associato a un dispositivo riconosciuto.

4.11 Processo di device enrollment

Il processo di *device enrollment* permette di registrare un nuovo dispositivo all'interno del sistema. Questo flusso è essenziale in un'architettura Zero Trust, perché il dispositivo deve diventare un'entità verificabile e non può essere considerato affidabile automaticamente.

Il processo si articola in due fasi principali:

- generazione della challenge;
- completamento dell'enrollment.

La prima fase viene gestita tramite la rotta:

`POST /api/devices/challenge`

Il client invia informazioni preliminari sul dispositivo, come nome, tipo, sistema operativo e tipo di chiave utilizzata. Il backend genera quindi:

- un identificativo univoco del dispositivo;
- una challenge casuale;
- una SAN URI associata al dispositivo;
- uno stato iniziale pari a `pending`.

Un esempio semplificato di risposta è il seguente:

Esempio di risposta alla richiesta di challenge

```
{
  device_id: ...,
  certificate_san_uri: urn:zerotrusthr:device:...,
  challenge: ...,
  status: pending
}
```

La seconda fase viene gestita tramite la rotta:

`POST /api/devices/enroll`

In questa fase il client invia la CSR, la chiave pubblica e, quando necessario, la firma della challenge. Il backend verifica che il dispositivo esista, che sia ancora in stato `pending` e che i dati inviati siano coerenti.

Nel caso dell'enrollment gestito dal frontend, il sistema può generare anche un'identità software di fallback. Tale modalità produce i materiali necessari per completare il flusso anche in assenza di TPM o Secure Enclave. Pur essendo meno robusta rispetto a una chiave hardware-bound, questa soluzione consente di testare l'intero processo di registrazione, certificazione e accesso mTLS.

Al termine dell'enrollment, il dispositivo non diventa automaticamente trusted. Rimane in stato `pending` fino all'approvazione amministrativa. Questa scelta evita che un dispositivo possa autoregistrarsi e ottenere immediatamente accesso alle risorse protette.

4.12 Stati del dispositivo

Ogni dispositivo registrato nel sistema possiede uno stato. Gli stati principali sono:

- `pending`;
- `active`;
- `suspended`;
- `revoked`.

Lo stato `pending` indica che il dispositivo ha avviato il processo di enrollment, ma non è ancora stato approvato. Lo stato `active` indica che il dispositivo è stato approvato e può essere considerato affidabile. Lo stato `suspended` indica un dispositivo sospeso o rifiutato, mentre lo stato `revoked` indica un dispositivo precedentemente valido ma successivamente revocato.

Le principali operazioni amministrative sono:

- approvazione del dispositivo;
- rifiuto del dispositivo;
- revoca del dispositivo;
- sincronizzazione dei dispositivi fidati.

Le rotte associate a queste operazioni sono:

- `POST /api/devices/:deviceId/approve`;
- `POST /api/devices/:deviceId/reject`;
- `POST /api/devices/:deviceId/revoke`;
- `POST /api/devices/sync-opa`.

La presenza di stati espliciti rende più chiara la gestione del ciclo di vita del dispositivo. Un device non è semplicemente presente o assente, ma può trovarsi in condizioni diverse, ognuna con implicazioni differenti sulla possibilità di accesso.

4.13 Approvazione del dispositivo e rilascio del certificato

Quando un dispositivo viene approvato, il backend avvia il processo di firma del certificato. Il dispositivo deve aver completato correttamente l'enrollment e deve trovarsi in stato `pending`.

Durante l'approvazione, il backend invia al servizio CA le informazioni necessarie per firmare il certificato, tra cui:

- CSR del dispositivo;
- identificativo del dispositivo;
- SAN URI;
- chiave pubblica verificata.

Se il processo ha esito positivo, il certificato viene restituito al backend, salvato e associato al dispositivo. Lo stato del dispositivo passa quindi ad `active`.

Da questo momento il dispositivo può presentare il proprio certificato client durante la connessione mTLS. Tuttavia, anche in presenza di un certificato valido, l'accesso finale resta subordinato alle policy di autorizzazione e alla valutazione del contesto.

4.14 CA Service

La firma dei certificati viene delegata a un servizio separato, denominato **CA Service**. Questa scelta consente di evitare che la chiave privata della Certification Authority venga gestita direttamente dal backend applicativo.

Il CA Service riceve le richieste di firma dal backend e verifica che siano presenti i dati necessari. In particolare, controlla:

- presenza della CSR;
- presenza del device ID;
- coerenza della SAN URI;
- validità formale della CSR;
- corrispondenza tra chiave pubblica dichiarata e chiave contenuta nella CSR.

Solo se i controlli hanno esito positivo, il servizio firma il certificato client tramite la CA interna.

Questa separazione migliora la sicurezza del sistema perché isola una funzione particolarmente sensibile. La chiave privata della CA rappresenta infatti un asset critico: se compromessa, un attaccante potrebbe generare certificati apparentemente validi. Separare il servizio di firma dal backend riduce quindi la superficie di esposizione.

4.15 Identità hardware-bound tramite TPM e Secure Enclave

Nel progetto sono implementate diverse modalità di gestione della chiave del dispositivo:

- chiave protetta tramite `TPM`, per ambienti Windows;
- chiave protetta tramite `Secure Enclave`, per ambienti macOS compatibili;
- chiave software come modalità di fallback.

Nel caso di TPM e Secure Enclave, l'obiettivo è rendere più difficile la copia o il trasferimento dell'identità del dispositivo.

Il progetto include componenti distinti per gli agenti locali macOS e Windows. L'agente macOS sfrutta meccanismi legati alla Secure Enclave, mentre l'agente Windows utilizza il TPM. Entrambi rafforzano il legame tra dispositivo fisico e identità crittografica utilizzata durante l'enrollment. Entrambi hanno lo scopo di rafforzare il legame tra dispositivo fisico e identità crittografica utilizzata durante l'enrollment.

Durante l'enrollment, il dispositivo può dimostrare il possesso della chiave privata firmando la challenge generata dal backend. Il backend verifica poi la firma utilizzando la chiave pubblica ricevuta. Se la verifica ha esito positivo, il sistema ottiene una prova crittografica del possesso della chiave privata.

Questa parte rafforza il concetto di identità dispositivo. Il certificato client non è più soltanto un file associato a un device, ma diventa parte di un'identità legata all'hardware fisico. Questo principio è particolarmente coerente con la Zero Trust Architecture, nella quale non basta conoscere le credenziali dell'utente: è necessario verificare anche il dispositivo utilizzato.

Nel caso di fallback software, il sistema mantiene comunque la possibilità di completare l'enrollment, ma con un livello di sicurezza inferiore. Per questo motivo possono essere richiesti controlli aggiuntivi, come fingerprint di rete, informazioni sul dispositivo o ulteriori attributi contestuali.

Si mostrano, ora, i diagrammi di flusso delle modalità di gestione della chiave del dispositivo:

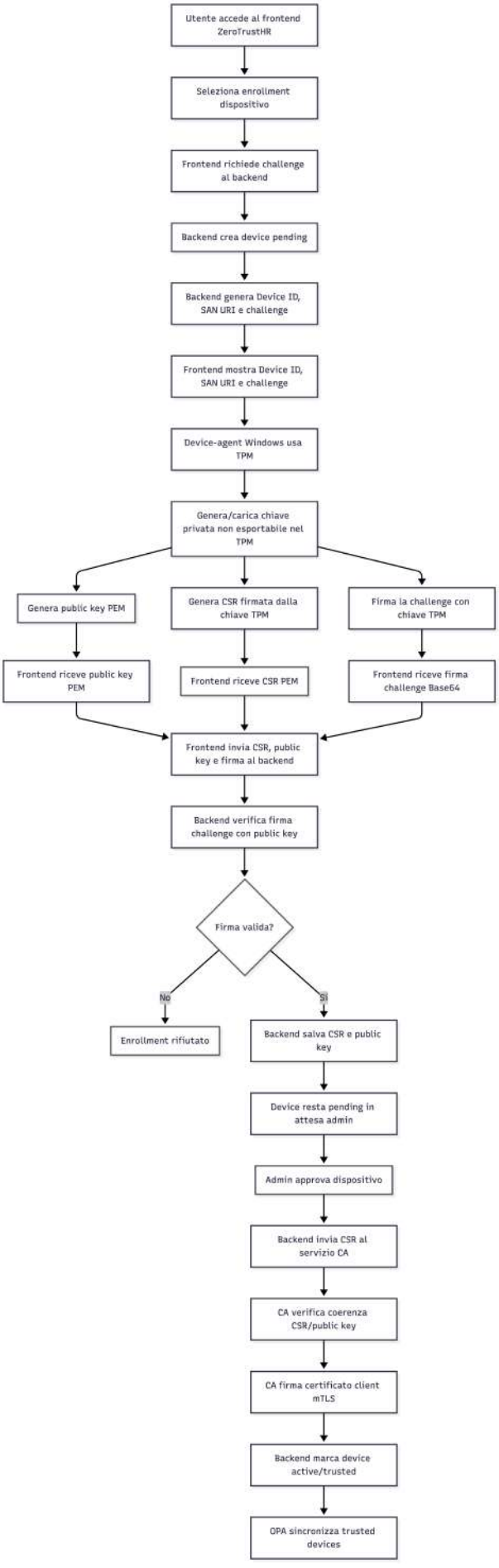


Figura 12: Flusso del TPM

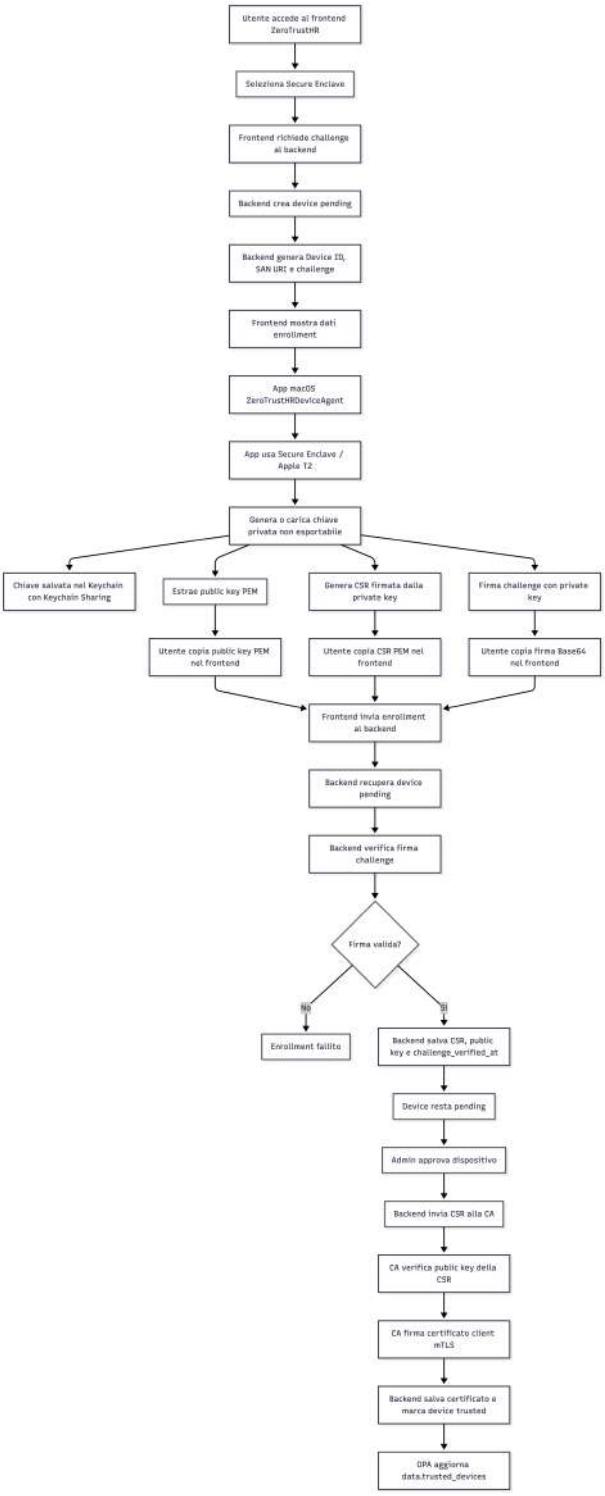


Figura 13: Flusso del Secure Enclave

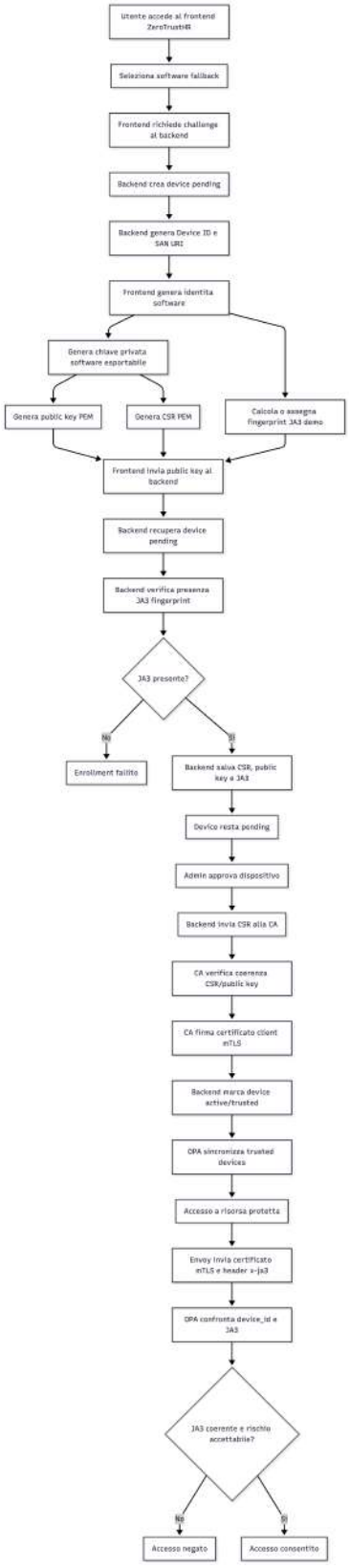


Figura 14: Flusso del Software Fallback

4.16 Revoca e sospensione dei dispositivi

Oltre all'approvazione, il sistema prevede anche la possibilità di rifiutare o revocare un dispositivo. Questa funzionalità è importante perché un dispositivo può cambiare stato nel tempo: può essere smarrito, compromesso, dismesso o semplicemente non più autorizzato.

Il rifiuto viene applicato generalmente a un dispositivo ancora in stato `pending`. In questo caso il dispositivo viene posto in stato `suspended` e non può essere utilizzato come trusted device.

La revoca riguarda invece un dispositivo già approvato. Quando viene revocato, il suo stato passa a `revoked`. A livello logico, ciò significa che il dispositivo non deve più essere considerato affidabile.

La revoca non è soltanto una modifica nel database, ma ha effetto anche sulle decisioni di accesso. Dopo la revoca, infatti, l'elenco dei dispositivi fidati deve essere aggiornato affinché il sistema non conceda più accesso sulla base di un device non valido.

4.17 Sincronizzazione dei dispositivi fidati

Dopo operazioni come approvazione o revoca, il backend aggiorna l'elenco dei dispositivi fidati. Questa sincronizzazione permette ai componenti di sicurezza di utilizzare dati aggiornati durante le decisioni di accesso.

Il flusso può essere riassunto così:

1. un amministratore approva o revoca un dispositivo;
2. MongoDB viene aggiornato con il nuovo stato;
3. il backend ricostruisce la lista dei dispositivi attivi e fidati;
4. la lista viene resa disponibile al motore di policy;
5. le richieste successive vengono valutate usando lo stato aggiornato.

Questa sincronizzazione è fondamentale perché l'affidabilità del dispositivo non è permanente. In un modello Zero Trust, la fiducia è sempre temporanea e deve poter essere modificata quando cambiano le condizioni.

Il dettaglio della valutazione delle policy sui dispositivi verrà approfondito nel Capitolo 6.

4.18 Persistenza dei dati con MongoDB

MongoDB viene utilizzato come database principale del progetto. Le informazioni vengono salvate in collection differenti, ciascuna dedicata a un tipo di dato.

Le principali collection utilizzate sono:

- `users`, per gli utenti;
- `devices`, per i dispositivi registrati;
- `access_policies`, per le policy applicative;
- `access_requests`, per le richieste di accesso;
- `blocked_users`, per gli utenti bloccati tramite logica adattiva;
- `employee_records`, per i dati HR;
- `company_policies`, per le policy aziendali;
- `public_products`, per le risorse a bassa sensibilità.

L'utilizzo di MongoDB permette di gestire strutture dati flessibili, adatte a un sistema nel quale gli attributi rilevanti possono variare nel tempo. Ad esempio, un dispositivo può contenere informazioni su sistema operativo, stato, chiave pubblica, certificato, fingerprint e dati di enrollment.

Il database viene eseguito come servizio Docker separato e i dati vengono mantenuti tramite volume persistente. In questo modo, le informazioni non vengono perse al riavvio dei container.

4.19 Seeder e dati dimostrativi

Per rendere il progetto testabile, il backend può inizializzare il database con dati dimostrativi. Tali dati includono utenti, dispositivi, risorse e policy applicative.

Questa scelta è utile perché consente di simulare immediatamente scenari differenti, senza dover inserire manualmente tutte le informazioni. Ad esempio, è possibile testare:

- un utente amministratore;
- un utente HR;
- un dipendente ordinario;

- un cliente con privilegi limitati;
- una risorsa sensibile;
- una risorsa aziendale;
- una risorsa pubblica.

La presenza di dati dimostrativi permette anche di verificare più facilmente il comportamento del sistema durante i test con Postman, frontend o richieste manuali.

4.20 Endpoint di valutazione applicativa dell'accesso

Oltre al flusso completo basato su Envoy e OPA, il progetto include anche una rotta applicativa per simulare una valutazione di accesso:

```
POST /api/access/evaluate
```

Questa rotta permette di inviare al backend una richiesta contenente informazioni come:

- username;
- nome del dispositivo;
- risorsa richiesta;
- azione richiesta;
- indirizzo IP sorgente;
- zona di rete;
- valore dell'attributo `mfa_verified`.

Il backend recupera utente, dispositivo e risorsa dal database, quindi produce una decisione applicativa. Questa modalità è utile soprattutto durante la fase di sviluppo e test, perché permette di controllare la logica del backend senza dover attraversare ogni volta l'intero flusso infrastrutturale.

Un esempio di richiesta è il seguente:

Esempio di richiesta di valutazione applicativa

```
{
  username: admin,
  device_name: Admin Laptop,
  resource_name: employee_records,
  action: read,
  source_ip: 192.168.1.10,
  network_zone: internal,
  mfa_verified: true
}
```

La risposta contiene l'esito della valutazione e alcune informazioni sul contesto analizzato:

Esempio di risposta della valutazione applicativa

```
{
  message: Valutazione accesso completata,
  username: admin,
  role: admin,
  device_name: Admin Laptop,
  device_status: active,
  device_trusted: true,
  resource_name: employee_records,
  action: read,
  network_zone: internal,
```

```
  mfa_verified: true,
  decision: allow,
  reason: policy_satisfied
}
```

Questa rotta non sostituisce il flusso Zero Trust completo, ma costituisce uno strumento utile per verificare il comportamento della logica applicativa.

Nel progetto questa rotta ha quindi un ruolo di supporto e verifica applicativa. Il flusso principale di enforcement per le risorse protette rimane quello basato su Envoy, mTLS e OPA, mentre la valutazione dinamica del rischio viene approfondita nel capitolo successivo.

4.21 Logging applicativo

Il backend registra gli eventi principali generati dal sistema. I log sono utili sia per il debug durante lo sviluppo, sia per ricostruire le attività effettuate dagli utenti.

Tra gli eventi registrabili rientrano:

- login riusciti o falliti;
- richieste di accesso;
- decisioni allow o deny;
- enrollment di dispositivi;
- approvazione o revoca di dispositivi;
- errori applicativi;
- operazioni amministrative.

In una prospettiva Zero Trust, il logging è particolarmente importante perché consente di osservare il comportamento del sistema nel tempo. Tuttavia, l'analisi avanzata degli eventi tramite Splunk, la valutazione dinamica del rischio e il blocco adattivo degli utenti verranno trattati nel Capitolo 6.

4.22 Flusso complessivo implementato

Il flusso generale implementato nel progetto può essere sintetizzato nel modo seguente:

1. l'utente effettua il login;
2. il backend verifica le credenziali;
3. il backend rilascia un JWT;
4. il dispositivo viene eventualmente registrato tramite enrollment;
5. un amministratore approva il dispositivo;
6. il CA Service firma il certificato client;
7. il dispositivo utilizza il certificato per stabilire una connessione mTLS;
8. Envoy riceve la richiesta e verifica la presenza di un certificato client valido;
9. Envoy invia gli attributi della richiesta a OPA tramite il filtro di autorizzazione esterna;
10. OPA valuta le policy e restituisce una decisione;
11. la richiesta viene inoltrata al backend solo se la decisione è positiva;
12. il backend restituisce la risorsa richiesta;
13. l'evento viene registrato a fini di audit.

Il flusso può essere rappresentato anche in forma sintetica:

Login → JWT → Enrollment dispositivo → Certificato client → mTLS → Envoy → OPA → Backend → Risorsa

Questo schema mostra come il sistema combini identità utente e identità dispositivo. L'accesso non è basato soltanto sulla password o sul token, ma anche sul possesso di un certificato client e sullo stato del dispositivo registrato.

4.23 Sintesi dei flussi del capitolo

La Tabella 4 riassume i principali flussi implementativi descritti in questo capitolo.

Flusso	Descrizione
Login utente	Verifica delle credenziali e generazione del JWT.
Verifica JWT	Controllo del token nelle richieste autenticate.
Accesso risorse protette	Accesso alle risorse HR tramite rotte dedicate, controllo mTLS, autorizzazione OPA e verifiche lato backend.
mTLS	Verifica del certificato client per rafforzare l'identità del dispositivo.
Device enrollment	Registrazione iniziale del dispositivo tramite challenge e CSR.
Frontend enrollment	Supporto lato interfaccia alla registrazione del dispositivo, generazione dell'identità software e download dei materiali di configurazione.
Agenti locali	Componenti macOS e Windows pensati per rafforzare l'identità del dispositivo tramite Secure Enclave o TPM.
Approvazione dispositivo	Passaggio del dispositivo allo stato active e rilascio del certificato.
CA Service	Firma dei certificati client tramite servizio separato.
Revoca dispositivo	Aggiornamento dello stato del dispositivo e rimozione della fiducia.
Persistenza MongoDB	Salvataggio di utenti, dispositivi, risorse, richieste e policy applicative.
Logging	Registrazione degli eventi principali generati dal sistema.

Tabella 4: Sintesi dei principali flussi implementativi descritti nel Capitolo 5.

4.24 Considerazioni finali

L'implementazione applicativa di ZeroTrustHR mostra come un sistema Zero Trust richieda la cooperazione di più componenti. Il backend gestisce utenti, dispositivi e risorse; il frontend consente l'interazione con l'utente e supporta il flusso di enrollment; MongoDB conserva lo stato del sistema; Envoy protegge l'accesso alle risorse; mTLS rafforza l'identità del dispositivo; il CA Service consente il rilascio dei certificati client; gli agenti locali introducono il supporto all'identità hardware-bound tramite TPM e Secure Enclave.

Questo capitolo ha descritto la base tecnica del sistema. Il capitolo successivo approfondirà invece il cuore decisionale dell'architettura, cioè la valutazione delle policy tramite OPA e Rego, il ruolo del rischio, l'integrazione con Splunk e i meccanismi adattivi di blocco o negazione degli accessi.



Policy, autorizzazione e valutazione del rischio

5.1 Introduzione

Nel capitolo precedente sono stati descritti i principali flussi applicativi del progetto **ZeroTrustHR**: autenticazione, gestione dei dispositivi, mTLS, enrollment e accesso alle risorse protette. Tuttavia, il cuore decisionale del sistema non risiede soltanto nel backend applicativo, ma soprattutto nella valutazione delle policy di accesso.

Questo capitolo approfondisce il meccanismo attraverso cui il sistema decide se una richiesta debba essere consentita o negata. La decisione non si basa su un singolo parametro, ma sulla combinazione di diversi fattori: identità dell'utente, ruolo, dipartimento, azione richiesta, risorsa, stato del dispositivo, MFA, zona di rete, fingerprint, orario e rischio associato alla richiesta.

L'obiettivo è mostrare come il progetto implementi una logica coerente con il paradigma Zero Trust: ogni richiesta viene valutata singolarmente, anche se proviene da un utente già autenticato o da un dispositivo già registrato.

Il capitolo è organizzato attorno ai seguenti aspetti:

- ruolo di OPA come Policy Decision Point;
- struttura delle policy Rego;
- separazione tra controlli utente, dispositivo e rischio;
- motivi di negazione delle richieste;
- valutazione dinamica del rischio;
- integrazione con Splunk e Splunk AI Toolkit;
- utilizzo degli alert Snort nella componente di rischio di rete;
- logica BARAC basata sul confronto tra rischio e beneficio;
- aggiornamento della blocklist e risposta adattiva.

5.2 OPA come Policy Decision Point

Nel progetto **ZeroTrustHR**, **Open Policy Agent** viene utilizzato come *Policy Decision Point*, cioè come componente incaricato di prendere la decisione autorizzativa. OPA riceve una richiesta strutturata, valuta le regole definite in Rego e restituisce una risposta che indica se l'accesso deve essere consentito oppure negato.

La separazione tra backend e motore di policy rappresenta una scelta architetturale importante. Il backend gestisce utenti, dispositivi, risorse e logica applicativa, mentre OPA si occupa della decisione di accesso. In questo modo, le regole di autorizzazione non sono disperse all'interno del codice applicativo, ma sono definite in modo centralizzato e dichiarativo.

Questa impostazione presenta diversi vantaggi:

- le policy possono essere modificate senza riscrivere la logica del backend;
- le decisioni sono più facilmente testabili;
- le regole sono più leggibili e verificabili;
- la logica di sicurezza risulta separata dalla logica applicativa;
- il sistema è più vicino a un modello reale di governance delle policy.

Nel flusso principale, Envoy intercetta la richiesta e la invia a OPA tramite il meccanismo `ext_authz`. OPA valuta gli attributi ricevuti e restituisce una decisione. Envoy applica poi tale decisione, inoltrando la richiesta al backend solo in caso di esito positivo.

5.3 Policy Enforcement Point e Policy Decision Point

All'interno del progetto è possibile distinguere due ruoli fondamentali:

- **Policy Enforcement Point**, rappresentato da Envoy;
- **Policy Decision Point**, rappresentato da OPA.

Envoy non contiene direttamente la logica delle policy. Il suo compito è intercettare la richiesta, raccogliere gli attributi necessari e chiedere a OPA se l'accesso debba essere consentito. OPA, invece, non inoltra direttamente la richiesta al backend: si limita a produrre una decisione.

Questa separazione è rappresentata nella Figura logica seguente:

Client → Envoy PEP → OPA PDP → Decisione → Envoy → Backend

Questa distinzione è coerente con i modelli Zero Trust moderni, nei quali la decisione e l'applicazione della decisione vengono separate. Ciò permette di centralizzare il ragionamento sulle policy, lasciando al proxy il compito di applicare materialmente il risultato.

5.4 Struttura delle policy Rego

Le policy del progetto sono scritte in **Rego**, il linguaggio dichiarativo utilizzato da OPA. Per mantenere il codice più ordinato, le regole sono state suddivise in più file, ciascuno dedicato a un aspetto specifico della valutazione.

La struttura logica delle policy può essere riassunta come segue:

File Rego	Funzione principale
`access.rego`	Definisce la decisione finale di accesso e combina i controlli sugli altri moduli.
`user.rego`	Contiene i controlli relativi all'utente, al ruolo, al dipartimento, all'azione richiesta e all'MFA.
`device.rego`	Contiene i controlli relativi al dispositivo, come stato, affidabilità, sistema operativo, zona di rete e fingerprint.
`risk.rego`	Contiene le regole per la valutazione del rischio, la fascia oraria e l'eventuale presenza dell'utente in blocklist.
`helpers.rego`	Contiene funzioni di supporto per estrarre attributi da header, token, certificato e input della richiesta.

Tabella 5: Organizzazione logica delle policy Rego.

Questa separazione evita di concentrare tutta la logica in un unico file. La policy principale può quindi limitarsi a combinare controlli più specifici, rendendo il codice più leggibile e più facilmente estendibile.

5.5 Decisione finale di accesso

La decisione finale viene definita nella policy principale. L'accesso viene consentito solo se tutte le condizioni richieste risultano soddisfatte. La logica generale può essere espressa nel modo seguente:

allow = utente valido + azione consentita + ruolo autorizzato + device trusted + rischio accettabile + contesto conforme

Questa impostazione è volutamente restrittiva: se anche una sola condizione non viene rispettata, l'accesso viene negato. Tale comportamento riflette il principio Zero Trust secondo cui la fiducia non deve essere concessa implicitamente, ma deve essere dimostrata a ogni richiesta.

Tra le condizioni considerate nella decisione finale rientrano:

- utente non bloccato;
- ruolo tra quelli autorizzati per la risorsa;
- azione richiesta consentita;
- dipartimento compatibile;
- attributo MFA coerente con la policy;
- dispositivo fidato;
- dispositivo in stato attivo;
- sistema operativo supportato;
- IP appartenente a una zona consentita;
- fingerprint JA3 non bloccato;
- accesso in orario consentito;
- punteggio di rischio entro la soglia prevista.

La decisione finale non dipende quindi da un controllo statico, ma da un insieme di attributi che descrivono il contesto complessivo della richiesta.

5.6 Controlli sull'utente

Il modulo relativo all'utente verifica che l'identità autenticata sia compatibile con la risorsa richiesta. In particolare, vengono analizzati attributi come ruolo, dipartimento, azione e MFA.

Il ruolo è uno degli attributi principali. Ad esempio, una risorsa sensibile come `employee_records` può richiedere ruoli più privilegiati rispetto a una risorsa pubblica come `public_products`. Tuttavia, il ruolo da solo non è sufficiente. Due utenti con lo stesso ruolo potrebbero appartenere a dipartimenti differenti o avere stati di sicurezza diversi.

La valutazione dell'utente considera quindi:

- ruolo dell'utente;
- dipartimento;
- azione richiesta;
- stato dell'MFA;
- eventuale presenza dell'utente in blocklist.

Nel progetto, la MFA viene rappresentata tramite un attributo della richiesta, ad esempio `mfa_verified`. Tale attributo non corrisponde a un meccanismo MFA realmente implementato, ma viene utilizzato per simulare nelle policy la presenza o l'assenza di una verifica multifattore. In questo modo è possibile testare come cambierebbe la decisione di accesso nel caso in cui una risorsa o un'azione richiedesse un livello di autenticazione più elevato.

Questa scelta consente di differenziare il livello di protezione in base alla sensibilità della risorsa.

5.7 Controlli sul dispositivo

Il modulo relativo al dispositivo verifica che la richiesta provenga da un device riconosciuto e coerente con le policy. In un'architettura Zero Trust, infatti, l'identità dell'utente non basta: anche il dispositivo deve essere valutato.

Le principali condizioni considerate sono:

- dispositivo trusted;
- stato del dispositivo pari ad `active`;
- sistema operativo consentito;
- indirizzo IP appartenente a una zona ammessa;
- fingerprint JA3 non presente tra quelli bloccati;
- coerenza tra certificato client e dispositivo registrato.

Il controllo sul dispositivo è particolarmente importante perché impedisce che un utente autenticato possa accedere da un device sconosciuto o compromesso. Anche se le credenziali dell'utente sono corrette, il sistema può negare la richiesta se il dispositivo non soddisfa i requisiti previsti.

Nel progetto, questa logica si collega al processo di enrollment descritto nel Capitolo 5. Solo i dispositivi registrati, approvati e sincronizzati con il motore di policy possono essere considerati affidabili.

5.8 Controlli sul contesto di rete

Oltre a utente e dispositivo, il sistema valuta anche il contesto di rete della richiesta. In particolare, vengono analizzati attributi come:

- indirizzo IP sorgente;
- zona di rete;
- fingerprint JA3;
- eventuale provenienza da rete interna, VPN, esterna o sconosciuta.

La zona di rete permette di distinguere richieste provenienti da ambienti differenti. Una richiesta proveniente dalla rete interna o da VPN può essere considerata meno rischiosa rispetto a una richiesta proveniente da una rete esterna non riconosciuta. Tuttavia, anche la rete interna non viene considerata automaticamente sicura: essa rappresenta solo uno degli attributi valutati.

La fingerprint JA3 consente invece di caratterizzare il client TLS sulla base di alcuni parametri del handshake. Nel progetto viene utilizzata come ulteriore elemento di controllo, soprattutto nei casi in cui non sia disponibile una protezione hardware-bound forte.

L'obiettivo non è fidarsi ciecamente della rete, ma usare il contesto di rete come segnale aggiuntivo all'interno di una decisione più ampia.

5.9 Motivi di negazione

Un aspetto utile dell'implementazione è la presenza di motivi espliciti di negazione. Quando una richiesta viene rifiutata, OPA non restituisce soltanto un generico `deny`, ma può indicare le condizioni che non sono state soddisfatte.

Alcuni esempi di motivi di rifiuto sono:

- `role_not_allowed`;
- `action_not_allowed`;
- `mfa_required`;
- `department_not_allowed`;
- `untrusted_device`;
- `device_not_active`;
- `ja3_fingerprint_blocked`;

- `ip_not_in_allowed_zone`;
- `unsupported_os`;
- `risk_score_too_high`;
- `outside_working_hours`;
- `user_blocked_by_barac`.

Queste motivazioni sono importanti per almeno tre ragioni. In primo luogo, aiutano durante la fase di sviluppo e debug, perché permettono di capire rapidamente quale condizione abbia causato il rifiuto. In secondo luogo, migliorano la trasparenza del sistema, poiché la decisione non è opaca. Infine, sono utili in ottica di audit, perché consentono di ricostruire le cause di un accesso negato.

5.10 Esempio di risposta decisionale

La risposta di OPA può essere strutturata in modo da includere non solo l'esito, ma anche informazioni contestuali sulla richiesta. Un esempio semplificato è il seguente:

Esempio semplificato di decisione OPA

```
{
  allow: false,
  user: demo_customer,
  resource: employee_records,
  action: read,
  risk_score: 0.35,
  reasons: [
    role_not_allowed,
    department_not_allowed
  ]
}
```

In questo caso l'accesso viene negato non perché il rischio sia eccessivo, ma perché il ruolo e il dipartimento dell'utente non sono compatibili con la risorsa richiesta. Questo esempio mostra come il sistema possa distinguere le diverse cause di rifiuto.

Un accesso consentito può invece essere rappresentato in questo modo:

Esempio semplificato di accesso consentito

```
{
  allow: true,
  user: alice_hr,
  resource: employee_records,
  action: read,
  risk_score: 0.12,
  reasons: []
}
```

La decisione positiva è possibile solo perché tutte le condizioni richieste sono soddisfatte.

5.11 Valutazione del rischio

La valutazione del rischio rappresenta una delle parti più importanti del progetto. In un sistema Zero Trust, l'accesso non dovrebbe essere deciso soltanto sulla base di policy statiche, ma anche considerando il comportamento osservato nel tempo, il contesto della richiesta e la criticità della risorsa richiesta.

Nel progetto, il rischio viene calcolato combinando più componenti. In particolare, viene considerata una componente applicativa, derivata dagli eventi di accesso e dalle decisioni allow/deny, e una componente di rete, alimentata anche dagli alert prodotti da Snort. In forma semplificata, il rischio può essere interpretato come:

$$\textbf{Rischio} = \textbf{Probabilità} \times \textbf{Impatto}$$

La probabilità viene stimata considerando il valore più significativo tra la componente applicativa e quella di rete:

```
probability = max(P_app, P_net)
```

La componente `P_app` è legata al comportamento applicativo dell'utente, come numero di richieste negate, deny recenti, deny su risorse privilegiate, variazione degli IP e rapporto tra richieste negate e richieste totali. La componente `P_net`, invece, rappresenta il rischio derivante da eventi di rete, in particolare dagli alert generati da Snort e raccolti in Splunk.

Il rischio complessivo tiene poi conto dell'impatto della risorsa e dell'azione richiesta. Una richiesta di lettura su una risorsa pubblica ha un impatto inferiore rispetto a una richiesta di scrittura su dati sensibili. Per questo motivo, il progetto utilizza lookup Splunk come `asset_weights.csv` e `resource_impact.csv`, utili ad associare pesi e criticità alle diverse risorse e operazioni.

Questa impostazione consente di ottenere una valutazione più realistica: il rischio non dipende soltanto dal comportamento dell'utente, ma anche dal valore della risorsa richiesta e dal contesto in cui avviene l'accesso.

5.12 Risk score e soglie di accesso

Il *risk score* viene rappresentato come un valore numerico. Le policy confrontano questo valore con una soglia massima associata alla risorsa o alla richiesta. Se il rischio supera la soglia prevista, l'accesso viene negato.

La logica può essere rappresentata come segue:

Accesso consentito solo se

```
risk_score <= max_risk_threshold
```

Questa impostazione consente di applicare policy diverse a risorse diverse. Una risorsa ad alta sensibilità può richiedere un rischio molto basso, mentre una risorsa pubblica può tollerare una soglia più elevata. Nel progetto è prevista anche una soglia più restrittiva per i dispositivi che utilizzano identità software di fallback, distinguendoli dai dispositivi che impiegano una protezione hardware-bound tramite TPM o Secure Enclave. In questo caso, non essendo presente una protezione hardware-bound tramite TPM o Secure Enclave, il sistema può applicare un limite di rischio più basso, ad esempio richiedendo che il valore di `risk_score` sia inferiore o uguale a `0.25`.

In questo modo, il sistema non applica una regola uniforme a tutte le risorse, ma adatta la severità del controllo al valore e alla criticità dei dati richiesti.

5.13 Endpoint di valutazione del rischio

Il backend espone un endpoint dedicato alla valutazione del rischio:

```
POST /api/risk/evaluate
```

Questo endpoint riceve informazioni relative alla richiesta e restituisce un punteggio di rischio. Tra gli attributi considerati possono essere presenti:

- username;
- ruolo;
- dipartimento;
- risorsa richiesta;
- azione;
- stato del dispositivo;
- sistema operativo;
- indirizzo IP;
- fingerprint JA3;

- zona di rete.

La risposta può contenere informazioni come:

- `risk_score`;
- `severity`;
- sorgente della valutazione;
- numero di deny recenti;
- numero di allow recenti;
- numero di IP distinti;
- numero di fingerprint JA3 distinti.

In questo modo, il motore di policy può utilizzare un valore di rischio aggiornato invece di affidarsi soltanto a valori statici o dichiarati dal client.

5.14 Integrazione tra OPA e servizio di rischio

Nel progetto, OPA può interagire con il backend per ottenere una valutazione aggiornata del rischio. La policy costruisce una richiesta verso il servizio di rischio e utilizza il valore restituito per completare la decisione di accesso.

Nel file `risk.rego`, OPA può richiedere una valutazione aggiornata al backend attraverso l'endpoint interno:

```
http://backend:3000/api/risk/evaluate
```

In questo modo, la decisione di accesso può utilizzare un valore di rischio calcolato dinamicamente, invece di basarsi esclusivamente su un valore statico ricevuto tramite header.

Il flusso può essere descritto così:

1. Envoy invia a OPA una richiesta di autorizzazione;
2. OPA estrae gli attributi della richiesta;
3. OPA invoca il servizio di rischio del backend;
4. il backend calcola o recupera il risk score;
5. OPA confronta il risk score con la soglia prevista;
6. se il rischio è accettabile, la valutazione prosegue;
7. se il rischio è troppo alto, la richiesta viene negata.

Questa integrazione rende la policy più dinamica. Invece di valutare soltanto dati statici, OPA può basarsi su informazioni aggiornate e su eventi osservati nel tempo.

5.15 Fallback prudenziale

Un aspetto importante riguarda il comportamento del sistema nel caso in cui il servizio di rischio non sia disponibile o non restituisca dati validi. In un'architettura di sicurezza, un errore nella valutazione non dovrebbe trasformarsi automaticamente in un accesso consentito.

Per questo motivo il progetto adotta una logica di fallback prudenziale. Se il rischio non può essere calcolato correttamente, il sistema utilizza un valore conservativo oppure considera la richiesta più rischiosa.

Questa scelta è coerente con il principio di sicurezza *fail closed*: in caso di errore o incertezza, il sistema tende a negare o limitare l'accesso, invece di consentirlo senza una valutazione completa.

Quando non è disponibile una valutazione live del rischio, il sistema può utilizzare un valore di fallback proveniente dagli attributi della richiesta. Tuttavia, tale valore viene trattato in modo prudente, soprattutto nel caso di dispositivi non hardware-bound. Questo evita che l'assenza temporanea del servizio di rischio diventi un modo per ottenere accessi meno controllati.

5.16 Ruolo di Splunk nella valutazione del rischio

Splunk viene utilizzato come sistema di raccolta, indicizzazione e analisi degli eventi. I log generati da backend, Envoy, OPA, nftables e Snort vengono raccolti e indicizzati, rendendo possibile osservare il comportamento del sistema nel tempo.

Nel progetto, Splunk non ha soltanto una funzione di osservabilità, ma contribuisce direttamente alla valutazione dinamica del rischio. Gli eventi applicativi permettono di analizzare il comportamento degli utenti, mentre gli alert di Snort forniscono un segnale relativo al rischio di rete.

La componente applicativa del rischio può considerare, ad esempio:

- numero totale di richieste negate;
- numero di deny concentrati in una finestra temporale;
- deny relativi a risorse privilegiate;
- deny legati all'autenticazione;
- numero di indirizzi IP distinti;
- rapporto tra richieste negate e richieste totali.

La componente di rete, invece, può essere alimentata dagli alert IDS prodotti da Snort. In questo modo, una richiesta può risultare più rischiosa non solo perché l'utente ha prodotto molti deny applicativi, ma anche perché il traffico associato presenta caratteristiche sospette.

Splunk supporta inoltre saved searches e lookup dedicati al progetto. I lookup `asset_weights.csv` e `resource_impact.csv` permettono di associare valori di peso e impatto alle risorse e alle azioni, mentre le ricerche salvate consentono di automatizzare il calcolo del rischio e l'attivazione di webhook verso il backend.

Questa integrazione consente di passare da una logica puramente statica a una logica più dinamica e contestuale, nella quale le decisioni future possono essere influenzate dal comportamento osservato.

5.17 Splunk AI Toolkit e Random Forest

Per stimare la probabilità di comportamento anomalo, il progetto integra anche lo **Splunk AI Toolkit**.

Il modello non sostituisce le policy Rego, ma fornisce un segnale aggiuntivo utilizzabile nella valutazione del rischio. Le policy continuano infatti a stabilire le condizioni minime di accesso, mentre il modello contribuisce a individuare pattern sospetti che potrebbero non emergere da semplici regole statiche.

Tra le feature considerate dal modello possono rientrare:

- numero di richieste negate;
- deny recenti;
- deny su risorse ad alto privilegio;
- deny di autenticazione;
- numero di IP distinti;
- rapporto tra richieste negate e richieste totali.

L'output del modello rappresenta una probabilità di comportamento anomalo, utilizzabile come componente applicativa del rischio, indicata come `P_app`. Tale valore può poi essere confrontato o combinato con la componente di rete `P_net`, ottenuta dagli eventi IDS, e con l'impatto della risorsa richiesta.

In questo modo, il sistema integra policy dichiarative e analisi comportamentale, mantenendo però una separazione chiara tra decisione autorizzativa e stima del rischio.

5.18 Audit log e osservabilità

L'audit log è un elemento essenziale in un'architettura Zero Trust. Ogni decisione di accesso dovrebbe essere ricostruibile a posteriori, indicando chi ha richiesto cosa, da quale dispositivo, in quale contesto e con quale esito.

Nel progetto vengono registrate informazioni come:

- utente;
- ruolo;

- dipartimento;
- risorsa richiesta;
- azione;
- decisione;
- motivazione;
- risk score;
- indirizzo IP;
- fingerprint JA3;
- stato del dispositivo;
- timestamp della richiesta.

Queste informazioni permettono di analizzare il comportamento del sistema e di individuare eventuali pattern sospetti. Inoltre, consentono di dimostrare il rispetto delle policy applicate e di ricostruire le cause di eventuali accessi negati.

5.19 Logica BARAC

Nel progetto è presente una logica riconducibile al modello **BARAC**, cioè un approccio di controllo degli accessi basato sul confronto tra rischio e beneficio. L'idea è che una decisione di accesso non dipenda soltanto dal rischio tecnico, ma anche dal valore dell'operazione richiesta e dall'utilità attesa.

Nel progetto, questa logica viene supportata da Splunk attraverso saved searches dedicate. In particolare, la ricerca `BARAC_RiskProfit_RealTime` consente di stimare il bilancio tra rischio e beneficio, mentre `BARAC_BlockUser_Webhook` può attivare il blocco adattivo di un utente tramite webhook verso il backend. La ricerca `BARAC_Dashboard_Summary` fornisce invece una sintesi utile per l'osservazione del comportamento complessivo.

In forma semplificata, il bilancio può essere espresso come:

```
profit = total_benefit - total_risk
```

Se il profitto risulta negativo, significa che il rischio complessivo dell'operazione supera il beneficio stimato. In questo caso, il sistema può decidere di bloccare l'utente o impedire ulteriori richieste.

Questa logica è particolarmente interessante perché rende il sistema più adattivo. Non tutte le richieste hanno lo stesso peso: accedere a dati pubblici non equivale ad accedere a record HR sensibili o a risorse finanziarie. Il controllo di accesso deve quindi tenere conto sia della pericolosità del contesto sia del valore della risorsa.

5.20 Webhook Splunk e blocco adattivo

Il progetto prevede una rotta dedicata alla ricezione di webhook da Splunk:

```
POST /api/splunk-webhook
```

Questa rotta consente a Splunk di notificare al backend la presenza di condizioni anomale. Ad esempio, se un utente accumula troppe richieste negate o presenta un comportamento ritenuto sospetto, Splunk può inviare un evento al backend.

Il backend verifica che la richiesta provenga da una sorgente autorizzata controllando un segreto condiviso nell'header:

```
x-splunk-webhook-secret
```

Se il segreto è valido e il payload è corretto, il backend aggiorna la lista degli utenti bloccati.

Un esempio semplificato di payload può essere:

Esempio di payload inviato da Splunk

```
{
  username: demo_customer,
  action: block,
  reason: too_many_denied_requests,
  risk_total: 0.85,
  deny_count: 12,
  source: splunk
}
```

Il webhook viene quindi utilizzato come meccanismo di risposta automatica: Splunk rileva una condizione anomala, calcola un livello di rischio o un profitto negativo secondo la logica BARAC e invia al backend un comando di blocco. Il backend aggiorna la collection `blocked_users` e sincronizza l'informazione con il motore di policy.

Questo meccanismo consente al sistema di reagire automaticamente a eventi osservati nel tempo, passando da un modello statico a un modello adattivo.

5.21 Blocklist degli utenti

Gli utenti segnalati da Splunk possono essere inseriti in una blocklist. La blocklist viene salvata nel database e sincronizzata con il motore di policy, in modo che OPA possa utilizzarla durante le decisioni successive.

Il flusso può essere riassunto così:

1. Splunk rileva un comportamento sospetto;
2. Splunk invia un webhook al backend;
3. il backend valida il segreto della richiesta;
4. il backend aggiorna la collection degli utenti bloccati;
5. il backend sincronizza la blocklist con OPA;
6. OPA nega le richieste successive dell'utente bloccato.

Questa logica permette di introdurre una risposta adattiva. L'utente non viene bloccato manualmente nel codice, ma può essere bloccato in seguito a eventi osservati e analizzati.

5.22 Aggiornamento dinamico delle decisioni

Una caratteristica importante del progetto è che alcune decisioni possono cambiare nel tempo. Un utente autorizzato potrebbe essere bloccato dopo comportamenti sospetti. Un dispositivo precedentemente trusted potrebbe essere revocato. Una richiesta normalmente consentita potrebbe essere negata se il rischio cresce oltre la soglia prevista.

Questa dinamicità è coerente con la Zero Trust Architecture. La fiducia non è permanente, ma deve essere verificata continuamente. Gli attributi che contribuiscono alla decisione possono cambiare e il sistema deve essere in grado di riflettere tali cambiamenti.

Tra gli elementi dinamici del progetto rientrano:

- stato del dispositivo;
- lista dei dispositivi trusted;
- risk score;
- blocklist degli utenti;
- eventi raccolti da Splunk;
- motivi recenti di deny;
- fingerprint o IP osservati.

Questo rende il sistema più realistico rispetto a un semplice modello RBAC statico.

5.23 Confronto tra RBAC, ABAC e approccio adottato

Il progetto non utilizza soltanto un modello RBAC tradizionale. Nel controllo di accesso basato sui ruoli, la decisione dipende principalmente dal ruolo assegnato all'utente. Questo modello è semplice e diffuso, ma può risultare insufficiente in scenari dinamici.

ZeroTrustHR adotta invece una logica più vicina ad ABAC, cioè *Attribute-Based Access Control*. La decisione dipende da più attributi:

- ruolo;
- dipartimento;
- azione;
- risorsa;
- dispositivo;
- rete;
- MFA;
- rischio;
- orario;
- stato dell'utente.

La Tabella 6 riassume la differenza.

Modello	Caratteristiche
RBAC	Decisione basata principalmente sul ruolo dell'utente. Semplice, ma poco flessibile in contesti dinamici.
ABAC	Decisione basata su attributi multipli, come ruolo, risorsa, dispositivo, rete, rischio e contesto.
ZeroTrustHR	Approccio ibrido e adattivo: utilizza ruolo e attributi contestuali, integrando anche rischio dinamico e blocklist.

Tabella 6: Confronto tra RBAC, ABAC e approccio adottato nel progetto.

Questa scelta permette di ottenere una granularità maggiore e di applicare controlli più aderenti al contesto reale della richiesta.

5.24 Esempi di scenari decisionali

Per comprendere meglio il funzionamento del sistema, è utile descrivere alcuni scenari.

5.24.1 Accesso consentito

Un utente del dipartimento HR, con MFA abilitata, da un dispositivo trusted e attivo, richiede la lettura di `employee_records`. Il rischio è basso e la richiesta avviene in un orario consentito. In questo caso tutte le condizioni sono soddisfatte e OPA restituisce `allow`.

5.24.2 Accesso negato per ruolo non autorizzato

Un utente con ruolo `customer` prova ad accedere a `employee_records`. Anche se l'utente fosse autenticato e il rischio fosse basso, il ruolo non è compatibile con la risorsa. La richiesta viene quindi negata con motivo `role_not_allowed`.

5.24.3 Accesso negato per dispositivo non fidato

Un utente HR prova ad accedere a una risorsa sensibile da un dispositivo non registrato o non approvato. Il ruolo è corretto, ma il dispositivo non è trusted. La richiesta viene negata con motivo `untrusted_device`.

5.24.4 Accesso negato per rischio elevato

Un utente normalmente autorizzato effettua molte richieste negate in breve tempo o cambia spesso IP e fingerprint. Splunk osserva il comportamento e il risk score supera la soglia prevista. Anche se ruolo e dispositivo sono corretti, la richiesta viene negata per rischio eccessivo.

5.24.5 Accesso negato per blocklist

Un utente viene inserito nella blocklist a seguito di una segnalazione Splunk. Le richieste successive vengono negate direttamente da OPA con motivo `user_blocked_by_barac`.

5.25 Sintesi dei controlli decisionali

La Tabella 7 riassume i principali controlli usati nel processo decisionale.

Controllo	Descrizione
Ruolo	Verifica che il ruolo dell'utente sia autorizzato ad accedere alla risorsa.
Azione	Controlla che l'azione richiesta sia consentita.
Dipartimento	Verifica la compatibilità tra dipartimento dell'utente e risorsa.
MFA	Considera l'attributo `mfa_verified` come segnale contestuale nelle policy, senza implementare un vero flusso MFA.
Dispositivo	Verifica che il device sia trusted, attivo e coerente con le policy.
Rete	Considera IP, zona di rete e fingerprint JA3 come attributi contestuali.
Soglia di rischio	Confronta il risk score con la soglia massima prevista.
Orario	Verifica che l'accesso avvenga in una finestra temporale consentita.
Rischio applicativo	Considera eventi applicativi, deny recenti, deny privilegiati, IP distinti e rapporto tra deny e richieste totali.
Rischio di rete	Considera gli alert IDS prodotti da Snort e raccolti in Splunk.
Impatto della risorsa	Pesa la criticità della risorsa e dell'azione richiesta tramite lookup dedicati.
Blocklist	Nega l'accesso a utenti bloccati in seguito ad anomalie o eventi Splunk.

Tabella 7: Principali controlli decisionali utilizzati nel progetto.

5.26 Considerazioni finali

Il sistema di policy e valutazione del rischio rappresenta il nucleo Zero Trust del progetto. L'accesso non viene concesso sulla base di una fiducia implicita, ma attraverso una valutazione composta da più attributi: utente, ruolo, dispositivo, rete, risorsa, azione, MFA, rischio e comportamento osservato.

L'utilizzo di OPA e Rego permette di separare la logica decisionale dal backend, rendendo le policy più leggibili, modificabili e testabili. L'integrazione con Splunk, Snort, Splunk AI Toolkit e logica BARAC introduce inoltre un elemento adattivo, poiché il sistema può modificare le decisioni future sulla base degli eventi rilevati, del rischio stimato e del bilancio tra rischio e beneficio.

Nel complesso, il progetto mostra come un'architettura Zero Trust non sia semplicemente un insieme di strumenti, ma un modello di controllo continuo. Ogni richiesta viene analizzata nel proprio contesto e l'accesso viene consentito solo quando tutte le condizioni richieste risultano soddisfatte.



Integrazione SIEM e Machine Learning

6.1 Introduzione

L'architettura Zero Trust descritta nei capitoli precedenti basa la sua efficacia sulla capacità di valutare il rischio in modo dinamico e adattivo. Questo capitolo illustra nel dettaglio come il sistema SIEM (Security Information and Event Management), implementato tramite **Splunk**, è stato strutturato per raccogliere i dati, analizzarli attraverso tecniche di Machine Learning e validare l'intera infrastruttura attraverso simulazioni di attacco.

6.2 Architettura e Workflow Analitico di Splunk

All'interno del progetto, Splunk non agisce come un semplice archivio passivo di log, ma costituisce il nucleo dell'osservabilità e dell'intelligenza analitica. L'architettura è stata progettata affinché i dati attraversino una pipeline in quattro fasi (Ingestion, Normalizzazione, Analisi e Risposta).

Il workflow dei dati è strutturato come segue:

1. **Ingestion Asincrona (HEC):** I componenti distribuiti (backend Node.js, proxy Envoy, motore OPA e IDS Snort) comunicano con il SIEM in modalità *push* tramite il protocollo *HTTP Event Collector* (HEC). Questo meccanismo, protetto da token di autorizzazione, garantisce una trasmissione sicura e performante di log in formato JSON nativo, azzerando l'impatto prestazionale sui microservizi.

2. **Isolamento Logico (Indici):** Tutto il traffico di eventi viene centralizzato e isolato all'interno di un singolo indice dedicato (`zerotrust`). Questa scelta architettonica ottimizza i tempi di elaborazione delle query in fase di Machine Learning e isola i dati di sicurezza da flussi spazzatura.

3. **Normalizzazione e Tassonomia (Sourcetype):** Per facilitare le operazioni di *Data Wrangling* propedeutiche all'addestramento dei modelli, gli eventi sono fortemente tipizzati alla sorgente. Le decisioni di accesso prese da OPA confluiscono sotto il sourcetype `opa_decision`, i log del proxy Envoy tracciano il traffico mTLS, mentre gli alert di rete di Snort vengono raccolti in formato `_json`.

4. **Chiusura del Ciclo (Webhook):** Una volta indicizzati, i dati vengono attivamente monitorati. Splunk funge da supervisore proattivo: interroga i dati e, al rilevamento di una condizione anomala aggregata (es. una violazione ripetuta registrata su OPA o un alert di rete di Snort), chiude il ciclo Zero Trust innescando un webhook sicuro verso il backend, andando ad aggiornare in tempo reale le policy di negazione.

Questa pipeline trasforma Splunk da un semplice collettore a un motore reattivo, fornendo fondamenta solide, normalizzate e in tempo reale per l'addestramento dei modelli predittivi e la difesa dell'architettura.

Da un punto di vista implementativo, l'intera configurazione del SIEM è stata ingegnerizzata seguendo il paradigma *Configuration as Code* (CaC). È stata sviluppata un'applicazione Splunk custom denominata `zerotrust_app` che raccoglie in modo dichiarativo le impostazioni degli indici (`indexes.conf`), dei ricevitori HEC (`inputs.conf`), delle regole di *parsing* (`props.conf`) e delle ricerche salvate (`savedsearches.conf`).

Questo approccio garantisce la totale riproducibilità dell'ambiente di sicurezza e il perfetto versionamento su Git dell'infrastruttura di monitoraggio.

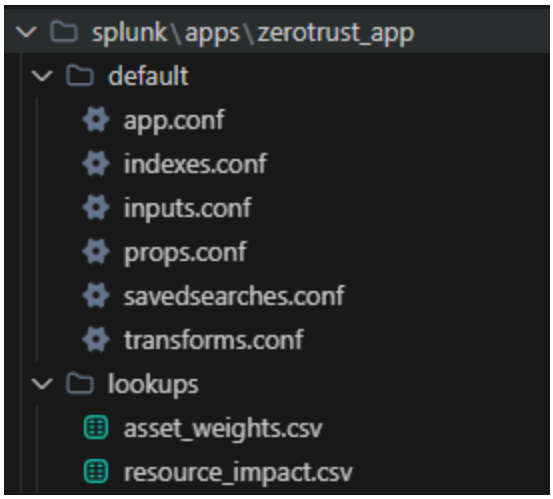


Figura 15: Struttura della directory dell'applicazione custom `zerotrust_app` utilizzata per il *Configuration as Code*.

Come illustrato nella Figura 15, l'applicazione è composta dai seguenti file di configurazione principali:

- `app.conf`: Definisce i metadati dell'applicazione, il suo stato di attivazione e la visibilità nell'interfaccia utente di Splunk.
- `indexes.conf`: Dichiara l'indice dedicato `zerotrust`, configurandone le policy di ritenzione dei dati e i percorsi di storage isolati, garantendo che i log di sicurezza non si mescolino con altri dati.
- `inputs.conf`: Abilita e configura il ricevitore *HTTP Event Collector* (HEC), esponendo le porte di ascolto e associando i token crittografici necessari per l'ingestion sicura in modalità *push*.
- `props.conf` e `transforms.conf`: Definiscono le regole di estrazione, normalizzazione e *parsing* (Data Wrangling). Garantiscono che i complessi JSON generati da OPA ed Envoy vengano interpretati correttamente e arricchiti con dati di contesto.
- `savedsearches.conf`: Contiene le query SPL pre-salvate e le regole di allerta (come quelle per il calcolo del Rischio di Rete e l'innesco automatico dei Webhook verso il backend), rendendo il SIEM operativo e reattivo in background in modo totalmente automatizzato.
- `lookups/`: Cartella contenente i file CSV statici (`asset_weights.csv` e `resource_impact.csv`), impiegati a runtime per risolvere dinamicamente l'impatto aziendale associato ad ogni specifica risorsa, disaccoppiando la metrica dal codice sorgente.

6.3 Il Modello di Machine Learning: Random Forest

Al fine di superare i limiti delle policy statiche (RBAC/ABAC), l'architettura integra lo **Splunk AI Toolkit (AITK)** per applicare algoritmi di Machine Learning al rilevamento delle minacce. L'obiettivo è calcolare la *Probabilità Applicativa* (P_{app}), ovvero la probabilità che una specifica identità utente sia stata compromessa o stia tenendo un comportamento malevolo.

L'algoritmo scelto per questo scopo è il **Random Forest Classifier**. La scelta ricade su questo modello supervisionato per la sua eccellente capacità di gestire relazioni non lineari tra molteplici variabili e per la sua intrinseca resistenza all'overfitting.

Il modello viene addestrato estraendo dai log di OPA (`opa_decision`) una serie di *feature* ingegnerizzate che descrivono il comportamento dell'utente nel tempo. Tra le feature principali rientrano:

- **Total Denies e Deny Ratio:** Il numero assoluto di richieste negate e il rapporto tra richieste bloccate e richieste totali. Un utente legittimo commette errori raramente, mentre un attaccante genera un alto volume di rifiuti.
- **Motivi di Rifiuto Specifici:** Il sistema estrae e conteggia separatamente le ragioni del blocco. I rifiuti legati a violazioni di privilegi (`role_not_allowed`, `department_not_allowed`) o ad anomalie temporali (`outside_working_hours`) hanno un peso classificatorio diverso.

- **Variazione del Contesto:** Il numero di indirizzi IP distinti (`distinct_ips`) utilizzati dallo stesso utente in un breve lasso di tempo. Un repentino cambio di IP è un forte indicatore di compromissione delle credenziali.

Sulla base di queste feature, il modello Random Forest classifica l'utente restituendo una probabilità di anomalia (P_{app}) compresa tra 0 e 1, che viene dinamicamente interrogata dal backend per ogni nuova richiesta di accesso.

Di seguito è riportato la query in linguaggio SPL (Splunk Processing Language) utilizzata per estrarre le feature descritte e avviare la fase di addestramento:

Query SPL per l'estrazione delle feature e l'addestramento del modello Random Forest

```
search (index=zerotrust sourcetype=opa_decision)
| spath input=_raw path=line.result.allowed output=allowed
| spath input=_raw path=line.result.user output=user_id
| eval is_opa=1
| eval is_deny=if(allowed=false, 1, 0)
| eval reason_time = if(is_deny=1 AND match('line.result.deny_reasons{}',
    outside_working_hours), 1, 0)
| eval reason_priv = if(is_deny=1 AND match('line.result.deny_reasons{}',
    role_not_allowed|action_not_allowed|department_not_allowed), 1, 0)
| eval reason_auth = if(is_deny=1 AND match('line.result.deny_reasons{}',
    untrusted_device|device_not_active|mfa_required|ja3_fingerprint_blocked|
    ip_not_in_allowed_zone|unsupported_os), 1, 0)
| eval device_ip_header = 'line.input.attributes.request.http.headers.x-device-ip'
| eval src_ip = coalesce(device_ip_header, src_ip, src)
| stats
    sum(is_deny) AS total_denies
    sum(reason_time) AS time_denies
    sum(reason_priv) AS priv_denies
    sum(reason_auth) AS auth_denies
    count(eval(is_opa=1)) AS total_opa_requests
    dc(eval(if(is_opa=1, src_ip, null()))) AS distinct_ips
    BY user_id
| eval deny_ratio = if(total_opa_requests > 0, total_denies / total_opa_requests, 0)
| eval is_attacker = if(user_id=eve_attacker OR user_id=oscar_attacker, 1, 0)
| fit RandomForestClassifier is_attacker from total_denies time_denies priv_denies
    auth_denies distinct_ips deny_ratio into app_risk_model
```



Figura 16: Tabella delle *feature* estratte in fase di addestramento del modello Random Forest

Come si evince dalla Figura 16, il modello non si basa esclusivamente sul volume dei rifiuti, ma sulla loro natura contestuale. L'utente legittimo `alice_hr`, ad esempio, registra un altissimo numero di rifiuti totali (282, pari a un *deny ratio* del 99.6%) dovuti unicamente a tentativi di accesso fuori dall'orario lavorativo (`time_denies`). Il Random Forest, riconoscendo questo pattern come non ostile, assegna correttamente all'utente la previsione di sicurezza (`predicted(is_attacker) = 0`). Al contrario, l'utente `eve_attacker`, pur con un numero nettamente inferiore di fallimenti (49), concentra le sue violazioni sul tentativo di forzare i privilegi di accesso (`priv_denies`) ed eludere l'autenticazione, venendo pertanto classificato matematicamente come minaccia (`predicted = 1`).

6.4 Il Calcolo Combinato del Rischio

Nell'architettura proposta, il calcolo finale del *Risk Score* non dipende esclusivamente dal Machine Learning, ma è il risultato della fusione tra l'intelligenza applicativa e quella infrastrutturale. Il calcolo si basa su tre componenti:

1. P_{app} **(Probabilità Applicativa):** Il punteggio calcolato dal modello Random Forest in base allo storico comportamentale dell'utente.

2. P_{net} **(Probabilità di Rete):** Un valore di rischio assegnato dinamicamente a livello di rete. Se l'IDS (Snort) rileva un attacco proveniente da un determinato indirizzo IP (es. Port Scanning), il P_{net} per quell'IP viene innalzato istantaneamente a valori critici (es. 0.95), scavalcando l'analisi comportamentale.

3. **Impatto (*Impact*):** Una metrica definita dalla sensibilità della risorsa richiesta e dalla criticità dell'asset. All'interno dell'architettura, la classificazione degli asset e il relativo peso non sono cablati nel codice, ma vengono gestiti dinamicamente tramite file di *Lookup* in Splunk (`resource_impact.csv` e `asset_weights.csv` inclusi nella `zerotrust_app`). Questo design disaccoppiato consente agli analisti di aggiornare i livelli di criticità delle risorse aziendali senza dover alterare il backend applicativo o ricaricare le policy OPA.

La formula del rischio implementata nel motore di policy è definita come il massimo tra la probabilità applicativa e quella di rete, moltiplicato per l'impatto della risorsa:

$$Risk = \max(P_{app}, P_{net}) \times Impact$$

Se il rischio calcolato supera la soglia di tolleranza predefinita per la risorsa specifica, OPA nega l'accesso istantaneamente. Inoltre, in presenza di un volume di richieste talmente elevato da saturare il modello (es. attacco Denial of Service), il sistema applica una logica di **Fail-Closed**, assegnando precauzionalmente un rischio massimo (1.0) a tutto il traffico anomalo fino alla stabilizzazione del carico.

6.5 Validazione e Test dell'Architettura

Al fine di validare l'efficacia della logica Zero Trust, il sistema è stato sottoposto a un rigoroso processo di test suddiviso in due fasi sequenziali, volte a dimostrare la resilienza dell'infrastruttura contro attacchi.

6.5.1 Fase 1: Addestramento del Modello Predittivo

Nella prima fase, è stato necessario popolare il SIEM con dati storici per addestrare l'algoritmo di Machine Learning. A tale scopo è stato eseguito uno script automatizzato incaricato di generare un volume di traffico misto, composto all'80% da richieste legittime (comportamento standard dei dipendenti) e al 20% da attacchi mirati (es. utenti compromessi che tentano di accedere a risorse non autorizzate). Tutti i log generati sono stati analizzati su Splunk, permettendo al Random Forest di calcolare i pesi delle feature e costruire il modello predittivo `app_risk_model`. Durante questa fase di "apprendimento", il sistema mantiene un rischio di base neutro, in quanto l'intelligenza artificiale necessita di un campione statistico iniziale per differenziare la normalità dall'anomalia.

6.5.2 Fase 2: Intervento Dinamico in Tempo Reale

Una volta addestrato il modello, è stata avviata una seconda simulazione per osservare l'intervento dell'AI in tempo reale. I risultati hanno confermato il corretto funzionamento del principio di *Continuous Verification*:

- **Utenti legittimi:** Utenti con un comportamento coerente con le policy aziendali hanno mantenuto un punteggio di rischio estremamente basso ($P_{app} \approx 0.1$). Piccole infrazioni, come l'accesso fuori orario, hanno generato leggeri incrementi temporanei, senza mai superare la soglia di blocco per le risorse non critiche.

- **Utenti compromessi:** L'esecuzione di operazioni ostili (come tentativi ripetuti di accesso a dati sensibili da parte di dipendenti non autorizzati) ha causato un rapido incremento del rischio ($P_{app} \geq 0.5$). Raggiunta la soglia limite, OPA ha automaticamente bloccato le identità compromesse ancor prima che l'attacco potesse avere successo.

In questa fase è stata inoltre verificata con successo la risposta adattiva di *Fail-Closed* a fronte di un picco anomalo di richieste simultanee (DoS).

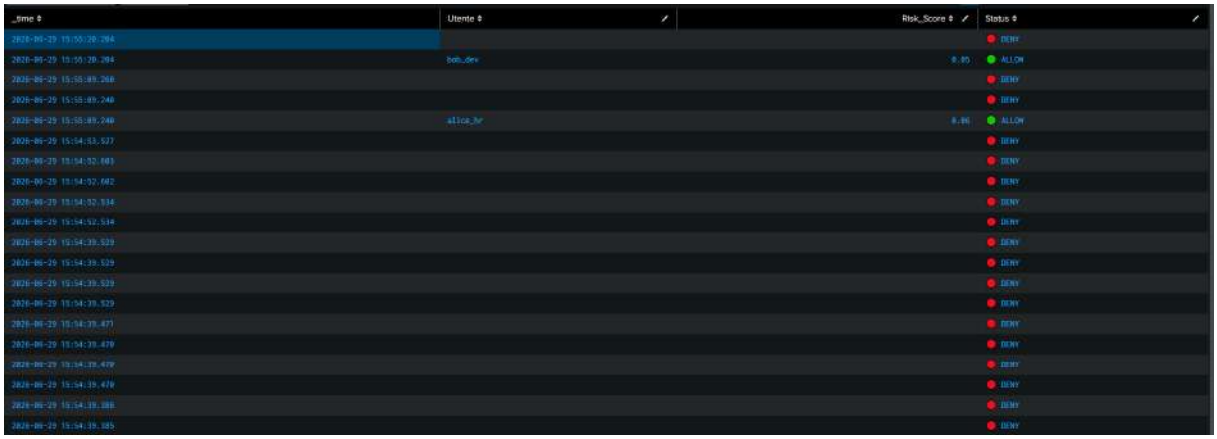


Figura 17: Dashboard Splunk: andamento del Risk Score durante le simulazioni di attacco applicativo.

La Figura 17 illustra l'intervento dinamico e reattivo del sistema. La tabella monitorata da Splunk mostra in tempo reale come l'utente `bob_dev` mantenga un *Risk Score* estremamente basso (0.05), ottenendo semaforo verde (`ALLOW`) da OPA. Le richieste in violazione, invece, registrano punteggi di rischio immediatamente superiori alla soglia di tolleranza, ricevendo un blocco incondizionato (`DENY`) che previene qualsiasi accesso non autorizzato alle risorse.



WSL2 e Configurazione nftables

7.1 Introduzione

In questa fase del progetto è stata configurata la componente host del laboratorio Zero Trust, basata su **Windows Subsystem for Linux 2 (WSL2)** con integrazione Docker Desktop. L'obiettivo principale è stato quello di:

- abilitare il forwarding a livello kernel Linux;
- installare e configurare nftables come firewall L3/L4;
- verificare il passaggio del traffico tra rete WAN simulata e rete Docker interna;
- validare il logging di eventi di rete generati da attacchi simulati tramite Kali Linux.

7.2 Configurazione del sistema host (WSL2)

Il sistema host utilizzato per l'intero laboratorio è basato su WSL2 con distribuzione Ubuntu. In questo ambiente il kernel Linux è virtualizzato tramite Windows, rendendo necessaria una configurazione esplicita per il routing dei pacchetti.

7.2.1 Abilitazione IP forwarding

Per consentire il forwarding tra interfacce di rete (fondamentale per il traffico tra subnet Docker e attaccante), è stato abilitato il parametro kernel:
sudo sysctl -w net.ipv4.ip$_{f}orward = 1$
Per rendere la modifica persistente è stato creato il file:
/etc/sysctl.d/99-ipforward.conf
contenente:

```
net.ipv4.ip_forward=1
```

La configurazione è stata applicata tramite:
sudo sysctl –system
La verifica è stata effettuata controllando:
cat /proc/sys/net/ipv4/ip$_{f}orward$
che ha restituito valore **1**, confermando l'attivazione del routing.

7.3 Installazione e configurazione di nftables

7.3.1 Installazione

Il firewall nftables non era inizialmente disponibile nel sistema WSL2 ed è stato installato tramite:
sudo apt update sudo apt install nftables -y
Durante l'installazione è stato necessario gestire il file di configurazione `/etc/nftables.conf`, sostituendo la versione di default.

7.3.2 Avvio e verifica

Dopo l'installazione, il ruleset è stato applicato con:
sudo nft -f /etc/nftables.conf sudo nft list ruleset
In ambiente WSL2 non è disponibile systemd, quindi non è stato possibile utilizzare:

- nftables.service
- systemctl enable nftables

Il firewall è stato quindi gestito manualmente tramite reload della configurazione.

7.4 Struttura delle regole firewall

Il file nftables è stato progettato per filtrare traffico L3/L4 e osservare eventi di rete generati dall'ambiente Docker.

7.4.1 Policy di base

La configurazione iniziale ha impostato una policy restrittiva:

```
chain input {
    type filter hook input priority filter;
    policy drop;
}
```

Questa scelta implementa un modello **default deny**, coerente con il paradigma Zero Trust.

7.4.2 Gestione forwarding Docker

Per permettere il funzionamento dei container, sono state introdotte regole specifiche:

```
iifname eth0 oifname br-* accept
iifname eth0 oifname docker0 accept
iifname br-* oifname br-* accept
```

Queste regole consentono traffico tra:

- rete host WSL2 (eth0)
- bridge Docker (br-*)
- rete interna zerotrust_net

7.5 Test di sicurezza con Kali Linux

Per verificare il comportamento del firewall, è stato utilizzato un container Kali Linux con accesso simultaneo alla rete WAN e alla rete interna Docker.

7.5.1 Test di scansione

Sono stati eseguiti diversi scan con nmap:
nmap -Pn -p- 172.20.0.2
e:
nmap -sT -p 5173,8000,8088 172.20.0.2
I risultati hanno mostrato:

- porte effettivamente chiuse o filtrate;
- assenza di risposta SYN in modalità stealth scan;
- traffico visibile solo a livello di connessione TCP completa.

7.5.2 Osservazione del forwarding

Per verificare il passaggio del traffico attraverso il kernel host, è stato monitorato il contatore nftables:
sudo nft list ruleset sudo nft reset counters
I contatori delle regole forward hanno mostrato incremento durante gli attacchi, confermando il passaggio dei pacchetti attraverso il layer di routing.

7.6 Problemi riscontrati

Durante la fase di implementazione sono emerse diverse limitazioni tipiche dell'ambiente WSL2:

- assenza di systemd (nftables non gestibile come servizio);
- necessità di installare manualmente tool come socat e netcat;
- differenze tra rete Windows e rete Linux (bridge Docker non sempre visibile direttamente);
- limitazioni del SYN scan in ambiente virtualizzato.

7.7 Conclusioni

La configurazione del firewall nftables su WSL2 ha permesso di costruire un primo livello di controllo del traffico L3/L4 all'interno dell'architettura Zero Trust.
Nonostante le limitazioni della virtualizzazione, è stato possibile:

- abilitare routing reale tra subnet;
- osservare traffico generato da attacchi simulati;
- validare il comportamento delle policy firewall;
- integrare il sistema con l'infrastruttura Docker esistente.

Questa fase costituisce la base per l'integrazione futura con sistemi IDS/IPS e SIEM (Snort e Splunk), già presenti nell'architettura del progetto.



Test

8.1 Introduzione

All'interno di questo capitolo saranno esposti i test effettuati, per mostrare il corretto funzionamento del sistema.

8.2 Test tramite Postman della logica di accesso

Per verificare il comportamento del sistema sono stati eseguiti diversi test tramite Postman. L'obiettivo principale è stato controllare il funzionamento dei flussi di autenticazione, la verifica del token JWT, il controllo del certificato client mTLS e la logica di autorizzazione basata su attributi.

Le prove sono state svolte in ambiente locale, utilizzando gli endpoint esposti dal backend. In particolare, la porta `3000` è stata utilizzata per testare direttamente le API applicative durante la fase di sviluppo. In un deployment reale, il traffico dovrebbe essere instradato tramite un canale cifrato e controllato, ad esempio attraverso un proxy HTTPS, evitando l'invio di credenziali o token su connessioni non protette.

La Tabella 8 riassume gli endpoint principali utilizzati nei test.

Endpoint	Metodo	Obiettivo del test
`/api/auth/login`	`POST`	Verificare il login dell'utente e il rilascio del token JWT.
`/api/auth/me`	`GET`	Verificare la validità del JWT e il recupero dei dati dell'utente autenticato.
`/api/protected/employee-records`	`GET`	Verificare che l'accesso a una risorsa protetta richieda un certificato client mTLS valido.
`/api/access/evaluate`	`POST`	Verificare la logica di autorizzazione al variare degli attributi della richiesta.

Tabella 8: Endpoint utilizzati nei test tramite Postman.

8.2.1 Test di autenticazione

Il primo test riguarda il login dell'utente. Tramite Postman è stata inviata una richiesta `POST` all'endpoint:

```
http://localhost:3000/api/auth/login
```

Nel caso di credenziali corrette, il backend ha restituito un token JWT e le informazioni principali dell'utente autenticato.

Risposta al login con credenziali corrette

```
{
  message: Login completato,
  token: ...,
  user: {
    id: 6a43c748b7c9f801c143fbf8,
    username: admin,
    role: admin,
    department: IT,
    name: {
      first: Pietro,
      last: Salvatore
    },
    email: admin@company.com,
    status: active,
    mfa_enabled: true
  }
}
```

Il risultato conferma che, in presenza di credenziali valide, il backend genera correttamente un token JWT. Tale token rappresenta l'identità dell'utente nelle richieste successive, ma non costituisce da solo una decisione di accesso alle risorse protette.

È stato poi eseguito un secondo test con credenziali non valide. In questo caso il backend non ha rilasciato alcun token e ha restituito un messaggio di errore.

Risposta al login con credenziali errate

```
{
  message: Credenziali non valide
}
```

Questo comportamento mostra che il token JWT viene generato solo dopo una verifica positiva delle credenziali.

8.2.2 Test di verifica del JWT

Dopo aver ottenuto il token tramite login, è stata effettuata una richiesta all'endpoint:

```
GET /api/auth/me
```

La richiesta è stata inviata includendo il JWT nell'header `Authorization`:

```
Authorization:  Bearer <token>
```

Il backend ha validato il token e ha restituito i dati dell'utente autenticato.

Risposta alla verifica del JWT

```
{
  user: {
    id: 6a43c748b7c9f801c143fbf8,
    username: admin,
    role: admin,
    department: IT,
    name: {
      first: Pietro,
      last: Salvatore
    },
    email: admin@company.com,
    status: active,
```

```
    mfa_enabled: true
  }
}
```

Il risultato conferma che il JWT generato durante il login viene validato correttamente dal backend. Tuttavia, nel modello adottato, il token serve a identificare l'utente, mentre la decisione finale di accesso dipende anche da altri fattori, tra cui dispositivo, certificato client, risorsa richiesta, azione, rete e attributi contestuali.

8.2.3 Test di accesso a una risorsa protetta senza certificato client

Per verificare che il solo possesso del JWT non fosse sufficiente per accedere alle risorse protette, è stata inviata una richiesta alla risorsa `/api/protected/employee-records` senza presentare un certificato client mTLS valido.

Il backend ha restituito il seguente messaggio:

Risposta in assenza di certificato mTLS

```
{
  message: Accesso negato: certificato mTLS mancante o non valido
}
```

Lo stesso comportamento è stato osservato anche includendo un token JWT valido ma senza certificato client. Questo risultato mostra che l'identità utente, da sola, non è sufficiente per accedere alle risorse protette. Il sistema richiede anche la presenza di un'identità dispositivo valida, rappresentata dal certificato client mTLS.

Questo comportamento è coerente con il modello Zero Trust adottato nel progetto: ogni richiesta deve essere verificata considerando più elementi, e non soltanto la presenza di un utente autenticato.

8.2.4 Test della logica di autorizzazione basata su attributi

Successivamente è stato testato l'endpoint applicativo:

```
POST /api/access/evaluate
```

Questo endpoint è stato utilizzato per verificare il comportamento della logica di autorizzazione al variare degli attributi della richiesta. In particolare, sono stati modificati parametri come utente, dispositivo, risorsa, azione, zona di rete e attributi contestuali.

L'attributo `mfa_verified` è stato utilizzato come valore simulato all'interno dei test. Il progetto non implementa un vero flusso di autenticazione multifattore completo, ma usa questo attributo per verificare come la policy reagisce alla presenza o assenza di una verifica multifattore dichiarata nel contesto della richiesta.

Nel primo caso è stata inviata una richiesta valida relativa all'utente `admin`, al dispositivo `Admin Laptop` e alla risorsa `employee_records`. L'azione richiesta è `read`, la zona di rete è `internal` e l'attributo `mfa_verified` è impostato a `true`.

Risposta in caso di accesso consentito

```
{
  message: Valutazione accesso completata,
  username: admin,
  role: admin,
  device_name: Admin Laptop,
  device_status: active,
  device_trusted: true,
  resource_name: employee_records,
  action: read,
  network_zone: internal,
  mfa_verified: true,
  decision: allow,
```

```
  reason: policy_satisfied
}
```


Il risultato mostra che, quando tutti gli attributi richiesti sono coerenti con la policy, il sistema restituisce una decisione `allow`.

Nel secondo caso è stata mantenuta la stessa configurazione, modificando però l'azione richiesta da `read` a `delete`.

Risposta in caso di azione non consentita


```
{
  message: Valutazione accesso completata,
  username: admin,
  role: admin,
  device_name: Admin Laptop,
  device_status: active,
  device_trusted: true,
  resource_name: employee_records,
  action: delete,
  network_zone: internal,
  mfa_verified: true,
  decision: deny,
  reason: action_not_allowed
}
```


Il risultato mostra che il sistema non concede l'accesso soltanto perché l'utente è amministratore o perché il dispositivo è trusted. Anche l'azione richiesta deve essere esplicitamente consentita dalla policy.

Nel terzo caso è stata nuovamente richiesta l'azione `read` sulla risorsa `employee_records`, ma impostando l'attributo `mfa_verified` a `false`. La richiesta è stata negata con motivazione `mfa_required`. Questo comportamento dimostra che la policy considera anche gli attributi contestuali associati alla richiesta e non soltanto ruolo, dispositivo e risorsa.

Nel quarto caso sono stati mantenuti utente, dispositivo, risorsa, azione e attributo MFA validi, modificando però la zona di rete da `internal` a `external`.

Risposta in caso di zona di rete non consentita


```
{
  message: Valutazione accesso completata,
  username: admin,
  role: admin,
  device_name: Admin Laptop,
  device_status: active,
  device_trusted: true,
  resource_name: employee_records,
  action: read,
  network_zone: external,
  mfa_verified: true,
  decision: deny,
  reason: network_zone_not_allowed
}
```


Il risultato conferma che la decisione non dipende soltanto da utente e dispositivo, ma anche dal contesto di rete. Anche in presenza di un utente amministratore e di un dispositivo trusted, una richiesta proveniente da una zona non autorizzata viene negata.

8.2.5 Test di validazione dell'input

Sono stati inoltre eseguiti test relativi alla validazione dell'input. In un primo caso è stata inviata una richiesta incompleta all'endpoint `/api/access/evaluate`. Il backend ha rifiutato la richiesta indicando esplicitamente i campi obbligatori mancanti.

Risposta in caso di campi obbligatori mancanti

```
{
  message: Campi obbligatori mancanti,
  required_fields: [
    username,
    device_name,
    resource_name,
    action
  ]
}
```

Questo comportamento mostra che l'endpoint effettua una validazione preliminare dei dati ricevuti prima di procedere con la valutazione dell'accesso.

In un secondo caso è stata inviata una richiesta utilizzando un dispositivo non associato all'utente indicato. Il backend ha rifiutato la richiesta restituendo un messaggio specifico relativo al dispositivo non trovato. Questo risultato mostra che la valutazione applicativa non si basa soltanto sul nome utente dichiarato, ma verifica anche che il dispositivo sia effettivamente associato all'utente.

Infine, sono state inviate richieste verso risorse per le quali non era presente una policy applicativa corrispondente nell'endpoint di valutazione. In questi casi il backend ha restituito una decisione negativa con motivazione `policy_not_found`. Questo comportamento è coerente con il principio *deny by default*: in assenza di una policy esplicita, l'accesso non viene consentito.

8.2.6 Sintesi dei risultati

La Tabella 9 riassume i principali test effettuati tramite Postman.

Scenario	Attributi principali	Risultato osservato
Login corretto	Credenziali valide per l'utente `admin`	Generazione del JWT e restituzione dei dati utente.
Login errato	Credenziali non valide	Nessun token rilasciato e messaggio `Credenziali non valide`.
Verifica JWT	Token valido nell'header `Authorization`	Restituzione dei dati dell'utente autenticato.
Risorsa protetta senza certificato	JWT assente o presente, ma certificato client mTLS mancante	Accesso negato per certificato mTLS mancante o non valido.
Accesso consentito	`admin`, `Admin Laptop`, `employee_records`, azione `read`, `network_zone=internal`, `mfa_verified=true`	`allow`, `policy_satisfied`.
Azione non consentita	Stessi attributi del caso consentito, ma azione `delete`	`deny`, `action_not_allowed`.
Attributo MFA simulato assente	Azione `read`, ma `mfa_verified=false`	`deny`, `mfa_required`.
Zona di rete non consentita	Stessi attributi del caso consentito, ma `network_zone=external`	`deny`, `network_zone_not_allowed`.
Dispositivo non associato	Utente `demo_customer` con dispositivo `Customer Tablet`	Errore: dispositivo non trovato per l'utente specificato.
Campi obbligatori mancanti	Richiesta incompleta	Errore con elenco dei campi richiesti.
Policy applicativa assente	Richiesta verso risorsa senza policy configurata nell'endpoint di valutazione applicativa	`deny`, `policy_not_found`.

Tabella 9: Sintesi dei test effettuati tramite Postman sulla logica di accesso.

I risultati ottenuti confermano che il processo decisionale non dipende da un singolo elemento, ma dalla valutazione congiunta di identità utente, dispositivo, risorsa, azione e attributi contestuali. Anche semplici variazioni, come un'azione non consentita, l'assenza dell'attributo `mfa_verified` o una zona di rete differente, modificano l'esito della decisione. Questo comportamento dimostra l'applicazione del paradigma Zero Trust e del principio del minimo privilegio: ogni richiesta viene valutata nel proprio contesto e l'accesso viene consentito solo quando tutte le condizioni richieste risultano soddisfatte.

8.3 Test enrollment di un dispositivo

Si mostrano, ora, i test effettuati, per verificare il funzionamento dell'enrollment di un nuovo dispositivo all'interno del sistema. Si esporranno, dunque, rispettivamente: Secure Enclave, TPM e software fallback.

8.3.1 Secure Enclave

Il test inizia dalla schermata di login. Qui, l'utente alice, appartenente al dipartimento delle risorse umane, inserisce le sue credenziali:

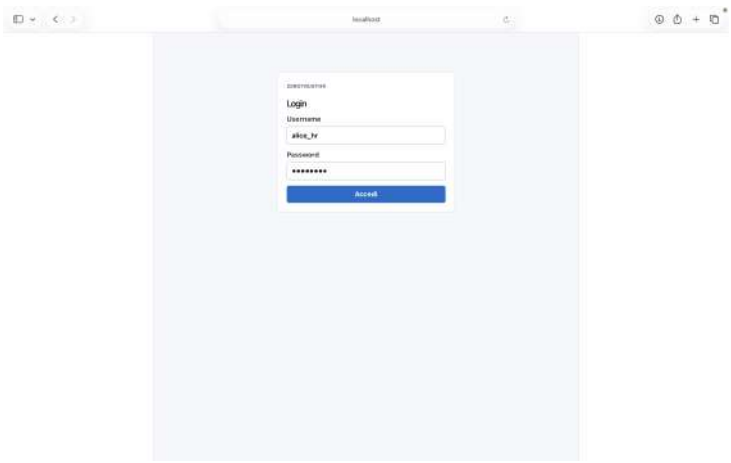


Figura 18: Schermata di login

Una volta effettuato il login, l'utente riceve un messaggio, in cui si afferma che non possiede ancora un dispositivo associato, e, quindi, non può accedere alle risorse protette:

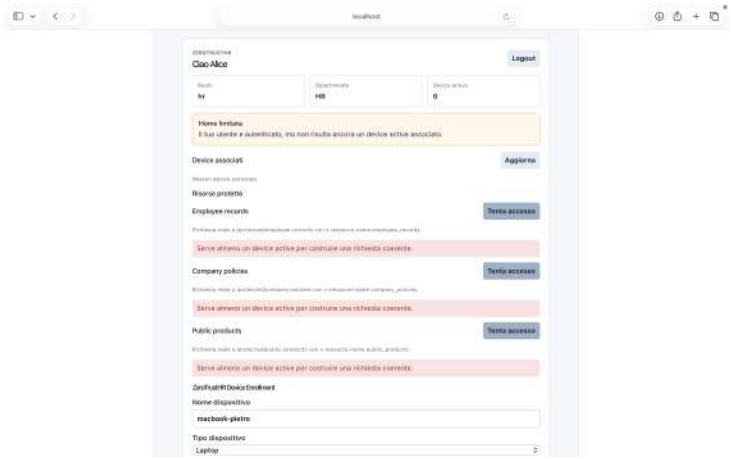


Figura 19: Messaggio di mancanza di dispositivo associato all'utente

Inizia, ora, la fase di enrollment effettiva. Qui, dopo aver scelto il nome per il dispositivo, il tipo di dispositivo, il sistema operativo e l'hardware key, l'utente preme preme il bottone "crea challenge". Sono, dunque, generati: il device ID, la SAN URI e la challenge da firmare. Il dispositivo assume lo stato di pending:



Figura 20: Creazione della challenge

Lo step successivo prevede l'utilizzo del device agent su XCode. La scelta di utilizzare un applicazione macOS su XCode deriva dal fatto che la chiave deve essere:

- generata localmente sul dispositivo;
- non esportabile;
- riutilizzabile dopo l'enrollment;
- associata a un'identità applicativa stabile.

Dunque, per avere una chiave permanente nel Keychain, dobbiamo utilizzare una vera applicazione macOS. Per la configurazione dell'applicazione macOS si rimanda al file README.md, presente nella cartella "device-agent".

Una volta avviata l'applicazione del device agent, l'utente copia qui i campi: device ID, SAN URI e challenge da firmare, ottenuti nello step precedente. Dopo aver premuto "Genera dati enrollment", l'applicazione crea un JSON con all'interno: la firma della challenge, csrPEM e public key PEM.

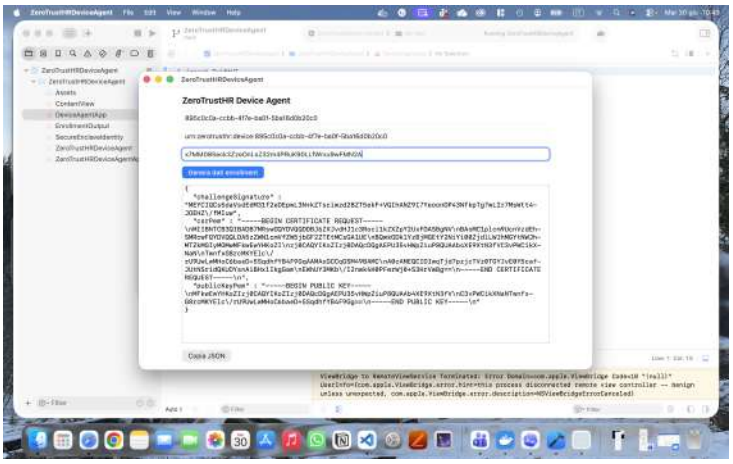


Figura 21: Firma della challenge

Adesso, l'utente inserisce i campi ricevuti dal device agent all'interno del frontend, e preme "Verifica enrollment":

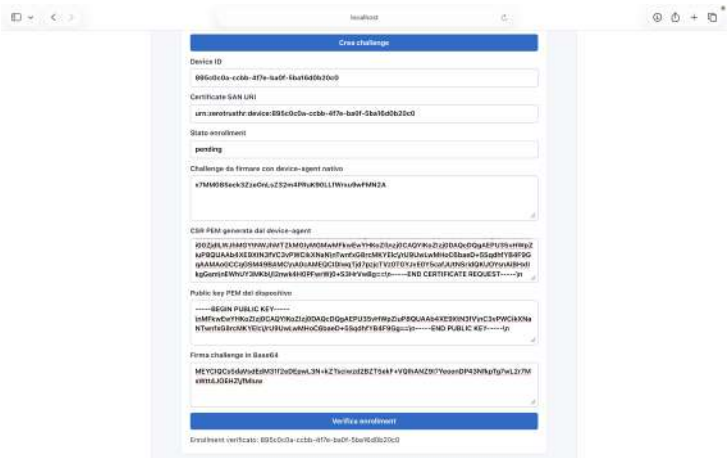


Figura 22: Verifica dell'enrollment

Si evince dal fondo dello screenshot precedente che l'enrollment è andato a buon fine dal messaggio: "Enrollment verificato".

Adesso, il dispositivo deve essere approvato da un utente admin del sistema:

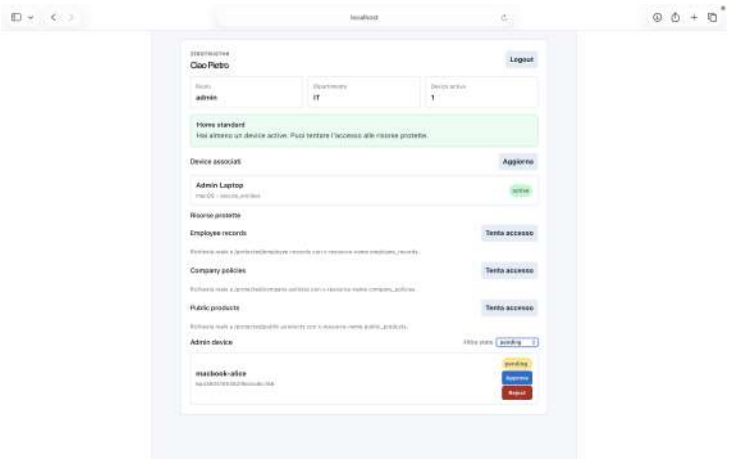


Figura 23: Dispositivo in pending nell'account di un admin

Nello screenshot successivo si mostra la fine della fase di enrollment, a seguito dell'approvazione del dispositivo da parte dell'utente admin:

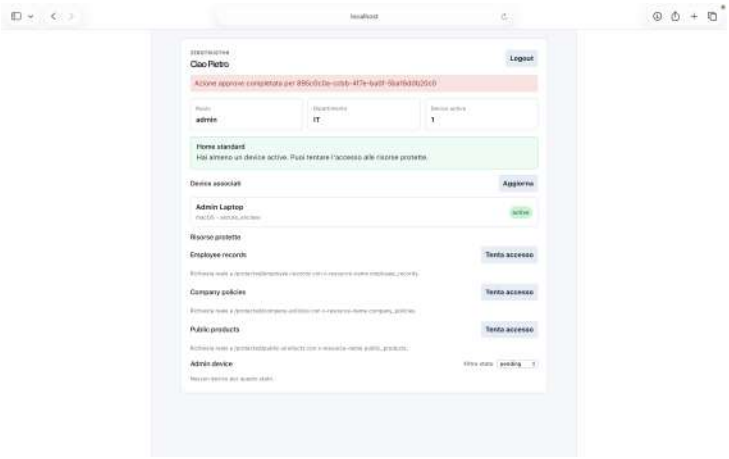


Figura 24: Messaggio di approvazione del dispositivo

Come ultima fase del test, si mostra l'utente che esegue il login e il suo dispositivo risulta approvato. Alice, adesso, può richiedere l'accesso alle risorse protette:

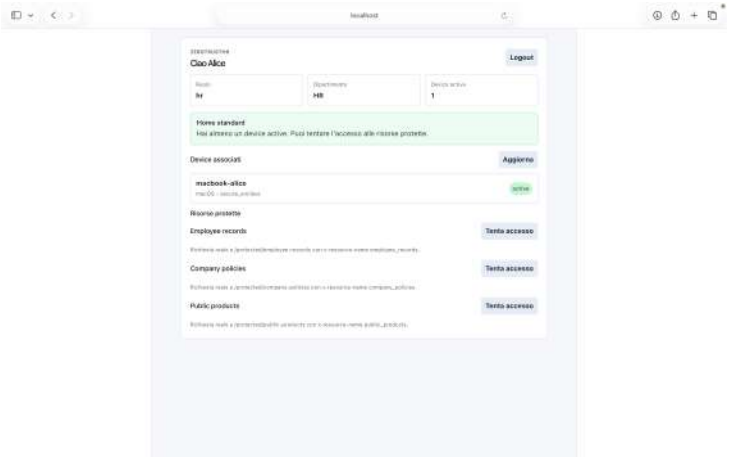


Figura 25: Dispositivo utente approvato

8.3.2 TPM

La logica di esecuzione per il test del TPM è molto simile a quella presentata per il Secure Enclave. Si mostrano, dunque, le sottili differenze nel flusso esecutivo.

Analogamente a quanto mostrato precedentemente, l'utente inserisce il nome del suo dispositivo, seleziona come sistema operativo Windows e come hardware key TPM. Dopo aver generato la challenge, per ricevere la firma, è necessario l'utilizzo del device-agent nativo di Windows. Dunque, l'utente deve eseguire questo comando da terminale:

```
cd device-agent\windows
dotnet build -c Release
```

Di seguito si mostra il risultato del comando:



Figura 26: Esecuzione della build

A seguito dell'esecuzione del codice, sono create due cartelle bin/, che contiene l'eseguibile, e obj/, che contiene file intermedi della build .NET.

Successivamente, l'utente lancia il seguente codice:

```
  .\bin\Release\net8.0-windows\ZeroTrustHR.TpmAgent.exe `
"DEVICE_ID_DAL_FRONTEND" `
"CERTIFICATE_SAN_URI_DAL_FRONTEND" `
"CHALLENGE_DAL_FRONTEND"
```

Sostituendo il comando con i valori ricevuti dal frontend per: device ID, SAN URI e challenge:

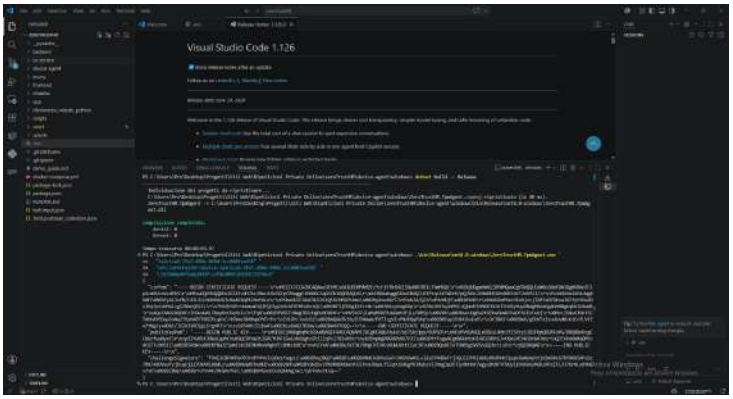


Figura 27: Firma della challenge

A questo punto, il device agent stampa il JSON con: la firma della challenge, la csr PEM e la public key PEM, che l'utente inserirà nel frontend.

Dopo aver premuto "Verifica enrollment", l'utente riceverà il messaggio di "Enrollment verificato", esattamente come mostrato per il Secure Enclave:

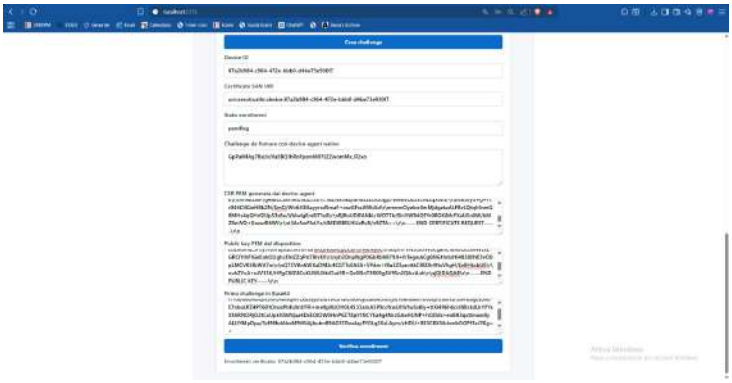


Figura 28: Enrollment completato

Gli step successivi di approvazione del dispositivo da parte dell'utente admin sono analoghi.

8.3.3 Software Fallback

Si mostra, infine, il fallback nel caso in cui l'utente non disponga di un device con hardware key TPM o Secure Enclave. Per questo test è stato utilizzato l'utente Marco, che rientra nel dipartimento "employee".

Quando l'utente esegue il login, di default è presente la voce "Software Fallback" per l'enrollment del dispositivo. Quindi, l'utente seleziona "Crea challenge e verifica enrollment":

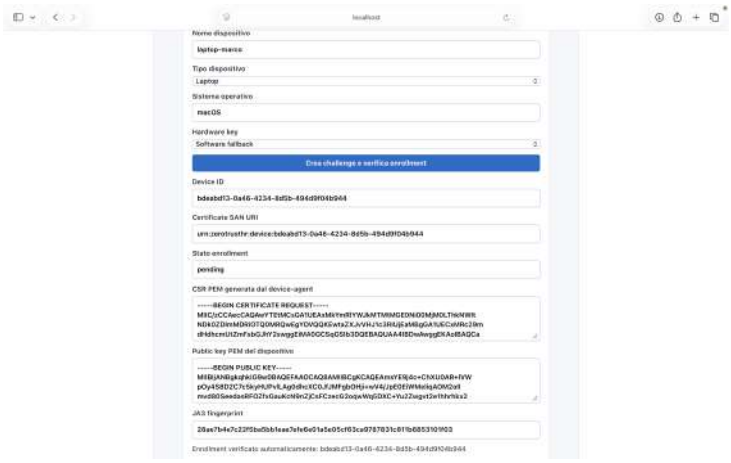


Figura 29: Creazione della challenge ed enrollment con Software Fallback

In questo caso, non vi è la necessità di alcun device agent, quindi la challenge è firmata automaticamente, e, quindi, l'utente riceve il messaggio: "Enrollment verificato automaticamente".

Anche in questo caso, il dispositivo deve essere approvato dall'utente admin:

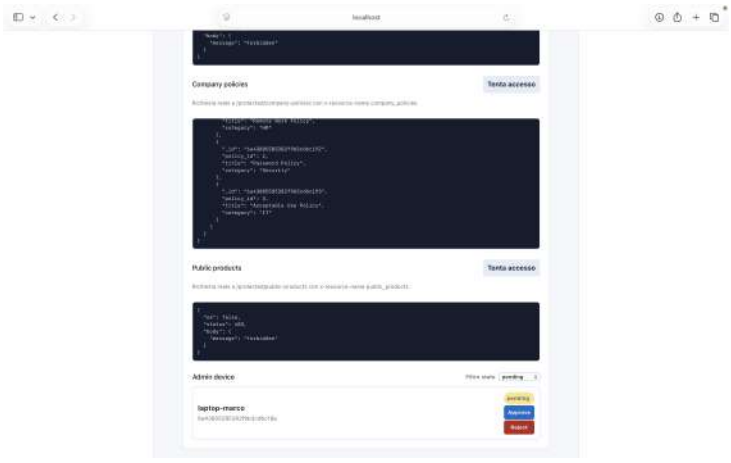


Figura 30: Approvazione del dispositivo da parte dell'admin

8.4 Test di rilevamento degli attacchi di rete (IDS e mTLS)

In questa sezione saranno esposti i test effettuati per validare la componente infrastrutturale dell'architettura Zero Trust. L'obiettivo principale è stato verificare il calcolo dinamico del rischio di rete (P_{net}) e la mitigazione tempestiva delle minacce tramite la crittografia perimetrale.

Per condurre questa validazione, l'infrastruttura è stata sottoposta a due diverse tipologie di attacco. Tali attacchi sono stati generati simulando un nodo compromesso all'interno della medesima rete virtuale locale (`zerotrusthr_zerotrust_net`).

8.4.1 Test di Port Scanning e rilevamento tramite IDS

Il primo test riguarda l'esecuzione di una scansione aggressiva delle porte TCP (TCP SYN Scan), finalizzata a mappare i servizi esposti dal proxy Envoy. Per l'esecuzione di tale attività è stato impiegato lo strumento **Nmap** tramite un container Docker isolato. Nello specifico, è stato eseguito il seguente comando:

```
docker run --rm --network zerotrusthr_zerotrust_net instrumentisto/nmap
  -sS -p- -T4 zerotrust-envoy
```

Il comando avvia un container temporaneo connesso alla rete interna del progetto, lanciando una scansione invisibile (`-sS`) su tutte le porte (`-p-`) del target `zerotrust-envoy`.

Come evidenziato in Figura 31, Nmap ha identificato le porte aperte sull'infrastruttura, inclusa la porta `10000` dedicata al servizio di autenticazione.

```
PS C:\Users\seba9\Documents\GitHub\zeroTrustHR> docker run --rm --network zerotrusthr_zerotrust_net instrumentisto/nmap -sS -p- -T4 zerotrust-envoy
Starting Nmap 7.98 ( https://nmap.org ) at 2026-06-30 17:11 +0000
Nmap scan report for zerotrust-envoy (172.20.0.8)
Host is up (0.00015s latency).
rDNS record for 172.20.0.8: zerotrust-envoy.zerotrusthr_zerotrust_net
Not shown: 65532 closed tcp ports (reset)
PORT      STATE SERVICE
9901/tcp  open  unknown
10000/tcp open  snet-sensor-mgmt
10001/tcp open  scp-config
MAC Address: 62:A4:BE:A9:FB:15 (Unknown)

Nmap done: 1 IP address (1 host up) scanned in 10.88 seconds
```

Figura 31: Esecuzione dell'attacco di Port Scanning tramite Nmap verso il proxy Envoy

Durante l'esecuzione, il componente Intrusion Detection System (**Snort**), configurato per ispezionare il traffico di rete, ha intercettato il pattern anomalo generato dai pacchetti SYN, emettendo il corrispondente allarme di sicurezza. Successivamente, il SIEM (**Splunk**) ha recepito l'evento, registrandolo come `Nmap Scan Detected (Port 10000)` e associandolo all'indirizzo IP della sorgente attaccante, come mostrato in Figura 32.



Figura 32: Rilevamento dell'attacco Nmap da parte di Snort e visualizzazione dell'alert su Splunk

Tale intercettazione innesca automaticamente la risposta del sistema Zero Trust: all'indirizzo IP malevolo viene associato un valore di P_{net} critico. Di conseguenza, il motore OPA riceve istruzione di negare incondizionatamente qualsiasi futura richiesta legittima proveniente da tale nodo, confermando l'importanza del contesto di rete nella valutazione del rischio globale.

8.4.2 Test di attacco Brute Force su endpoint mTLS

Nel secondo test è stato simulato un attacco contro l'endpoint di login HTTP, impiegando lo strumento **Hydra**. L'obiettivo del test era valutare la resilienza del livello di sicurezza perimetrale gestito da Envoy, il quale è configurato per imporre l'autenticazione tramite *Mutual TLS* (mTLS). Per sferrare l'attacco è stato eseguito il seguente comando, iniettando una wordlist di password:

```
docker run --rm -v "${PWD}/passwords.txt:/passwords.txt"
  --network zerotrusthr_zerotrust_net secsi/hydra -l admin -P /passwords.txt
  -s 10000 zerotrust-envoy
  http-post-form "/api/auth/login:username=^USER^&password=^PASS^:F=Unauthorized"
```

Il comando tenta un attacco di tipo forza bruta (*Brute Force*) sul modulo di login `http-post-form`, cercando di autenticarsi come utente `admin` provando sistematicamente le password contenute nel file locale.

I risultati del test sono visibili in Figura 33, i quali evidenziano la completa neutralizzazione della minaccia.

```
PS C:\Users\seba9\Documents\GitHub\zeroTrustHR> docker run --rm -v "${PWD}/passwords.txt:/passwords.txt" --network zerotrusthr_zerotrust_net secsi/hydra -l admin -P /pa
sswords.txt -s 10000 zerotrust-envoy http-post-form "/api/auth/login:username=^USER^&password=^PASS^:F=Unauthorized"
Hydra v9.7 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret service organizations, or for illegal purposes (this is non-binding, th
ese *** ignore laws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-06-30 17:14:23
[DATA] max 16 tasks per 1 server, overall 16 tasks, 17 login tries (l:1/p:17), ~2 tries per task
[DATA] attacking http-post-form://zerotrust-envoy:10000/api/auth/login:username=^USER^&password=^PASS^:F=Unauthorized
1 of 1 target completed, 0 valid password found
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2026-06-30 17:14:23
```

Figura 33: Fallimento istantaneo dell'attacco Brute Force tramite Hydra a causa del blocco mTLS operato da Envoy

Hydra ha completato i tentativi in una frazione di secondo, riportando esito negativo per ogni iterazione (`0 valid password found`). Il motivo di tale fallimento risiede nella dinamica del protocollo di trasporto. L'attaccante ha tentato di inviare richieste HTTP in chiaro; tuttavia, il proxy Envoy ha terminato la connessione a livello TCP/TLS, rifiutandola a monte a causa dell'assenza di un certificato client valido.

Questo comportamento dimostra l'efficacia dell'approccio **Defense in Depth**. La protezione perimetrale mTLS neutralizza le richieste illegittime prima che queste possano raggiungere i servizi di backend o richiedere l'intervento dell'Intrusion Detection System, garantendo così una prima linea di difesa estremamente robusta.

8.5 Test e configurazione di Splunk

In questa fase dei test è stata validata la componente analitica dell'architettura, governata dal SIEM (Splunk) e dall'algoritmo di Machine Learning (Random Forest). L'obiettivo è dimostrare l'effettivo funzionamento del calcolo dinamico del rischio applicativo (P_{app}) e il conseguente blocco adattivo delle minacce, in stretta sinergia con il motore OPA.

8.5.1 Fase 1: Generazione dei dati e Addestramento del Modello

Prima di poter valutare il rischio in tempo reale, il sistema necessita di un campione statistico iniziale per differenziare la normalità dall'anomalia. A tale scopo, è stato impiegato uno script automatizzato incaricato di generare dati storici, producendo un traffico misto composto all'80% da richieste legittime e al 20% da attacchi mirati.

La simulazione è stata avviata eseguendo il seguente comando da terminale:

```
py scripts/simulate_auth_attacks.py
```

Tutte le decisioni prese da OPA durante questa fase esplorativa sono state trasmesse via HEC (HTTP Event Collector) a Splunk. Successivamente, per estrarre le *feature* comportamentali e addestrare il modello predittivo, è stata eseguita la seguente query SPL (Splunk Processing Language) all'interno dell'interfaccia di ricerca del SIEM:

Query SPL per l'estrazione delle feature e l'addestramento del modello Random Forest

```
search (index=zerotrust sourcetype=opa_decision)
| spath input=_raw path=line.result.allowed output=allowed
| spath input=_raw path=line.result.user output=user_id
| eval is_opa=1
| eval is_deny=if(allowed=false, 1, 0)
| eval reason_time = if(is_deny=1 AND match('line.result.deny_reasons{}',
    outside_working_hours), 1, 0)
| eval reason_priv = if(is_deny=1 AND match('line.result.deny_reasons{}',
    role_not_allowed|action_not_allowed|department_not_allowed), 1, 0)
| eval reason_auth = if(is_deny=1 AND match('line.result.deny_reasons{}',
    untrusted_device|device_not_active|mfa_required|ja3_fingerprint_blocked|
    ip_not_in_allowed_zone|unsupported_os), 1, 0)
| eval device_ip_header = 'line.input.attributes.request.http.headers.x-device-ip'
| eval src_ip = coalesce(device_ip_header, src_ip, src)
| stats
```

```
    sum(is_deny) AS total_denies
    sum(reason_time) AS time_denies
    sum(reason_priv) AS priv_denies
    sum(reason_auth) AS auth_denies
    count(eval(is_opa=1)) AS total_opa_requests
    dc(eval(if(is_opa=1, src_ip, null()))) AS distinct_ips
    BY user_id
| eval deny_ratio = if(total_opa_requests > 0, total_denies / total_opa_requests, 0)
| eval is_attacker = if(user_id=eve_attacker OR user_id=oscar_attacker, 1, 0)
| fit RandomForestClassifier is_attacker from total_denies time_denies priv_denies
    auth_denies distinct_ips deny_ratio into app_risk_model
```

I risultati dell'addestramento (visibili in Figura 34) mostrano come il modello Random Forest (`app_risk_model`) riconosca i profili ostili non solo in base al volume dei rifiuti, ma alla natura contestuale delle violazioni (es. forzatura dei privilegi).

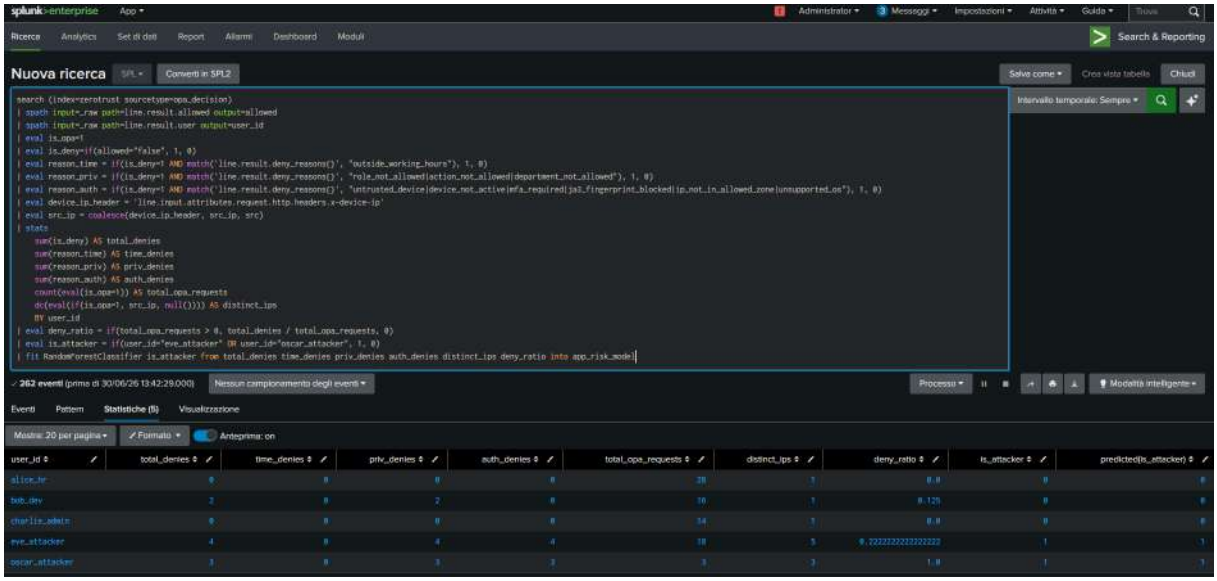


Figura 34: Interfaccia Splunk: esecuzione della query SPL per l'estrazione delle *feature* comportamentali e classificazione degli attaccanti.

8.5.2 Fase 2: Blocco Dinamico Predittivo in Tempo Reale

A modello addestrato, è stata avviata una seconda simulazione per osservare l'intervento dell'Intelligenza Artificiale a regime. Per testare l'innalzamento progressivo del rischio applicativo (P_{app}) è stato eseguito il seguente comando:

```
py scripts/demo_live_attacks.py
```

I risultati hanno confermato il corretto funzionamento del principio di *Continuous Verification*. Gli utenti legittimi (es. `alice_hr`, `bob_dev`) hanno mantenuto un punteggio di rischio estremamente basso o nullo, ottenendo l'accesso continuo (`Allowed: True`).

Al contrario, lo script ha generato per gli utenti `eve_attacker` e `oscar_attacker` una serie di richieste anomale. Il sistema, interrogando in tempo reale il modello `app_risk_model`, ha rilevato che le violazioni reiterate comportavano un netto incremento della probabilità applicativa.

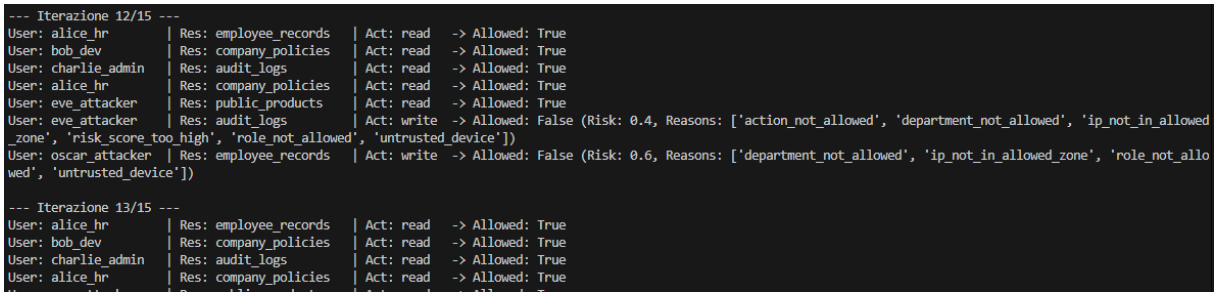
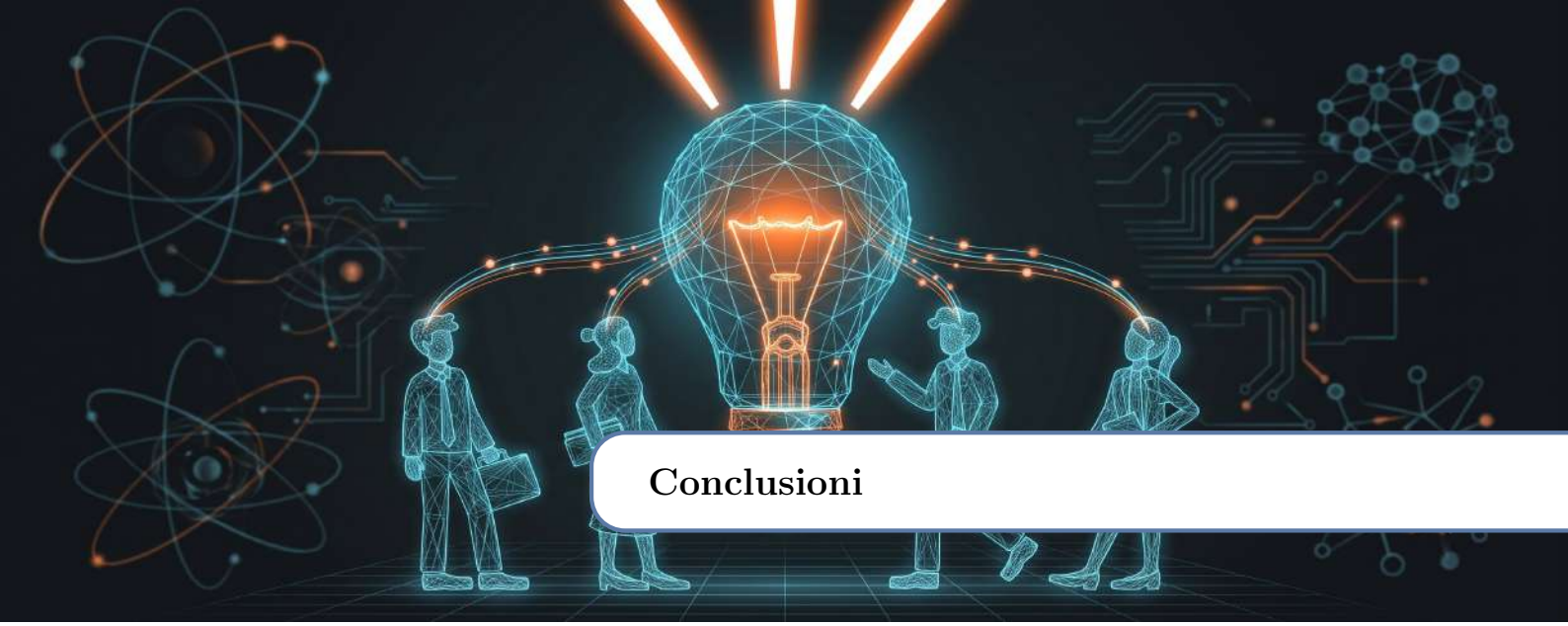


Figura 35: Output della simulazione: le violazioni ripetute innalzano esponenzialmente il Risk Score degli attaccanti, scatenando il blocco incondizionato da parte di OPA.

Come evidenziato in Figura 35, la combinazione matematica tra questa elevata probabilità e la severità delle risorse richieste ha infranto rapidamente le soglie di tolleranza. Di conseguenza, il motore OPA ha negato in modo adattivo gli accessi (`Allowed: False`), indicando espressamente `risk_score_too_high` come motivazione di repulsione, isolando con successo le identità compromesse ancor prima che potessero finalizzare l'esfiltrazione dei dati.



Conclusioni

9.1 Sintesi del progetto realizzato

Il progetto **ZeroTrustHR** ha avuto l'obiettivo di realizzare un prototipo di architettura Zero Trust applicato a un contesto aziendale, con particolare attenzione alla protezione di risorse sensibili in ambito HR. L'idea alla base del progetto è stata quella di superare un modello di sicurezza fondato sulla fiducia implicita, nel quale un utente autenticato o proveniente da una rete interna viene considerato automaticamente affidabile.

Nel sistema realizzato, ogni richiesta di accesso viene invece valutata in modo contestuale. La decisione non dipende da un singolo parametro, ma dalla combinazione di più fattori: identità dell'utente, ruolo, dipartimento, risorsa richiesta, azione, stato del dispositivo, certificato client, zona di rete, fingerprint, rischio e comportamento osservato nel tempo.

Dal punto di vista applicativo, il progetto integra un frontend sviluppato con React, Vite e TypeScript, un backend Node.js/TypeScript, un database MongoDB e diversi servizi di sicurezza orchestrati tramite Docker Compose. Il backend gestisce utenti, dispositivi, risorse, enrollment, certificati, richieste di accesso e integrazione con i componenti esterni.

Uno degli aspetti centrali del progetto è la distinzione tra autenticazione e autorizzazione. Il login utente, gestito tramite JWT, consente di identificare l'utente, ma non garantisce automaticamente l'accesso alle risorse. La decisione finale viene demandata a un sistema di policy basato su **OPA** e **Rego**, mentre **Envoy** agisce come punto di enforcement, intercettando le richieste e applicando la decisione ricevuta.

Il progetto introduce inoltre il concetto di identità del dispositivo. Attraverso il processo di device enrollment, un dispositivo può essere registrato, approvato e associato a un certificato client. L'utilizzo di **mTLS** permette quindi di rafforzare il controllo sull'origine della richiesta, poiché l'accesso alle risorse protette richiede non solo un utente autenticato, ma anche un dispositivo dotato di certificato valido.

A questo si aggiungono componenti locali per macOS e Windows, collegati rispettivamente a **Secure Enclave** e **TPM**. Queste tecnologie permettono di rafforzare il legame tra identità crittografica e dispositivo fisico, riducendo il rischio che una chiave privata possa essere copiata o riutilizzata su un altro dispositivo.

Un altro elemento importante è l'integrazione con **Splunk**, utilizzato come SIEM per la raccolta, l'indicizzazione e l'analisi degli eventi. Splunk contribuisce alla valutazione dinamica del rischio attraverso saved searches, lookup, alert, webhook e componenti di Machine Learning. In particolare, il modello Random Forest integrato tramite Splunk AI Toolkit permette di stimare una componente applicativa del rischio a partire dal comportamento osservato.

Infine, il sistema include anche componenti come **Snort** e **nftables**, utili per osservare il traffico di rete e introdurre un ulteriore livello di controllo. Gli alert di rete possono contribuire alla componente di rischio, rendendo la valutazione più completa rispetto a una semplice analisi applicativa.

Nel complesso, ZeroTrustHR mostra come un'architettura Zero Trust non sia il risultato di un singolo strumento, ma della cooperazione tra più livelli: autenticazione, autorizzazione, identità dispositivo, policy, rete, rischio, logging e risposta adattiva.

9.2 Livelli reali di privacy e sicurezza raggiungibili

Uno degli aspetti più significativi del progetto riguarda il rapporto tra privacy, sicurezza e controllo degli accessi. In un sistema informatico reale non è possibile garantire una privacy assoluta, soprattutto quando utenti e dispositivi devono accedere a risorse aziendali sensibili. Ogni richiesta richiede infatti un certo livello di identificazione, verifica e tracciamento.

Il punto centrale non è quindi eliminare ogni forma di raccolta dati, ma limitarla agli attributi necessari per prendere decisioni di sicurezza motivate e verificabili. Nel progetto, informazioni come ruolo, dipartimento, dispositivo, indirizzo IP, zona di rete, risorsa richiesta e decisione di accesso vengono utilizzate per valutare il contesto della richiesta. Questi dati non sono raccolti come fine a sé stessi, ma come elementi necessari per ridurre accessi impropri e comportamenti anomali.

La logica Zero Trust permette di migliorare il livello di sicurezza perché evita che l'accesso venga concesso sulla base di una fiducia iniziale e permanente. Un utente autenticato può comunque ricevere un rifiuto se il suo ruolo non è compatibile con la risorsa, se il dispositivo non è fidato, se il rischio è troppo alto o se compare nella blocklist.

Allo stesso tempo, il sistema rende le decisioni più trasparenti. Le policy Rego non producono soltanto una decisione di tipo `allow` o `deny`, ma possono restituire anche motivazioni specifiche, come `role_not_allowed`, `untrusted_device`, `risk_score_too_high` o `user_blocked_by_barac`. Questo consente di ricostruire le cause di una decisione e di supportare attività di audit.

L'utilizzo di mTLS e certificati client contribuisce a un livello maggiore di protezione rispetto al solo login. Un attaccante in possesso delle credenziali di un utente non avrebbe automaticamente accesso alle risorse protette, perché dovrebbe disporre anche di un certificato valido associato a un dispositivo riconosciuto. In questo senso, il progetto rafforza il legame tra identità utente e identità dispositivo.

Le tecnologie hardware-bound, come TPM e Secure Enclave, migliorano ulteriormente questo modello, poiché rendono più difficile l'esportazione o il riutilizzo della chiave privata. La modalità software di fallback rimane utile per compatibilità e test, ma offre un livello di garanzia inferiore rispetto a una chiave protetta tramite hardware.

La componente di osservabilità introduce un ulteriore equilibrio. Da un lato, raccogliere log è essenziale per individuare comportamenti sospetti, ricostruire eventi e calcolare il rischio. Dall'altro, i log possono contenere informazioni sensibili, come utenti, indirizzi IP, orari, risorse richieste e motivazioni di accesso negato. Per questo motivo, in un sistema reale sarebbe necessario definire policy chiare di retention, controllo degli accessi ai log e minimizzazione dei dati conservati.

Il progetto mostra quindi che privacy e sicurezza non sono concetti opposti, ma devono essere bilanciati. Una maggiore sicurezza richiede osservabilità e controllo, ma questi devono essere progettati in modo proporzionato e giustificato. L'obiettivo non è raccogliere informazioni in modo indiscriminato, ma utilizzare gli attributi necessari per proteggere risorse sensibili e rendere le decisioni verificabili.

9.3 Limiti del prototipo

Pur rappresentando un prototipo articolato, ZeroTrustHR presenta alcuni limiti che è importante evidenziare. Il primo riguarda la natura dell'ambiente di esecuzione. L'intero sistema è stato realizzato e orchestrato tramite Docker Compose, con l'obiettivo di dimostrare i concetti principali della Zero Trust Architecture in un ambiente riproducibile e controllato.

Questa scelta è adatta a un progetto sperimentale, perché permette di avviare e collegare rapidamente backend, frontend, MongoDB, Envoy, OPA, Splunk, Snort, nftables e CA Service. Tuttavia, in un contesto aziendale reale, l'architettura dovrebbe essere distribuita su più macchine o nodi, separando fisicamente e logicamente i componenti più critici. Ad esempio, database, SIEM, proxy, motore di policy e CA Service non dovrebbero necessariamente risiedere nello stesso host.

L'utilizzo di più macchine consentirebbe di avvicinare maggiormente il sistema a uno scenario reale, migliorando isolamento, scalabilità, resilienza e gestione dei guasti. In particolare, sarebbe opportuno separare i componenti con funzioni diverse: servizi applicativi, servizi di sicurezza, raccolta log, gestione dei certificati e sistemi di monitoraggio.

Un altro limite riguarda la gestione della MFA. Nel progetto, la MFA non è stata implementata come vero meccanismo di autenticazione multifattore. Essa è rappresentata tramite un attributo contestuale, ad esempio `mfa_verified`, utilizzato nelle policy e nei test per simulare la presenza o l'assenza di una verifica multifattore. Questo permette di testare il comportamento delle policy in scenari differenti, ma non equivale all'integrazione reale di un sistema MFA.

La parte relativa a TPM e Secure Enclave è invece funzionante e permette di rafforzare l'identità hardware-bound del dispositivo. Rimane comunque un limite legato al passaggio da prototipo ad ambiente enterprise: in un contesto reale sarebbe necessario integrare procedure più complete di attestazione, gestione del ciclo di vita delle chiavi, rotazione, revoca e verifica periodica dello stato del dispositivo.

Anche la componente di Machine Learning deve essere interpretata nel contesto del prototipo. Il modello Random Forest utilizzato tramite Splunk AI Toolkit consente di stimare una probabilità di comportamento anomalo a partire da feature derivate dagli eventi raccolti. Tuttavia, l'efficacia di un modello di questo tipo dipende fortemente dalla qualità, quantità e rappresentatività dei dati di addestramento. In un ambiente reale sarebbe necessario validare il modello su dataset più ampi e monitorare nel tempo falsi positivi e falsi negativi.

Infine, la gestione dei log richiederebbe ulteriori approfondimenti. Splunk consente di centralizzare e analizzare eventi, ma in un sistema reale bisognerebbe definire con precisione chi può accedere ai log, per quanto tempo conservarli, quali dati anonimizzare e come evitare che l'osservabilità diventi una raccolta eccessiva di informazioni.

Questi limiti non riducono il valore del progetto, ma ne chiariscono il perimetro. ZeroTrustHR non vuole essere una soluzione definitiva, bensì un prototipo realistico per dimostrare come i principi Zero Trust possano essere implementati combinando componenti applicativi, infrastrutturali e analitici.

9.4 Tecnologie emergenti e sviluppi futuri

Gli sviluppi futuri del progetto non richiederebbero necessariamente l'introduzione di molte nuove tecnologie, ma soprattutto un'evoluzione dell'architettura verso uno scenario più distribuito e vicino a un ambiente reale.

Il primo miglioramento riguarderebbe quindi la distribuzione dei componenti su più macchine. Invece di eseguire l'intero sistema su un singolo host tramite Docker Compose, si potrebbe separare l'infrastruttura in più nodi: uno per i servizi applicativi, uno per il database, uno per Splunk, uno per Envoy e OPA, e uno dedicato alla gestione dei certificati. Questa separazione permetterebbe di testare meglio aspetti come latenza, sicurezza delle comunicazioni interne, isolamento dei servizi e resilienza ai guasti.

Un secondo sviluppo concreto riguarda l'introduzione di un vero flusso MFA. Attualmente, come evidenziato, la MFA è rappresentata solo come attributo di policy. Una versione successiva potrebbe integrare un meccanismo reale di autenticazione multifattore, ad esempio tramite OTP, push notification, WebAuthn o passkey. Questo renderebbe più completo il processo di autenticazione e permetterebbe di applicare controlli aggiuntivi in base al rischio.

Un'altra possibile evoluzione riguarda l'integrazione con sistemi reali di Identity and Access Management. Nel prototipo, utenti, ruoli e dispositivi sono gestiti internamente. In un ambiente aziendale, invece, sarebbe utile integrare il sistema con directory centralizzate, provider di identità, Single Sign-On e sistemi di gestione dei dispositivi. Questo renderebbe l'architettura più vicina a scenari reali.

La componente hardware-bound potrebbe essere ulteriormente rafforzata tramite meccanismi di attestazione remota. Pur essendo già presente il supporto funzionante a TPM e Secure Enclave, una possibile evoluzione consisterebbe nel verificare non solo il possesso della chiave privata, ma anche lo stato di integrità e conformità del dispositivo.

Anche la componente di rischio potrebbe essere migliorata, ma senza cambiare l'impostazione generale. Il modello Random Forest rappresenta una base utile per stimare comportamenti anomali; una futura evoluzione potrebbe consistere nel validarlo su dati più ampi, confrontarlo con altri modelli e misurare in modo più preciso le prestazioni tramite metriche come precision, recall e F1-score.

Infine, la risposta adattiva potrebbe diventare più graduata. Nel progetto, Splunk può attivare webhook verso il backend per aggiornare la blocklist degli utenti. In futuro, invece del solo blocco, il sistema potrebbe applicare azioni intermedie, come richiesta di un vero controllo MFA, riduzione temporanea dei privilegi, aumento del livello di logging o apertura automatica di un ticket di sicurezza.

9.5 Considerazioni finali

ZeroTrustHR dimostra come sia possibile costruire un prototipo realistico di architettura Zero Trust combinando strumenti diversi: frontend, backend, MongoDB, Envoy, OPA, Rego, mTLS, CA Service, Splunk, Snort, nftables, Machine Learning e agenti locali.

Il valore del progetto non risiede in un singolo componente, ma nella loro integrazione. Ogni livello contribuisce a ridurre la fiducia implicita: il backend gestisce la logica applicativa, Envoy applica le decisioni, OPA valuta le policy, MongoDB conserva lo stato, mTLS rafforza l'identità del dispositivo, Splunk osserva gli eventi, Snort contribuisce al rischio di rete e il modello di Machine Learning supporta la valutazione comportamentale.

Il progetto mostra anche che la sicurezza moderna non può basarsi su controlli statici. Utenti, dispositivi e contesti cambiano nel tempo; di conseguenza, anche le decisioni di accesso devono poter cambiare. Un utente autorizzato può diventare sospetto, un dispositivo trusted può essere revocato, una richiesta normalmente accettabile può essere negata se il rischio cresce.

In conclusione, l'approccio Zero Trust non consiste semplicemente nell'aggiungere nuovi strumenti di sicurezza, ma nel cambiare il modo in cui viene concessa l'autorizzazione. Ogni richiesta deve essere

verificata, ogni attributo deve essere valutato nel proprio contesto e ogni decisione deve poter essere tracciata. Il progetto ZeroTrustHR rappresenta quindi una dimostrazione concreta di come questi principi possano essere tradotti in un'architettura funzionante, modulare e adattiva.